



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



Welcome to  
**PES University**  
Ring Road Campus, Bengaluru



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



# Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 12

# A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

# A Note on Security

- ➡ In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- ➡ To be clear, **you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network** without the express consent of the owner.
- ➡ In particular, **you will comply with all my instructions when doing the labs.**
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- ➡ Any violation is at **YOUR RISK!**  
**And may result in severe consequences.**

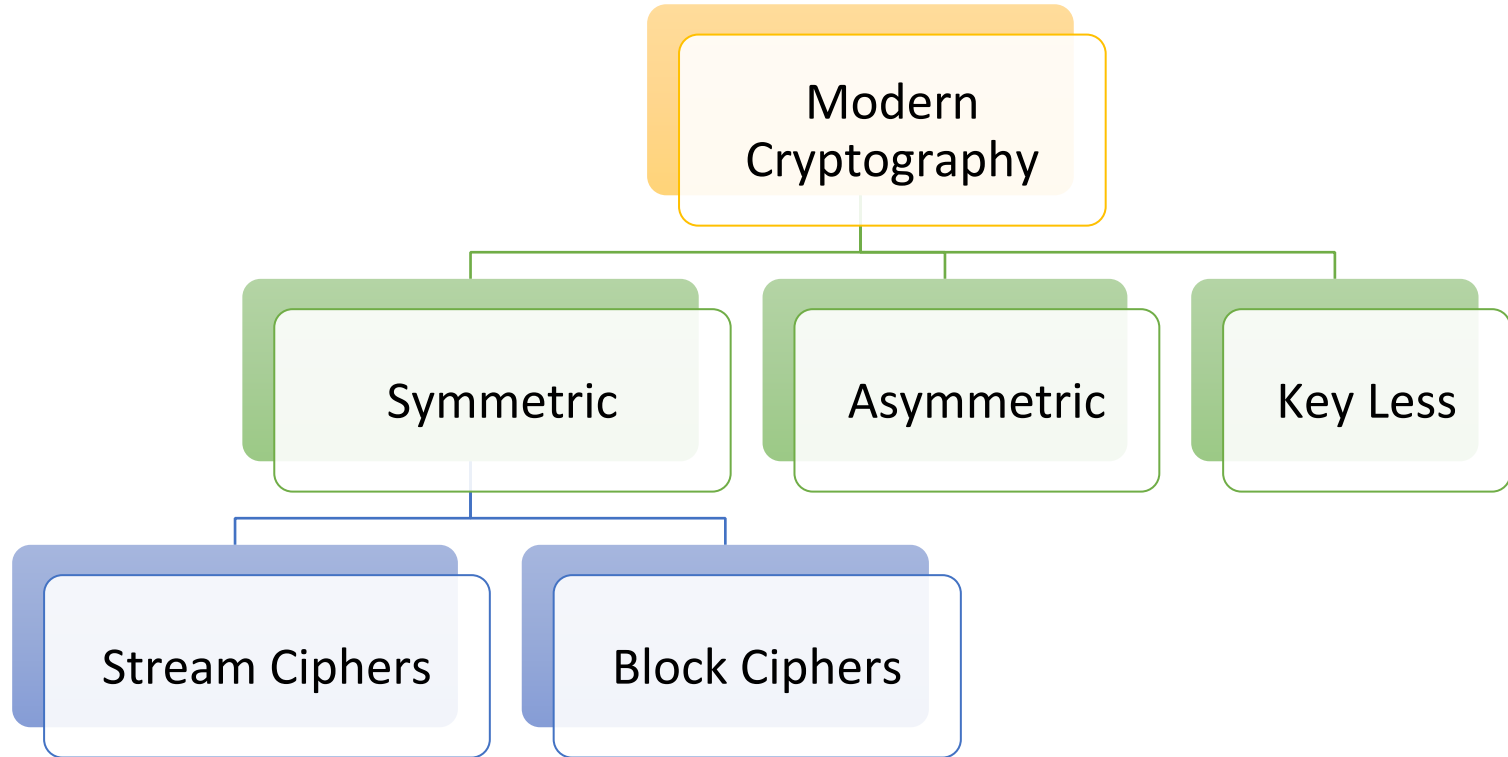


# Symmetric Key Cryptography

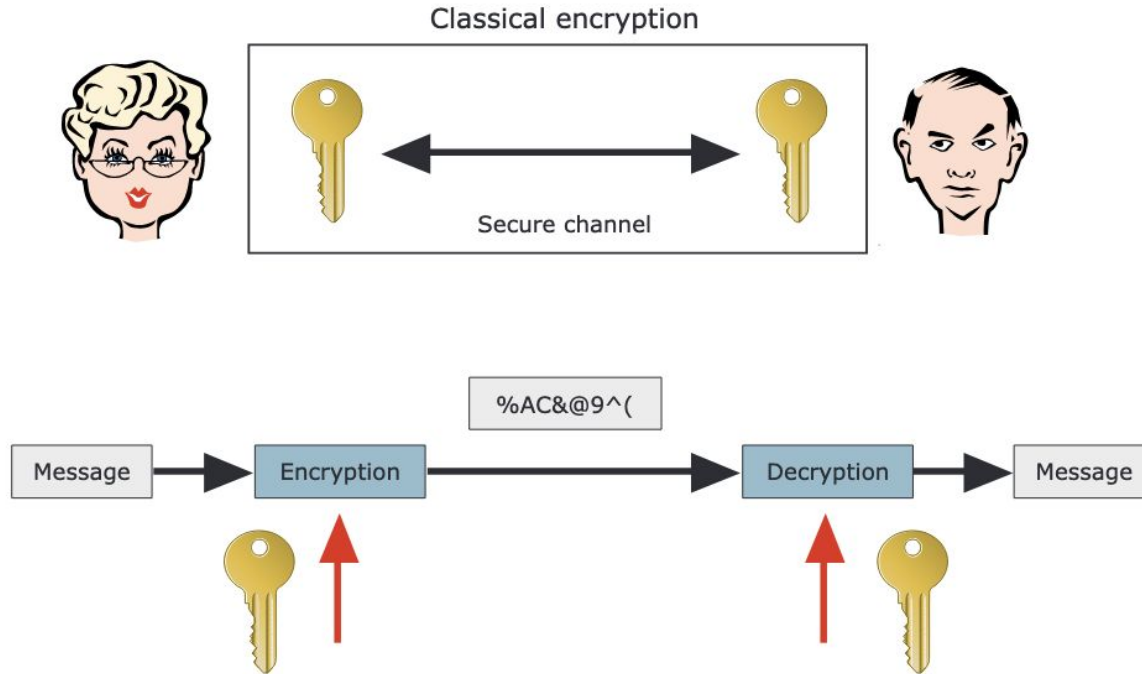
---

## Introduction to Symmetric Key Cryptography

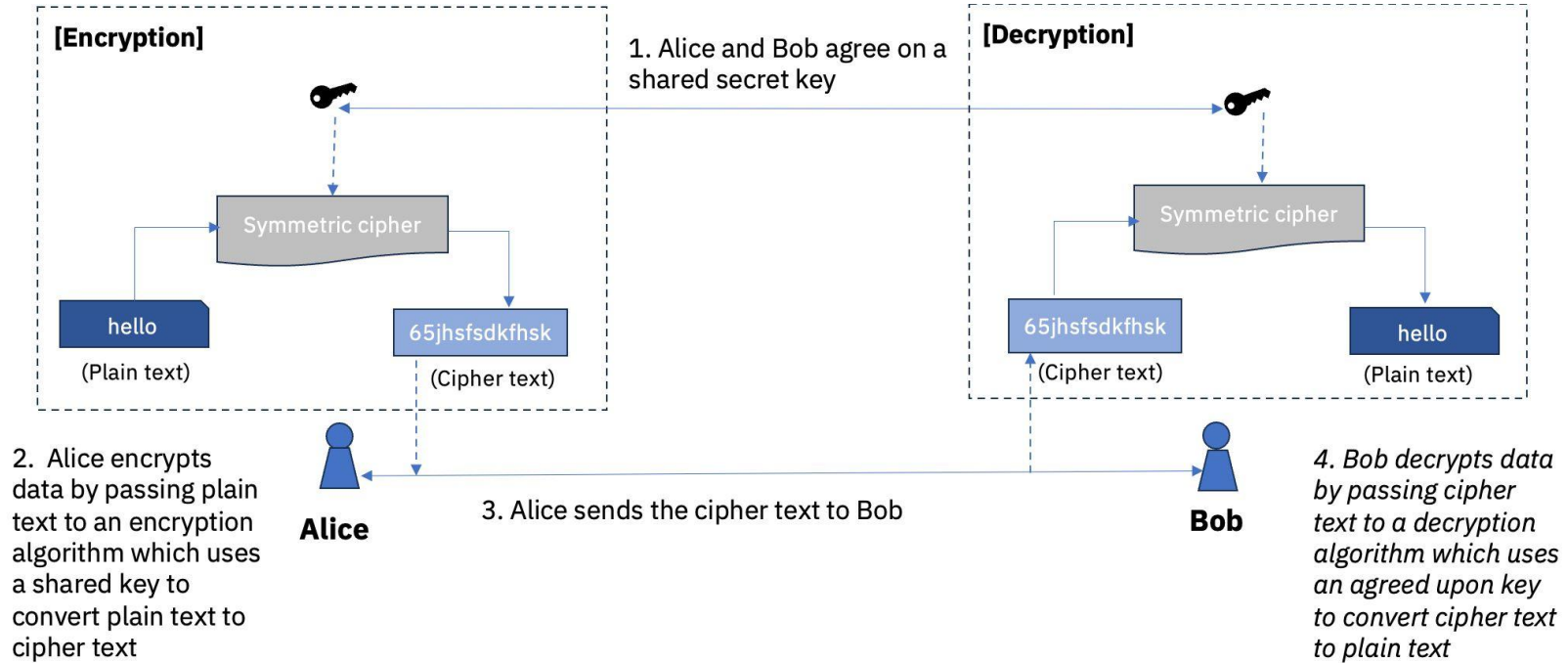
# Classification of Modern Cryptology



# Symmetric Key Cryptography



# Symmetric Key Cryptography





SKC involves:

- An encryption function that converts a given plain text instance to cipher text while utilizing a secret key.
- A decryption function that inverts the operation by converting the ciphertext back to plain text using the same secret key



Implementing a secure symmetric key cryptosystem involves two main tasks:

1. Employing a robust symmetric key encryption algorithm resistant to cryptographic attacks.
2. Ensuring confidentiality in the distribution and management of secret keys.



A symmetric key cryptosystem should ensure the following properties to secure messages-both statically stored data and/or communications over some transmission channel:

- **Confidentiality:** Refers to the property that the information content of encrypted messages is protected from unauthorized access.
- **Integrity:** Refers to the property that any tampering of encrypted messages during storage or transmission can be detected.
- **Authenticity:** Refers to the property that the receiver of a message can verify the identity of the sender and detect impersonation by an unauthorized party.

## **Data encryption and decryption**

- Examples include protecting user credentials, encrypting email messages, and securing financial transactions, among others.

## **Secure communication**

- Symmetric key encryption is much faster and hence can be used for encryption of messages of large size.

## **File and disk encryption**

## **And many more...**



So if there is increased risk why use symmetric key cryptography?

- The number one reason why a symmetric key would be used over an asymmetric key for data encryption is speed.
  - Symmetric key encryption and decryption is done quicker and with less processing power.
- Because it's built in. Many vendors use symmetric key technology within their products to protect sensitive data and operations.
  - The keys always remain local and there is no need for an entity to exchange or move those keys.



# Advantages of symmetric key cryptography

---

- ➡ **Speed and efficiency:** Symmetric key algorithms are more suitable for encrypting large volumes of data or for use in real-time communication scenarios since they are generally faster and less resource intensive than their asymmetric counterparts.
- ➡ **Scalability:** Owing to their relatively low computational overhead, symmetric key algorithms scale well with the number of users and the amount of data being encrypted.
- ➡ **Simplicity:** Symmetric encryption protocols are often easier to implement and understand compared to asymmetric key approaches, making them attractive for developers and users.

## So, what is the risk associated with symmetric key cryptography?

---

- ➡ Symmetric keys never expire so rotation dates must be tracked manually. For compliance purposes this requires extensive logging and auditing of the entire key lifecycle.
- ➡ The two parties involved must somehow securely exchange the key. Once exchanged both parties must maintain equivalent levels of security.
- ➡ Symmetric keys live forever. If a key is compromised there is no way to recall it, all data must be decrypted and re-encrypted using a new key.

# Stream Ciphers

---

# Stream Ciphers in the Field of Cryptology

---

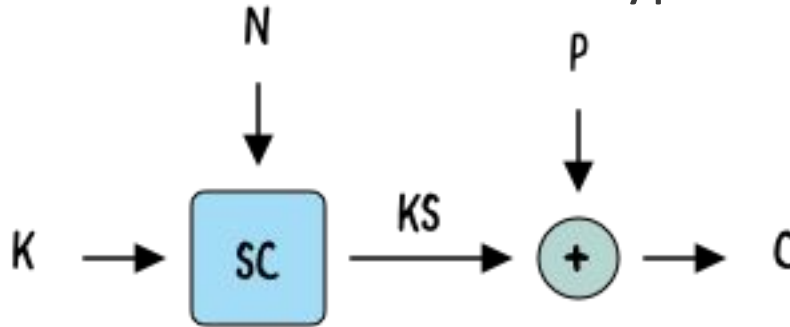
- ➡ A stream cipher is an encryption technique that works bit by bit to transform plain text into code that's unreadable to anyone without the proper key.
- ➡ Stream ciphers are linear, so the same key both encrypts and decrypts messages.
- ➡ While cracking them can be difficult, hackers have managed to do it.
- ➡ For that reason, experts feel stream ciphers aren't safe for widespread use.

# How do stream ciphers work?

- ➡ A stream cipher encrypts a continuous string of binary digits by applying time-varying transformations on plaintext data.
- ➡ Therefore, this type of encryption works bit-by-bit, using **keystreams** to generate ciphertext for arbitrary lengths of plain text messages.
- ➡ The cipher combines a key (128/256 bits) and a nonce digit (64-128 bits) to produce the keystream — a pseudorandom number XORed with the plaintext to produce ciphertext.
- ➡ While the key and the nonce can be reused, the keystream has to be unique for each encryption iteration to ensure security.
- ➡ Stream encryption ciphers achieve this using feedback shift registers to generate a **unique nonce (number used only once)** to create the keystream.

# How do stream ciphers work?

- ➡ Stream ciphers produce a pseudorandom stream of bits called the **keystream**, denoted KS in the diagram.
- ➡ The keystream is XORed to plaintext to encrypt it and then XORed again to the ciphertext to decrypt it.
- ➡ In symbols, a stream cipher generates a keystream  $KS = SC(K, N)$  encrypts as  $C = P \oplus KS$  and decrypts as  $P = C \oplus KS$ .



# Stream ciphers : Making OTP Practical

👉 Stream Ciphers cannot have perfect secrecy.

👉 They need a different Definition for secrecy

👉 Security will depend on specific PRG

- PRG must be unpredictable

👉 We say an  $G:K \rightarrow \{0,1\}^n$  is predictable if

👉  $\Pr[A(G,K) \mid_{1\dots i} = G(K) \mid_{i+1}] \geq \frac{1}{2} + \epsilon$

Def: PRG is **unpredictable** if it is not predictable

$\Rightarrow \forall i$ : no “eff” adv. can predict bit  $(i+1)$  for “non-neg”  $\epsilon$

## Rivest Cipher (RC4)

- RC4/ARC4/ARCFOUR is a fast, simple encryption algorithm developed in 1987 to implement byte-by-byte encryption using 64 or 128 bits long keys.
- RC4 is widely used in Transport Layer Security, Secure Sockets Layer, and the IEEE 802.11 WLAN standard.
- The popular encryption scheme comes in various flavors, including **SPRITZ**, **RC4A**, and **RC4A+**, among others.

## Salsa20

- Salsa20 is an efficient, modern encryption cipher that relies on an expansion function to produce the encryption keystream.
- In addition, Salsa20 depends on a core function that maps the key, a nonce digit, and constant vectors extracted from the expansion function to the keystream using add-rotate-XOR (ARX) operation.

 Other examples of stream ciphers include **PANAMA**, **Scream**, **Rabbit**, **HC-256**, and **Grain**, among others.





Stream ciphers come with plenty of benefits, including:

- **Speed.** This form of encryption is typically faster than others, including block ciphers.
- **Low complexity.** It's easy to incorporate stream ciphers into modern programs, and developers don't need complex hardware to make it happen.
- **Serial nature.** Some companies deal with messages written in a trickle. With their bit-by-bit processing, stream ciphers allow them to send information when it's ready rather than waiting for everything to be done.
- **Ease of use.** Stream ciphers are symmetrical encryption tools, so companies aren't forced to bother with public and private keys. And mathematical concepts that underlie modern stream ciphers allow computers to determine the proper decryption key to use.

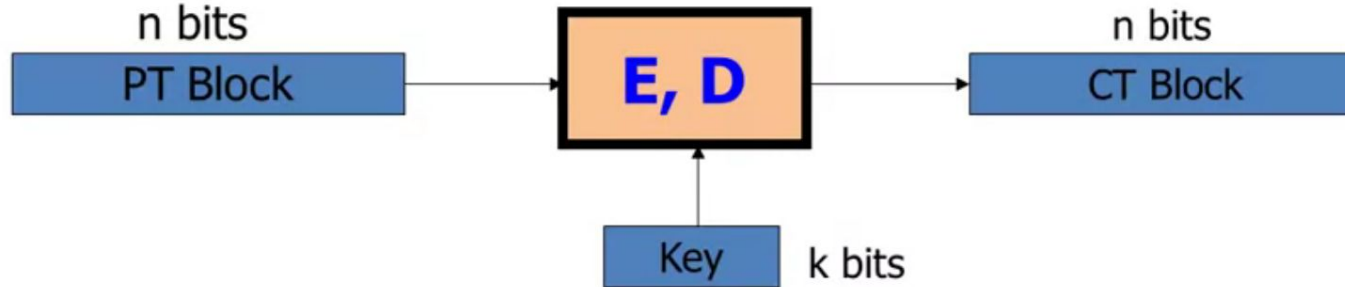
# Are stream ciphers safe from hackers?

- ➡ As we learned during WWII, even sophisticated stream ciphers can be cracked with the proper amount of creativity and persistence.
- ➡ No encryption method is 100 percent safe from people determined to get inside.
- ➡ Read through hacker forums, and you'll learn that most of them consider stream ciphers extremely easy to reverse
- ➡ The best way to ensure security with stream ciphers is to use keys just once.
- ➡ And each key should be unrelated to any other you've used.
- ➡ That way, even if attackers crack one key, the rest of your system's security will remain intact.

# BLOCK CIPHERS

---

# What is a block Cipher?



Canonical examples:

1. 3DES:  $n = 64$  bits,  $k = 168$  bits
2. AES:  $n = 128$  bits,  $k = 128, 192, 256$  bits

- ➡ A block cipher is an encryption method that applies a deterministic algorithm along with a symmetric key to encrypt a block of text, rather than encrypting one bit at a time as in stream ciphers.
- ➡ For example, a common block cipher, AES, encrypts 128 bit blocks with a key of predetermined length: 128, 192, or 256 bits.
- ➡ Block ciphers are pseudorandom permutation (PRP) families that operate on the fixed size block of bits.
- ➡ PRPs are functions that cannot be differentiated from completely random permutations and thus, are considered reliable, until proven unreliable.

# What is the advantage of a block cipher?

- ➡ A block cipher has high diffusion (information from one plaintext symbol is spread into several cipher-text symbols). It is also fairly difficult for an attacker to insert symbols without detection, because they can't easily insert them into the middle of a block.
- ➡ However, it is slower than a stream cipher (an entire block needs to be transmitted before encryption/decryption can happen) and if an error does occur, it can propagate throughout the block, corrupting the entire section.

# Characteristics of Block ciphers

- ☞ **Confusion:** Confusion is the characteristic whereby each bit in the ciphertext depends on multiple bits of the secret key. It ensures that a small change in the secret key modifies almost all the bits of the ciphertext, obscuring the relationship between the ciphertext and the secret key.
- ☞ **Diffusion:** Diffusion is the characteristic whereby flipping a single bit in the plain text should modify roughly half the bits in the ciphertext and vice versa. Diffusion hides statistical relationships between the plain text and ciphertext. Ciphers with adequate diffusion satisfy the so-called avalanche criteria of cryptography.



There are few considerations to be made while selecting a block size. They are as follows:

- **Avoid small block size:** The block size to be used should not be too small in length. If an attacker discovers some ciphertext block corresponding to a plain text block, then it's possible for an attacker to launch Dictionary Attack by building a dictionary of ciphertext and plain text pairs. The large block size makes this attack difficult to perform.
- **Avoid blocks of bigger size:** With a bigger block size, it becomes very difficult to operate the cipher.
- **8-bit multiples:** It is advisable to consider block size of multiple of 8. 8 bits and its multiple are easy to process and don't need padding.

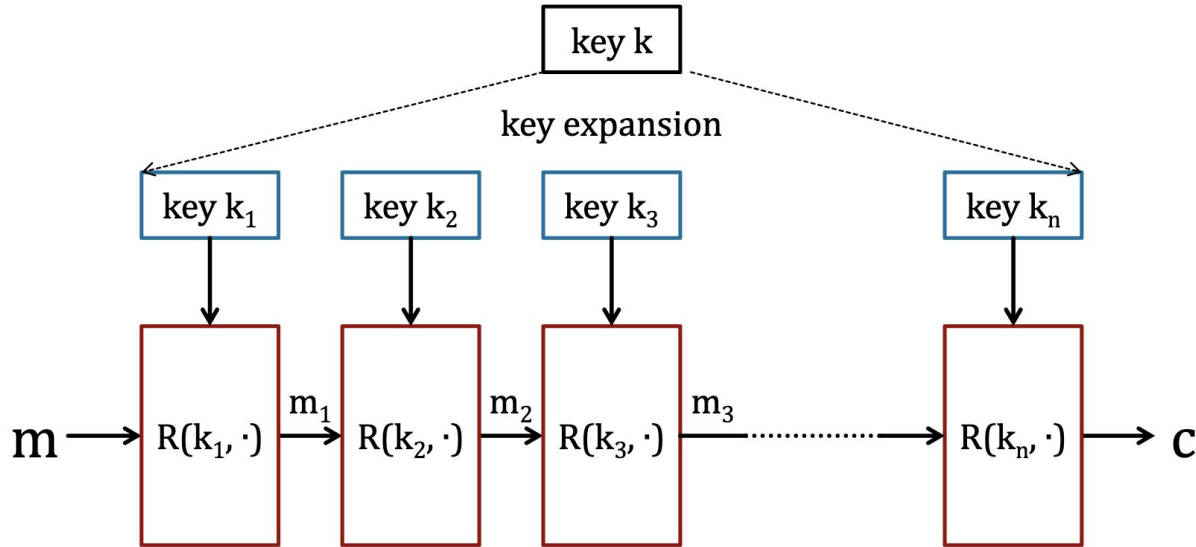




Block ciphers have many schemes to operate. Some of the widely used and popular ones are as follows:

- **DES:** Digital Encryption Standard. It's broken and no more used.
- **Triple DES:** Repeated DES. Much stronger algos are available than this.
- **AES:** Advanced Encryption Standard.
- **Serpent:** Has block size of 128 bits and keys of length 128,196 or 256 bits. Slower but very secure than other block ciphers.
- **TwoFish:** With a block size of 128 bits, it is based on an earlier block cipher called Blowfish which uses a block of 64 bits

# Block ciphers built by iteration



$R(k, m)$  is called a round function

Ex: 3DES ( $n=48$ ), AES128 ( $n=10$ )

- **Block size**

- increasing size improves security, but slows cipher.

- **Key size**

- increasing size improves security, makes exhaustive key searching harder, but may slow cipher

- **Number of rounds**

- increasing number improves security, but slows cipher

- **Subkey generation**

- greater complexity can make analysis harder, but slows cipher

- **Round function**

- greater complexity can make analysis harder, but slows cipher

# Difference between Block Cipher and Stream Cipher

| S.NO | Block Cipher                                                                                                       | Stream Cipher                                                                                               |
|------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 1.   | Block Cipher Converts the plain text into cipher text by taking plain text's block at a time.                      | Stream Cipher Converts the plain text into cipher text by taking 1 byte of plain text at a time.            |
| 2.   | Block cipher uses either 64 bits or more than 64 bits.                                                             | While stream cipher uses 8 bits.                                                                            |
| 3.   | The complexity of block cipher is simple.                                                                          | While stream cipher is more complex.                                                                        |
| 4.   | Block cipher Uses confusion as well as diffusion.                                                                  | While stream cipher uses only confusion.                                                                    |
| 5.   | In block cipher, reverse encrypted text is hard.                                                                   | While in-stream cipher, reverse encrypted text is easy.                                                     |
| 6.   | The algorithm modes which are used in block cipher are ECB (Electronic Code Book) and CBC (Cipher Block Chaining). | The algorithm modes which are used in stream cipher are CFB (Cipher Feedback) and OFB (Output Feedback).    |
| 7.   | Block cipher works on transposition techniques like rail-fence technique, columnar transposition technique, etc.   | While stream cipher works on substitution techniques like Caesar cipher, polygram substitution cipher, etc. |
| 8.   | Block cipher is slow as compared to a stream cipher.                                                               | While stream cipher is fast in comparison to block cipher.                                                  |

# Question 1

---

Which of the following is a block cipher?

- a) RC4
- b) AES**
- c) A5/1
- d) Salsa20

## Question 2

---

**A block cipher operates on:**

- a) One bit at a time
- b) Fixed-size group of bits**
- c) Variable-size group of bits
- d) Character by character

## Question 3

---

**A stream cipher operates on:**

- a) One block at a time
- b) A stream of characters or bits**
- c) Only ASCII values
- d) 16 bytes at a time

# Feistel Cipher

---

Used by DES



# Types of Symmetric key cipher

---



## Stream cipher:

- algorithm operates on individual bits (or bytes) one at a time
- Example RC4 cipher system



## Block cipher:

- operates on fixed-length groups of bits called blocks
- Example DES, Triple DES and AES

# What is Feistel Cipher?

- ➡ Feistel cipher is a structure proposed by a **Horst Feistel** which was considered while developing many symmetric block ciphers.
- ➡ The structure proposed by Feistel is based on the **Shannon structure** which was proposed in 1945.
- ➡ The Shannon structure shows the implementation of confusion and diffusion alternately.



## Confusion

- Making the relationship between the encryption and the ciphertext as complex as possible.
- The relationship between CT and PT is obscured.
- Given CT, no information about PT, Key, Encryption Algorithm etc.,

Example: Substitution



## Diffusion

- Making each Plaintext bit affect as many ciphertext bits as possible.
- 1 bit change in PT, significant effect on CT.

Example: Transposition or Permutation

❖ **Feistel Cipher**: the fundamental building block of DES

👉 designed by IBM.

❖ DES was adopted as a US federal standard for commercial encryption in 1975.

❖ Design requirements:

- must provide high level of security (commercial standard)
- Security must not depend on secrecy of algorithm

👉 (**Kerckhoff's principle**)

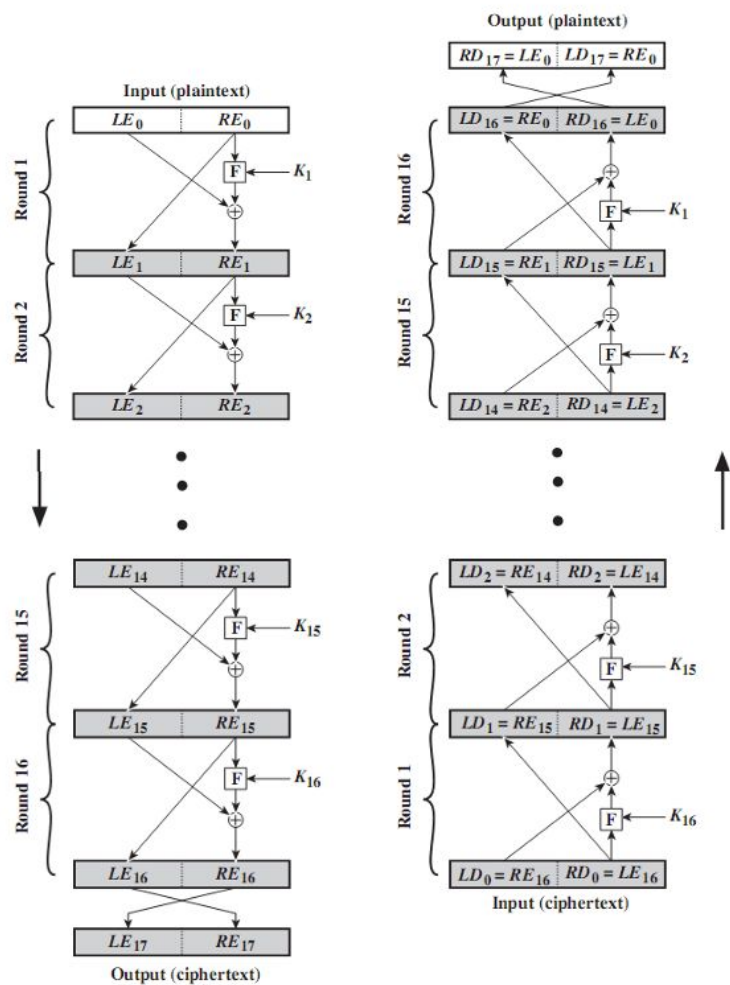
- Must be easily and economically implemented

- ➡ most symmetric block ciphers are based on a **Feistel Cipher Structure**
- ➡ needed since must be able to **decrypt** ciphertext to recover messages efficiently
- ➡ block ciphers look like an extremely large substitution
- ➡ would need table of  $2^{64}$  entries for a 64-bit block
- ➡ instead create from smaller building blocks
- ➡ using idea of a product cipher

- ➡ in 1949 Claude Shannon introduced idea of substitution-permutation (S-P) networks
  - modern substitution-transposition product cipher
- ➡ these form the basis of modern block ciphers
- ➡ S-P networks are based on the two primitive cryptographic operations :
  - *substitution* (S-box)
  - *permutation* (P-box)
- ➡ provide *confusion* and *diffusion* of message

- ➡ Horst Feistel devised the **feistel cipher**
  - based on concept of invertible product cipher
- ➡ partitions input block into two halves
  - process through multiple rounds which
  - perform a substitution on left data half
  - based on round function of right half & subkey
  - then have permutation swapping halves
- ➡ implements Shannon's substitution-permutation network concept





# Feistel Cipher Structure(encryption):

- ➡ The inputs to the encryption algorithm are a plaintext block of length  $2w$  bits and a key  $K$ .
- ➡ The plaintext block is divided into two halves,  $L$ (left) and  $R$ (right) .
- ➡ The two halves of the data pass through  $n$  rounds of processing and then combine to produce the ciphertext block.
- ➡ Each round  $i$  has as inputs  $L_{i-1}$  and  $R_{i-1}$ , derived from the previous round, as well as a subkey  $K_i$ , derived from the overall  $K$ .
- ➡ In general, the subkeys  $K_i$  are different from  $K$  and from each other.

# Feistel Decryption Algorithm:

---

- ➡ Feistel Cipher structure does not have a different algorithm for decryption.
- ➡ The encryption and decryption function proposed by Feistel cipher are same with some rules which are as follows:
  - The input to the decryption algorithm is a cipher text block produced by the encryption algorithm.
  - The sequence of subkeys used in encryption are reversed.

# Feistel Decryption Algorithm

➡ The output of the first round of decryption is equal to 32 bit swap of the input to the sixteenth round of the encryption process.

➡ Consider the Encryption process :

$$LE_{16} = RE_{15}$$

$$RE_{16} = LE_{15} \times F(RE_{15}, K_{16})$$

On the decryption side,

$$LD_1 = RD_0 = LE_{16} = RE_{15}$$

$$RD_1 = LD_0 \times F(RD_0, K_{16})$$

$$= RE_{16} \times F(RE_{15}, K_{16})$$

$$= [LE_{15} \times F(RE_{15}, K_{16})] \times F(RE_{15}, K_{16})$$

# Design Features

---

- ➡ Block Size
- ➡ Key Size
- ➡ Number of Rounds
- ➡ Subkey Generation Algorithm
- ➡ Round Function
- ➡ Ease of Analysis
- ➡ Fast Software encryption/decryption

# Key Takeaways

- ➡ Feistel cipher was based on the structure proposed by **Shannon**.
- ➡ Shannon structure has an alternate implementation of **diffusion** and **confusion** to obtain cipher text block.
- ➡ Feistel cipher structure has alternate application **substitution** and **permutation** on plain text block to obtain cipher text block.
- ➡ Feistel block cipher operates on each block independently.
- ➡ The encryption and decryption algorithm in Feistel cipher is the same.
- ➡ The **key** used for encryption and decryption is the **same** but the sequence of application of subkey is reversed.
- ➡ During encryption a plain text block undergoes multiple rounds. But the function performed in each round is same.
- ➡ Generally, 16 rounds are performed in Feistel cipher.

# Question 1

---

Why is the Feistel structure particularly efficient for decryption compared to other block cipher designs?

- A) It uses fewer rounds for decryption than for encryption
- B) It only requires reversing the subkey order while keeping the same round structure**
- C) It replaces permutation with substitution during decryption
- D) It avoids the need for confusion and diffusion

## Question 2

---

If a single plaintext bit is flipped in a Feistel cipher, which cryptographic principle ensures that many ciphertext bits are affected after several rounds?

- A) Confusion
- B) Key Expansion
- C) Diffusion**
- D) Transposition



## Question 3

Claude Shannon's concept of substitution-permutation networks inspired Feistel's design. What practical problem did the Feistel cipher solve that Shannon's original S-P idea did not?

- A) Ensuring resistance against brute-force attacks
- B) Avoiding the need for key scheduling
- C) Making decryption computationally efficient without inverting the round function**
- D) Eliminating the use of permutations

# Thank you!

## Follow us



isfcr.pesu



[www.isfcr.pes.edu](http://www.isfcr.pes.edu)



ISFCR



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience





PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



Welcome to  
**PES University**  
Ring Road Campus, Bengaluru



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



# Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 14

# A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

# A Note on Security

- ➡ In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- ➡ To be clear, **you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network** without the express consent of the owner.
- ➡ In particular, **you will comply with all my instructions when doing the labs.**
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- ➡ Any violation is at **YOUR RISK!**  
**And may result in severe consequences.**

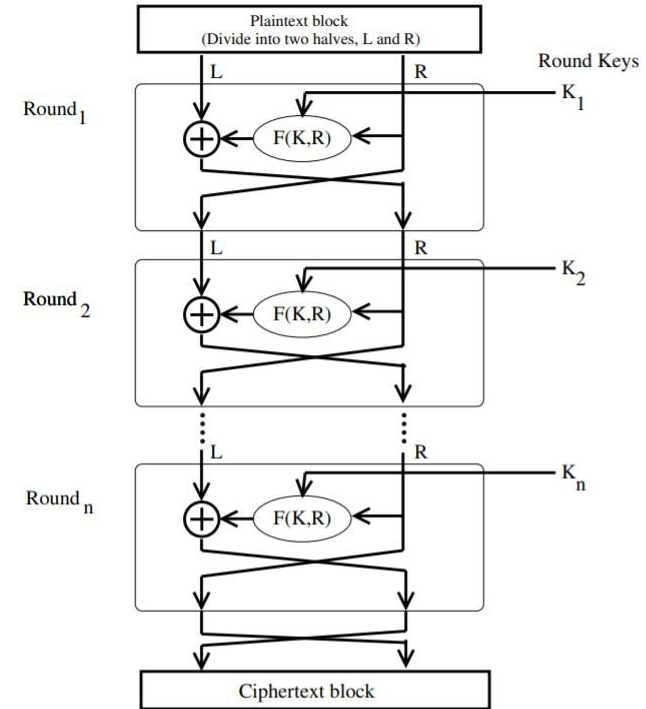
## Definition

- ➡ A **Feistel Network** divides the plaintext block into two equal halves: **Left (L)** and **Right (R)**.
- ➡ Each round uses a **round key** and a **round function F**.
- ➡ The right half is passed through the round function
- ➡ The output of the function is combined with the left half using **XOR**.
- ➡ The two halves are then **swapped** for the next round.
- ➡ The same process is repeated for several rounds to produce the ciphertext.
- ➡ **DES** is a well-known example of a Feistel-based block cipher

# Feistel Round

$$L_{i+1} = R_i$$

$$R_{i+1} = L_i \oplus F(R_i, K_i)$$





# Example

Imagine a message is divided into two teams:

LEFT = "ABCD"

RIGHT = "WXYZ"

**The RIGHT side enters the round function:**

$F(WXYZ, K_1)$

**Its output is XORed with LEFT, and then the two sides swap.**

**So each round follows:**

Split  $\rightarrow$  Function  $\rightarrow$  XOR  $\rightarrow$  Swap  $\rightarrow$  Repeat

This simple structure, repeated many times, creates a highly complex ciphertext.

# Example

Imagine a message is divided into two teams:

LEFT = "ABCD"

RIGHT = "WXYZ"

**The RIGHT side enters the round function:**

$F(WXYZ, K_1)$

**Its output is XORed with LEFT, and then the two sides swap.**

**So each round follows:**

Split  $\rightarrow$  Function  $\rightarrow$  XOR  $\rightarrow$  Swap  $\rightarrow$  Repeat

This simple structure, repeated many times, creates a highly complex ciphertext.

# Substitution-Permutation Network (SPN)

## Definition

- ☞ **SPN is a block-cipher design structure** used to transform plaintext into ciphertext.
- ☞ It repeatedly applies **substitution** and **permutation/mixing** operations.
- ☞ **Substitution** changes the actual values of the data.
- ☞ **Permutation/Mixing** changes how the data is distributed.
- ☞ Substitution mainly provides **confusion**.
- ☞ Permutation/mixing mainly provides **diffusion**.
- ☞ Multiple rounds make it difficult for an attacker to relate plaintext, key, and

# Example

---

Suppose the input is:

ABCD

After substitution:

X7P2

After permutation:

7X2P

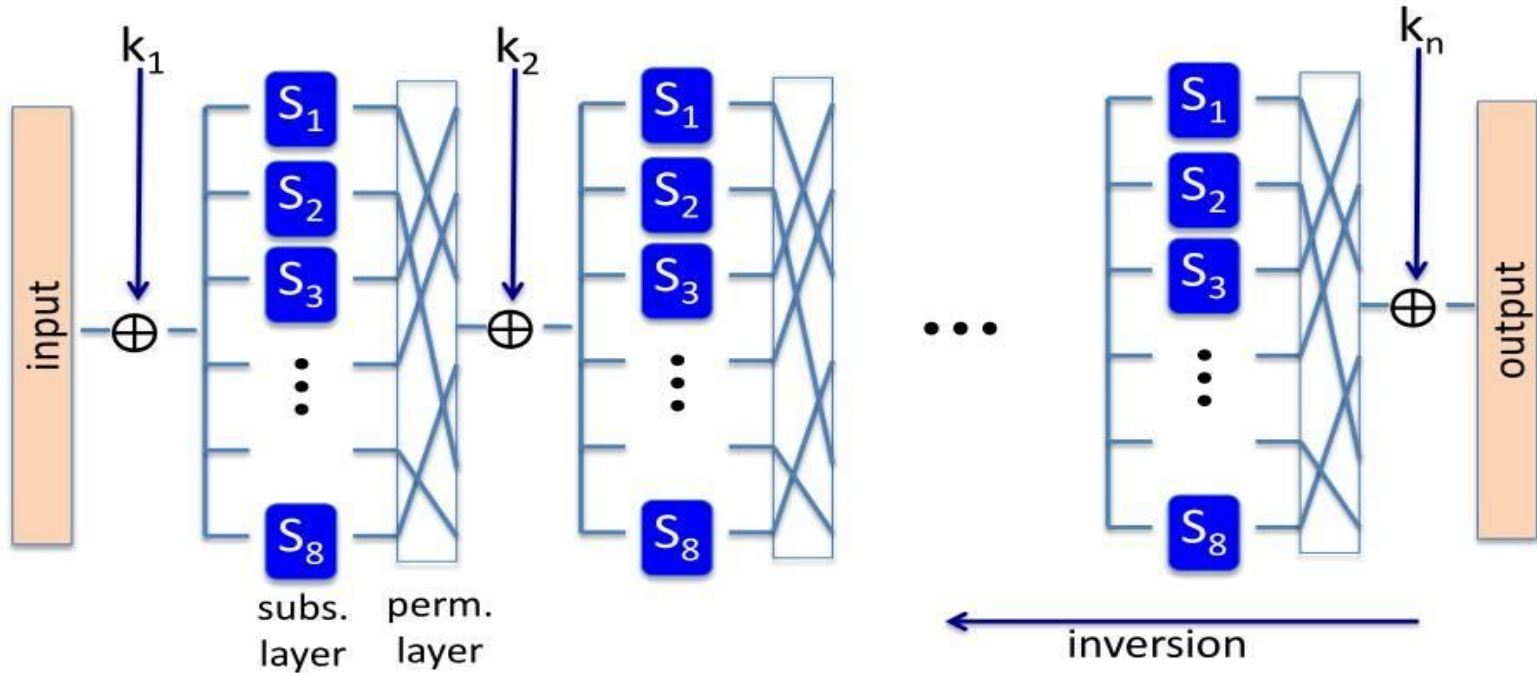
The process is repeated for several rounds until the final ciphertext is produced.

## Real Example

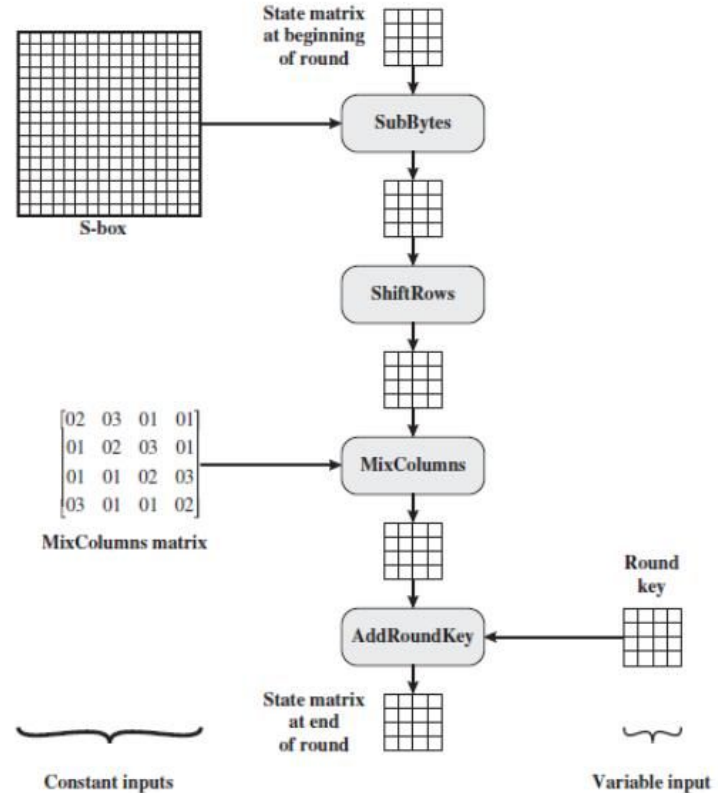
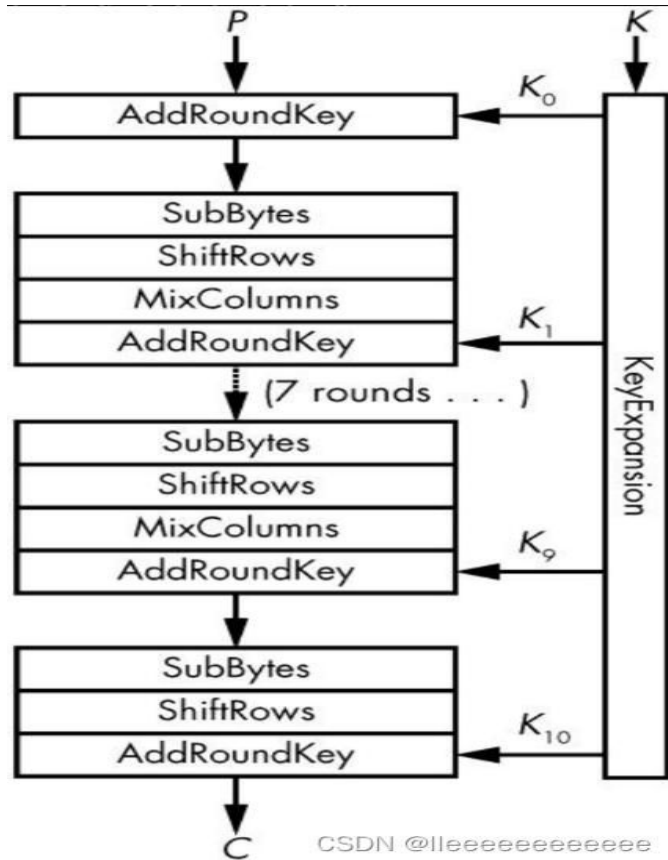
**AES** follows an SPN-style design using transformations

such as **SubBytes**, **ShiftRows**, **MixColumns**, and **AddRoundKey**.

## AES is a Subs-Perm network (not Feistel)



# Example



## Definition

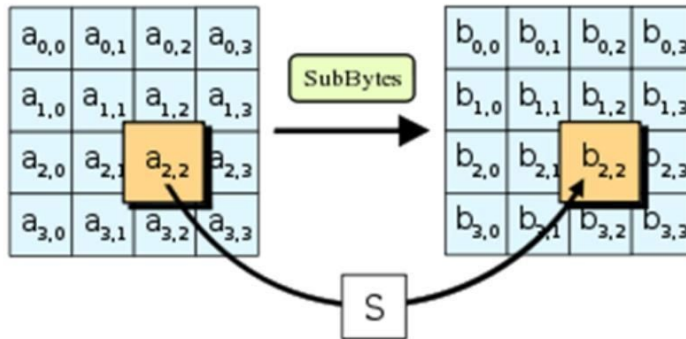
- ➡ Substitution replaces one input value with another value.
- ➡ It is performed using an **S-Box (Substitution Box)**.
- ➡ The same input produces a defined output.
- ➡ It introduces **non-linearity** into the cipher.
- ➡ It makes the relationship between the **key and ciphertext** difficult to analyze.
- ➡ Substitution contributes mainly to **confusion**.

# Example

In **AES**, the **SubBytes** operation performs byte substitution using the AES S-Box.

## AES Advanced Encryption Standard

### 1. The SubByte Step



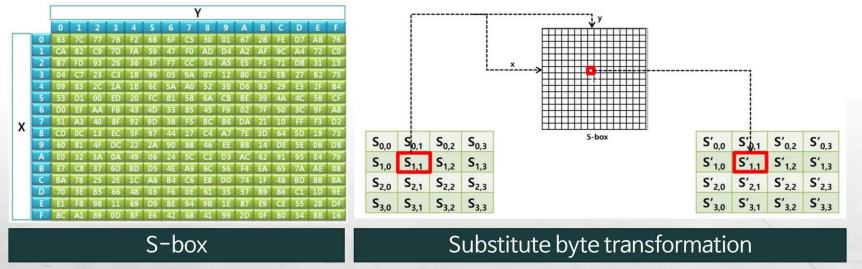
### Substitute Bytes

- Called SubBytes.
- S-box is 16\*16 array.
- Byte in state substitute another byte in S-box.

|    |    |    |    |
|----|----|----|----|
| EA | 04 | 65 | 85 |
| 83 | 45 | 5D | 96 |
| 5C | 33 | 98 | B0 |
| F0 | 2D | AD | C5 |

→

|    |    |    |    |
|----|----|----|----|
| 87 | F2 | 4D | 97 |
| EC | 6E | 4C | 90 |
| 4A | C3 | 46 | E7 |
| 8C | D8 | 95 | A6 |



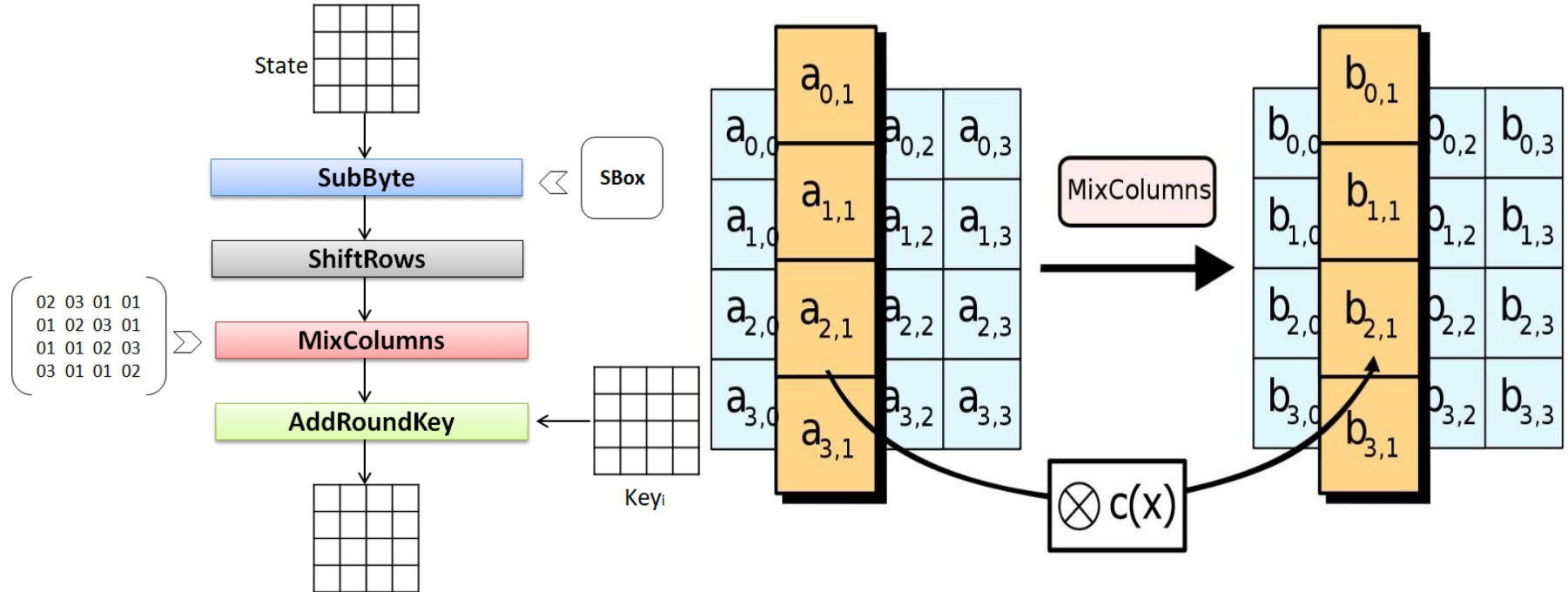


## Definition

- ➡ Permutation changes the **position of data elements**.
- ➡ Mixing spreads the influence of one input across multiple output positions.
- ➡ It prevents patterns from remaining visible in the ciphertext.
- ➡ It provides **diffusion**.
- ➡ A small change in plaintext can eventually affect many ciphertext bits.
- ➡ It works together with substitution to strengthen the cipher.

# Example

In AES, **ShiftRows** changes positions, while **MixColumns** mixes bytes within each column.

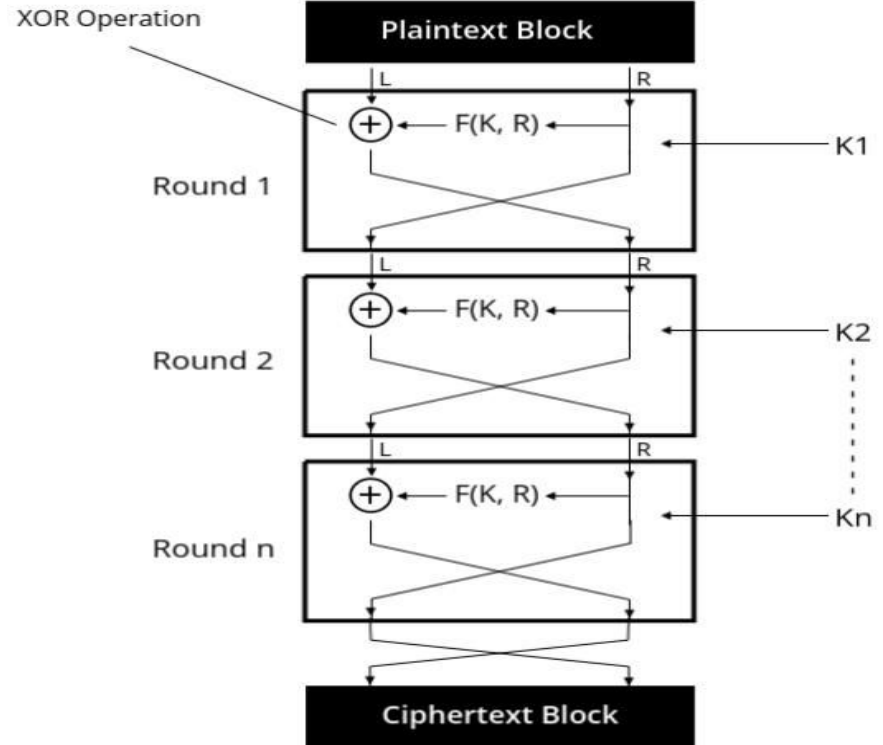
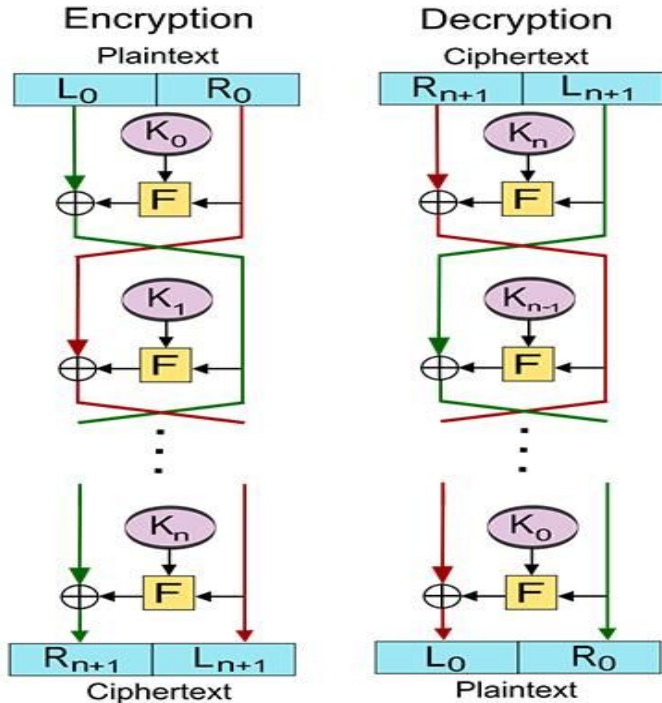


## Definition

- ➡ A **Feistel Network** is a block-cipher construction divided into two halves.
- ➡ The input is divided into **Left (L)** and **Right (R)**.
- ➡ The right half is passed through a **round function**.
- ➡ The result is combined with the left half using **XOR**.
- ➡ The two halves are then swapped.
- ➡ This process is repeated for multiple rounds.
- ➡ **DES** is the classic example of a Feistel cipher.

# Example

Even though the structure looks simple, repeating the process many times produces a highly complex ciphertext.



## Main Difference

- 👉 **Feistel Network** divides the block into **two halves**.
- 👉 **SPN** processes the block through **substitution and permutation/mixing layers**.
- 👉 Feistel uses a **round function + XOR + swapping**.
- 👉 SPN uses **S-Boxes + permutation/mixing + round keys**.
- 👉 Feistel round functions **do not have to be reversible**.
- 👉 SPN transformations generally need to support reversible decryption.
- 👉 **DES → Feistel Network**
- 👉 **AES → SPN-style design**

# Example

---

Think of them as two ways of scrambling a message:

**Feistel:**

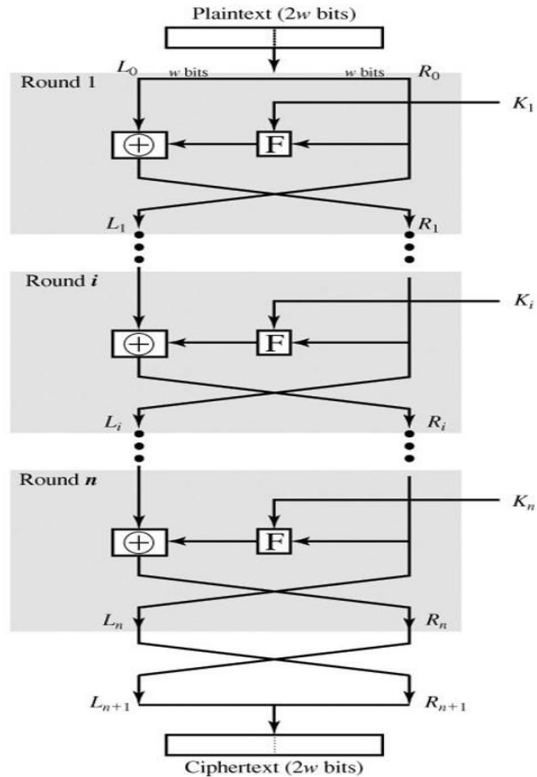
"Split the message into two teams, process one team, mix it with the other, and swap them."

**SPN:**

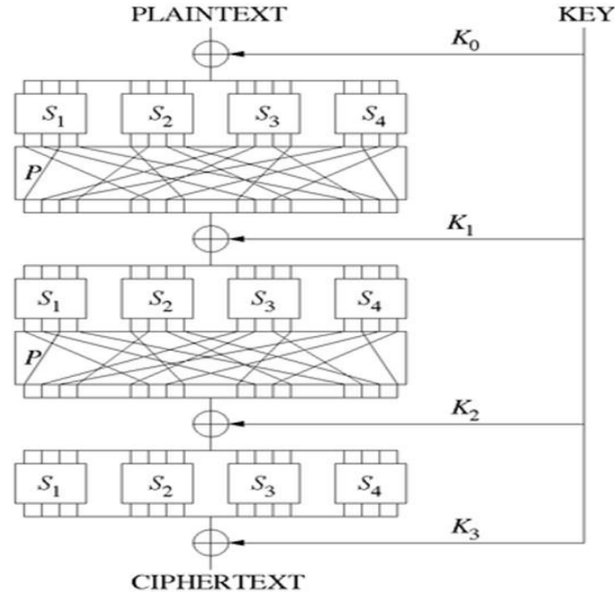
"Replace the values, rearrange/mix them, and repeat the process."

# Example

## FEISTEL



## SPN



# Why Multiple Rounds?

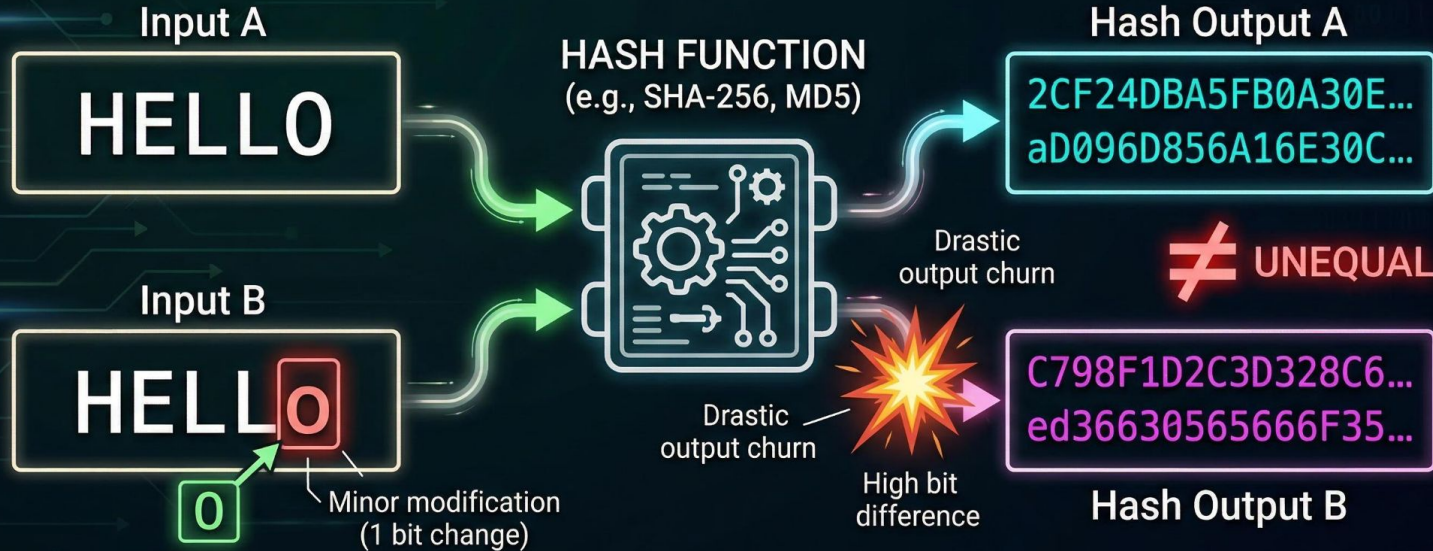
---

- ➡ One round is not enough to provide strong security.
- ➡ Multiple rounds repeatedly apply confusion and diffusion.
- ➡ Each round increases the complexity of the ciphertext.
- ➡ A small change in plaintext can spread through many bits.
- ➡ A small change in the key can also produce a significantly different ciphertext.
- ➡ This contributes to the **avalanche effect**.



## Avalanche Effect

Small input change → completely different output



\* The avalanche effect ensures predictability of the hash output based on the input is impossible, enhancing security.

**Q1. If the round function in a Feistel network is not reversible, can the entire cipher still be decrypted?**

**Answer:**

Yes. The **Feistel structure itself is reversible**, even if the round function is not. This is one of the major advantages of the Feistel design.

**Q2. AES and DES use different block-cipher structures. Which one uses SPN and which one uses Feistel?**

**Answer:**

**AES uses an SPN-style structure, while DES uses a Feistel network.**

**Q3. Why does an SPN use both substitution and permutation/mixing?**

**Answer:**

**Substitution provides confusion, while permutation/mixing provides diffusion.** Together, they make the relationship between plaintext, key, and ciphertext difficult to analyze.

# Thank you!

## Follow us



isfcr.pesu



[www.isfcr.pes.edu](http://www.isfcr.pes.edu)



ISFCR



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience





PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



Welcome to  
**PES University**  
Ring Road Campus, Bengaluru



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



# Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 15

# A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

# A Note on Security

- ➡ In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- ➡ To be clear, **you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network** without the express consent of the owner.
- ➡ In particular, **you will comply with all my instructions when doing the labs.**
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- ➡ Any violation is at **YOUR RISK!**  
**And may result in severe consequences.**

# DES

---

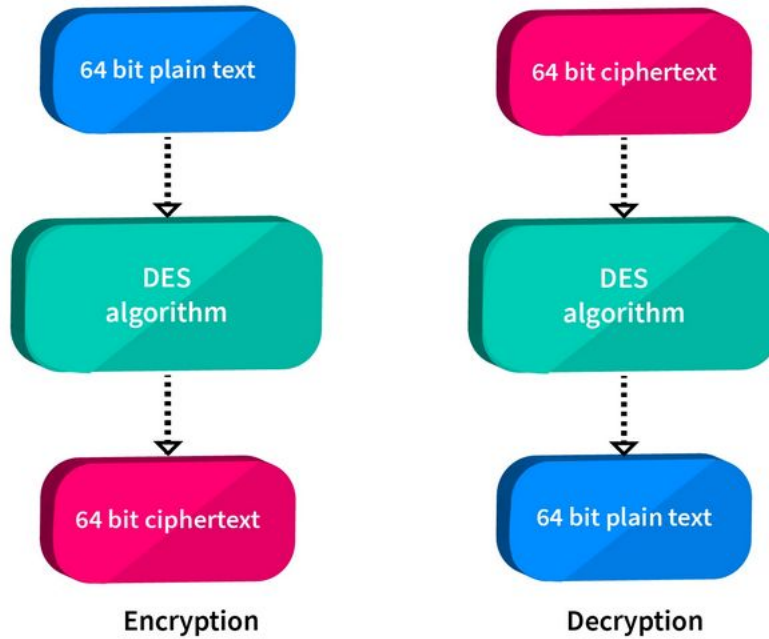
## DATA ENCRYPTION STANDARDS



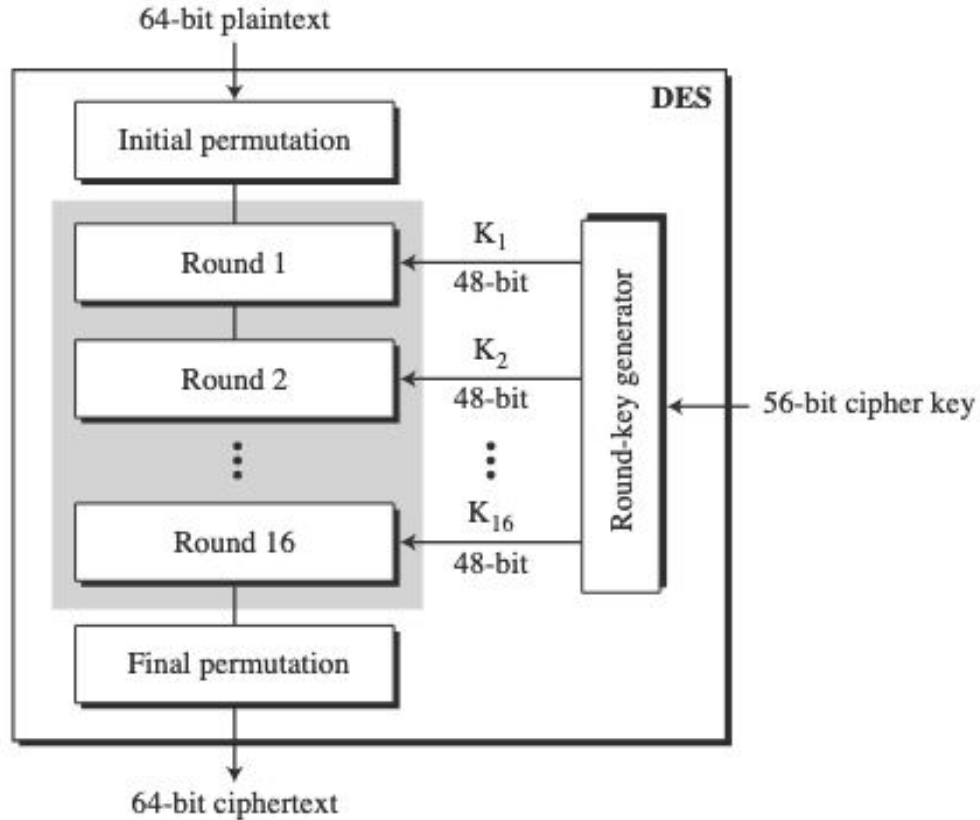
# History of DES Algorithm

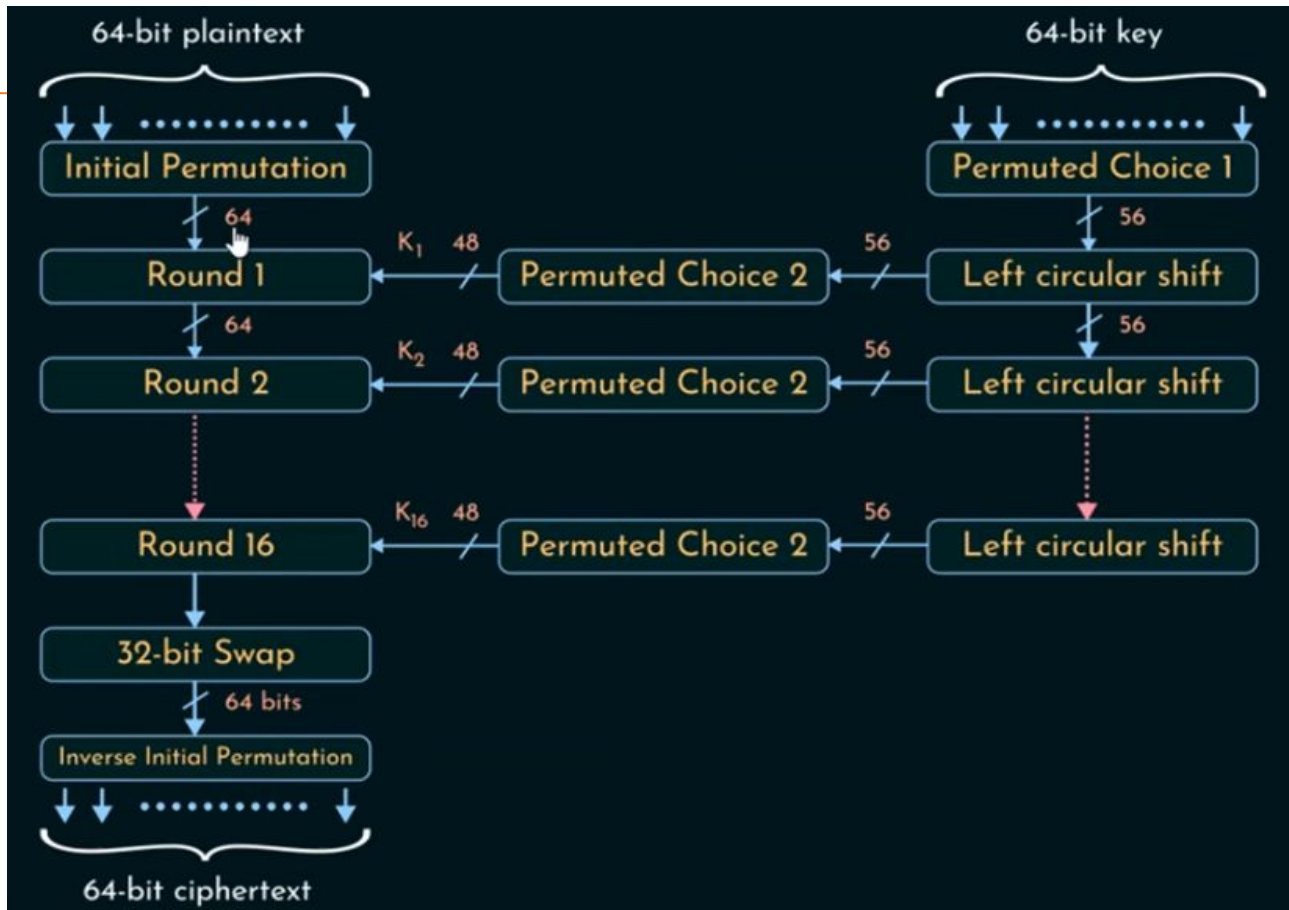
- ➡ DES is based on the Feistel block cipher, called LUCIFER, developed in 1971 by IBM cryptography researcher Horst Feistel.
- ➡ DES uses 16 rounds of the Feistel structure, using a different key for each round.
- ➡ DES became the approved federal encryption standard in November 1976 and was subsequently reaffirmed as the standard in 1983, 1988, and 1999.
- ➡ DES's dominance came to an end in 2002, when the Advanced Encryption Standard (AES) replaced the DES encryption algorithm as the accepted standard, following a public competition to find a replacement.

- ➡ The DES (Data Encryption Standard) algorithm is a symmetric-key block cipher created in the early 1970s by an IBM team and adopted by the National Institute of Standards and Technology (NIST).
- ➡ The algorithm takes the plain text in 64-bit blocks and converts them into ciphertext using 48-bit keys.
- ➡ Since it's a symmetric-key algorithm, it employs the same key in both encrypting and decrypting the data.



# General Structure of DES





# DES Algorithm Steps – Broad Overview



- The algorithm process breaks down into the following steps:
1. The process begins with the 64-bit plain text block getting handed over to an **initial permutation (IP) function**.
  2. The initial permutation (IP) is then performed on the plain text.
  3. Next, the initial permutation (IP) creates two halves of the permuted block, referred to as Left Plain Text (LPT) and Right Plain Text (RPT).
  4. Each LPT and RPT goes through **16 rounds** of the encryption process.
  5. Finally, the LPT and RPT are rejoined, and a **Final Permutation (FP)** is performed on the newly combined block.
  6. The result of this process produces the desired 64-bit ciphertext.

# Initial and Final Permutations

- ➡ The initial permutation, which seems to have no cryptographic significance.
- ➡ Each of these permutations takes a 64-bit input and permutes them according to a predefined rule.
- ➡ This means that the 58<sup>th</sup> bit of  $m$  becomes the 1<sup>st</sup> bit of  $m_0$ , the 50<sup>th</sup> bit of  $m$  becomes the 2<sup>nd</sup> bit of  $m_0$ , etc.
- ➡ These permutations are keyless straight permutations that are the inverse of each other

# EXAMPLE OF IP / FP

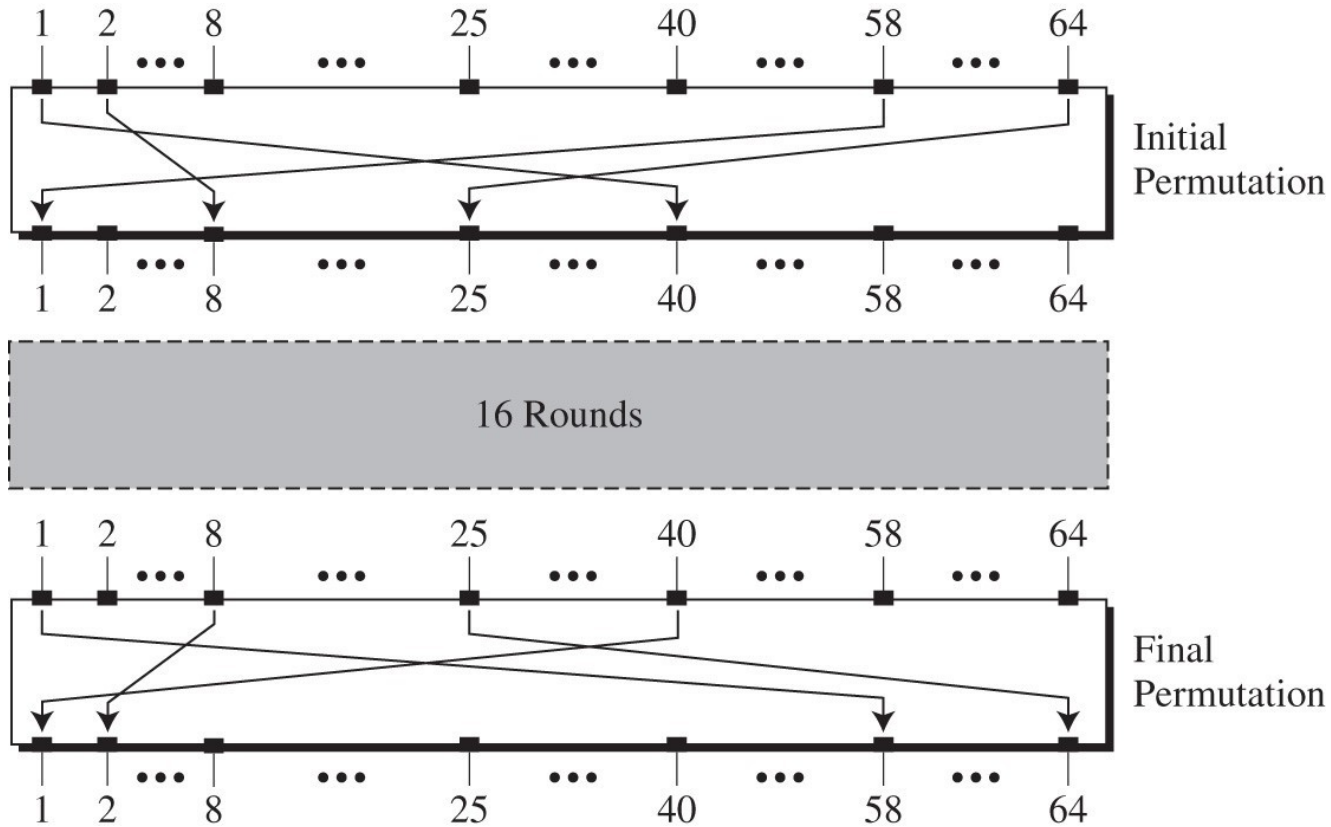
**Table 6.1** *Initial and final permutation tables*

| <i>Initial Permutation</i> | <i>Final Permutation</i> |
|----------------------------|--------------------------|
| 58 50 42 34 26 18 10 02    | 40 08 48 16 56 24 64 32  |
| 60 52 44 36 28 20 12 04    | 39 07 47 15 55 23 63 31  |
| 62 54 46 38 30 22 14 06    | 38 06 46 14 54 22 62 30  |
| 64 56 48 40 32 24 16 08    | 37 05 45 13 53 21 61 29  |
| 57 49 41 33 25 17 09 01    | 36 04 44 12 52 20 60 28  |
| 59 51 43 35 27 19 11 03    | 35 03 43 11 51 19 59 27  |
| 61 53 45 37 29 21 13 05    | 34 02 42 10 50 18 58 26  |
| 63 55 47 39 31 23 15 07    | 33 01 41 09 49 17 57 25  |

☞ Find the output of the initial permutation box when the input is given in hexadecimal as:  
0x0002 0000 0000 0001

- The input has only two 1s (bit 15 and bit 64); the output must also have only two 1s (the nature of straight permutation).
- Using Table, we can find the output related to these two bits.
- Bit 15 in the input becomes bit 63 in the output. Bit 64 in the input becomes bit 25 in the output.
- So, the output has only two 1s, bit 25 and bit 63.
- The result in hexadecimal is **0x0000 0080 0000 0002**





**Find the output of the initial permutation box when the input is 0x0000 0080 0000 0002**

| <i>Initial Permutation</i> |    |    |    |    |    |    |    |
|----------------------------|----|----|----|----|----|----|----|
| 58                         | 50 | 42 | 34 | 26 | 18 | 10 | 02 |
| 60                         | 52 | 44 | 36 | 28 | 20 | 12 | 04 |
| 62                         | 54 | 46 | 38 | 30 | 22 | 14 | 06 |
| 64                         | 56 | 48 | 40 | 32 | 24 | 16 | 08 |
| 57                         | 49 | 41 | 33 | 25 | 17 | 09 | 01 |
| 59                         | 51 | 43 | 35 | 27 | 19 | 11 | 03 |
| 61                         | 53 | 45 | 37 | 29 | 21 | 13 | 05 |
| 63                         | 55 | 47 | 39 | 31 | 23 | 15 | 07 |

- Input is in hexadecimal number
- Convert it into binary format
- Place the bit values according to initial
- permutation table
- Convert back the result to
- hexadecimal number

when the input is 0x0000 0080 0000 0002

Output is 0x0002 0000 0000 0001

# Prove that the initial and final permutations are the inverse of each other( Assignment for learners)

| Final Permutation |    |    |    |    |    |    |    |
|-------------------|----|----|----|----|----|----|----|
| 40                | 08 | 48 | 16 | 56 | 24 | 64 | 32 |
| 39                | 07 | 47 | 15 | 55 | 23 | 63 | 31 |
| 38                | 06 | 46 | 14 | 54 | 22 | 62 | 30 |
| 37                | 05 | 45 | 13 | 53 | 21 | 61 | 29 |
| 36                | 04 | 44 | 12 | 52 | 20 | 60 | 28 |
| 35                | 03 | 43 | 11 | 51 | 19 | 59 | 27 |
| 34                | 02 | 42 | 10 | 50 | 18 | 58 | 26 |
| 33                | 01 | 41 | 09 | 49 | 17 | 57 | 25 |

- From the previous slide input for final permutation is

0x0002 0000 0000 0001

When bit value changes according to final permutation table the result is

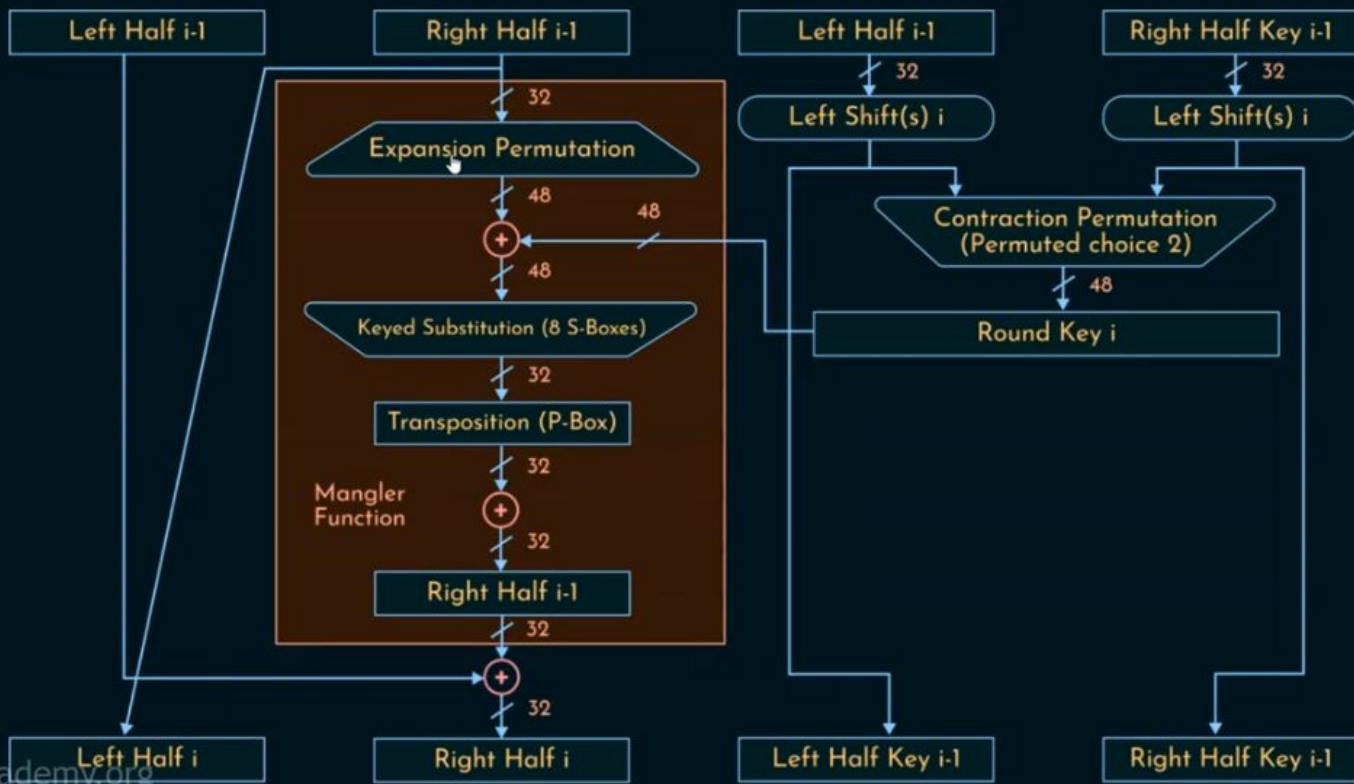
0x0000 0080 0000 0002

Which is same as input to initial permutation

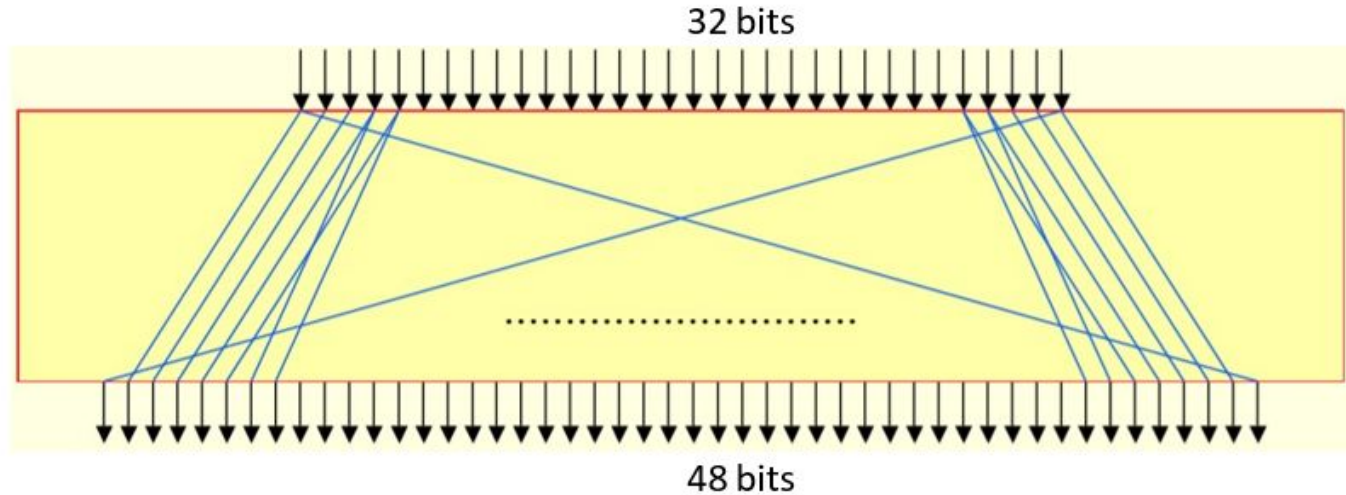
- Therefore initial and final permutation are inverse of each other



# Single Round of DES Algorithm

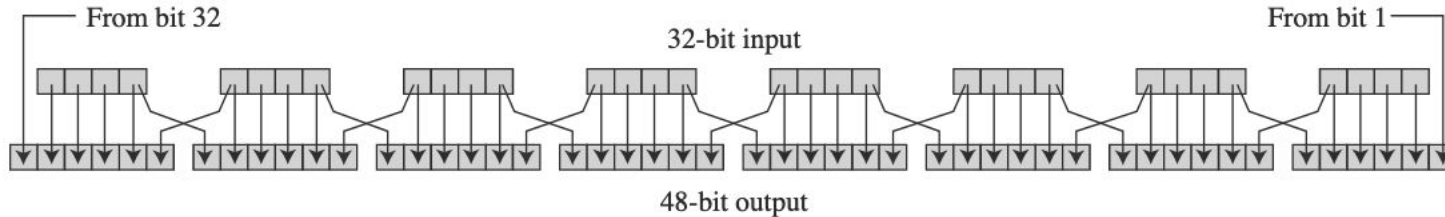


➡ E-box expands and permutes 32 bits to 48 bits



# Expansion Permutation

- During the expansion permutation, the RPT is expanded from 32 bits to 48 bits. Bits are permuted as well hence called as expansion permutation.
- This happens as the 32-bit RPT is divided into 8 blocks, with each block consisting of 4 bits.
- Then, each 4-bit block of the previous step is then expanded to a corresponding 6-bit block, i.e., per 4-bit block, 2 more bits are added.



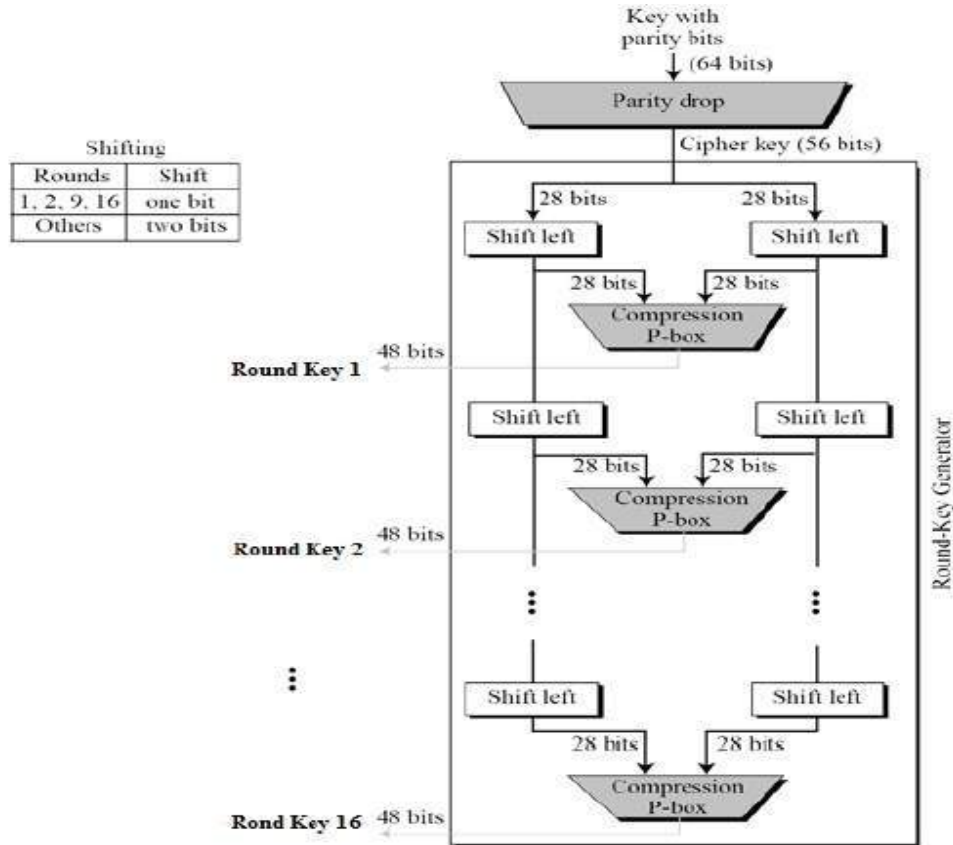
# Whitener (XOR)

---

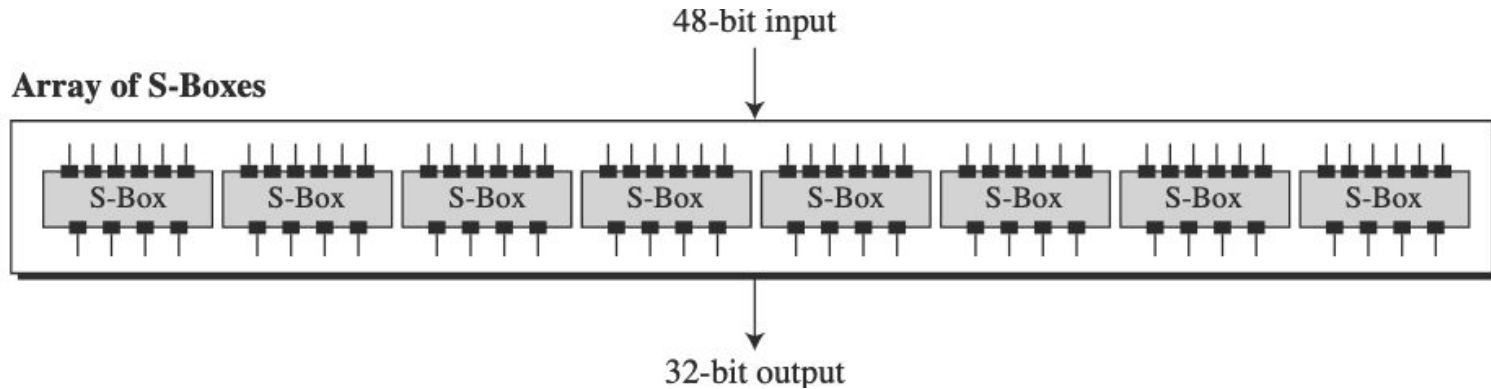
- ➡ After the expansion permutation, DES uses the XOR operation on the expanded right section and the round key.
- ➡ Note that both the right section and the key are 48-bits in length. Also note that the round key is used only in this operation.



# KEY TRANSFORMATION



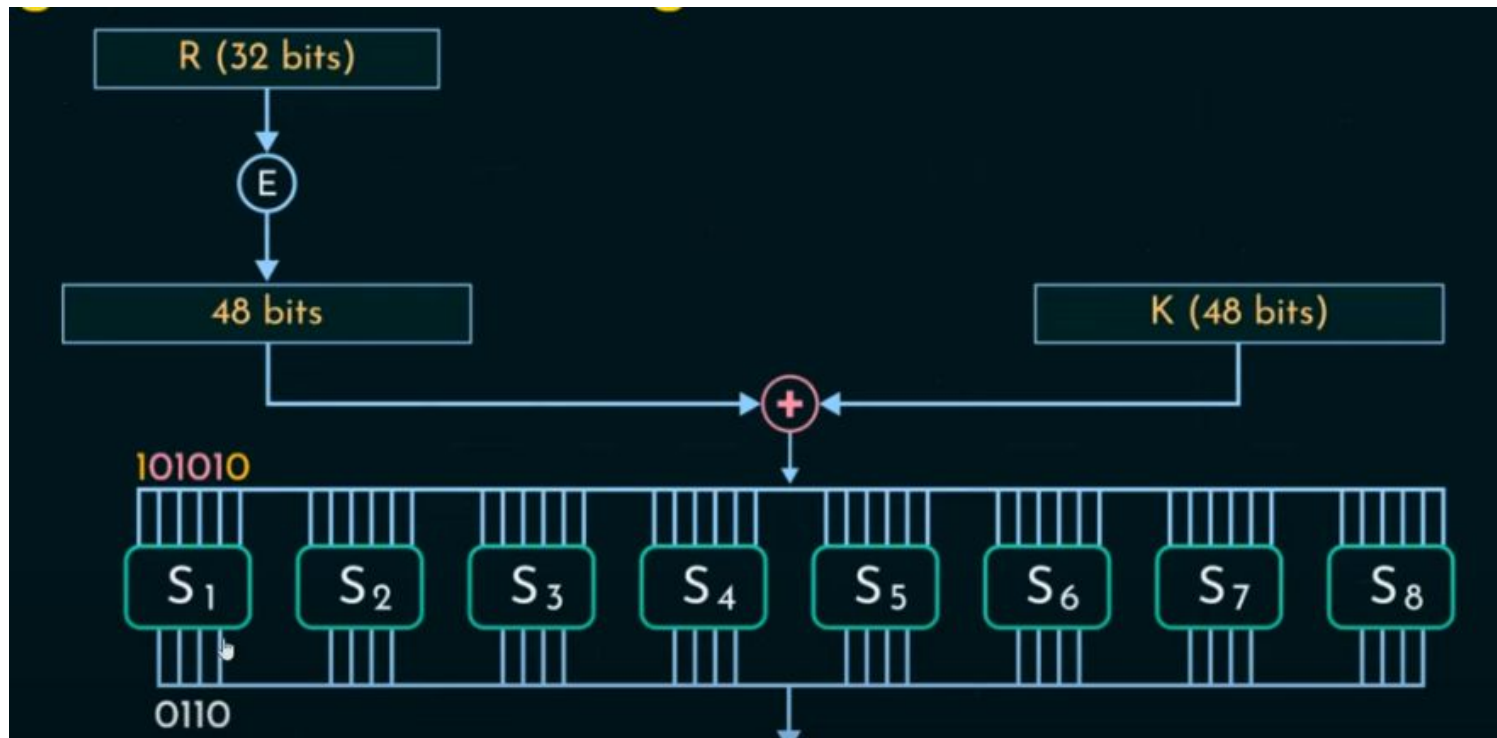
- ➡ The S-boxes do the real mixing (confusion). DES uses 8 S-boxes, each with a 6-bit input and a 4-bit output.



# Box $S_1$

|    | 0000 | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |
|----|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 00 | 14   | 4    | 13   | 1    | 2    | 15   | 11   | 8    | 3    | 10   | 6    | 12   | 5    | 9    | 0    | 7    |
| 01 | 0    | 15   | 7    | 4    | 14   | 2    | 13   | 1    | 10   | 6    | 12   | 11   | 6    | 5    | 3    | 8    |
| 10 | 4    | 1    | 14   | 8    | 13   | 6    | 2    | 11   | 15   | 12   | 9    | 7    | 3    | 10   | 5    | 0    |
| 11 | 15   | 12   | 8    | 2    | 4    | 9    | 1    | 7    | 5    | 11   | 3    | 14   | 10   | 0    | 6    | 13   |

For example,  $S_1(101010) = 6 = 0110$ .



# P Boxes – Straight Permutation

- ➡ **Straight Permutation** – The 32-bit output of S-boxes is then subjected to the straight permutation with rule shown in the following illustration:

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 16 | 07 | 20 | 21 | 29 | 12 | 28 | 17 |
| 01 | 15 | 23 | 26 | 05 | 18 | 31 | 10 |
| 02 | 08 | 24 | 14 | 32 | 27 | 03 | 09 |
| 19 | 13 | 30 | 06 | 22 | 11 | 04 | 25 |

## Criticism

- Reduction in key size of 72 bits
  - Too short to withstand with brute-force attack
- S-boxes were classified.
  - Weak points enable NSA to decipher without key.

## 56-bit keys have $2^{56} = 7.2 \times 10^{16}$ values

- Brute force search looks hard.
- A machine performing one DES encryption per microsecond would take more than a thousand year to break the cipher.



## Avalanche effect in DES

- If a small change in either the plaintext or the key, the ciphertext should change markedly.



DES exhibits a strong avalanche effect.

| (a) Change in Plaintext |                            | (b) Change in Key |                            |
|-------------------------|----------------------------|-------------------|----------------------------|
| Round                   | Number of bits that differ | Round             | Number of bits that differ |
| 0                       | 1                          | 0                 | 0                          |
| 1                       | 6                          | 1                 | 2                          |
| 2                       | 21                         | 2                 | 14                         |
| 3                       | 35                         | 3                 | 28                         |
| 4                       | 39                         | 4                 | 32                         |
| 5                       | 34                         | 5                 | 30                         |
| 6                       | 32                         | 6                 | 32                         |
| 7                       | 31                         | 7                 | 35                         |
| 8                       | 29                         | 8                 | 34                         |
| 9                       | 42                         | 9                 | 40                         |
| 10                      | 44                         | 10                | 38                         |
| 11                      | 32                         | 11                | 31                         |
| 12                      | 30                         | 12                | 33                         |
| 13                      | 30                         | 13                | 28                         |
| 14                      | 26                         | 14                | 26                         |
| 15                      | 29                         | 15                | 34                         |
| 16                      | 34                         | 16                | 35                         |



DES was proved insecure

- In 1997 on Internet in a few months
- in 1998 on dedicated h/w (EFF) in a few days
- In 1999 above combined in 22 hrs!



# DES: Key Search on Internet (1997)

---

- DES has  $2^{56}$  keys
- DES key found
- Search took 140 days
- Search used over 10,000 computers
- Peak rate:  $7 \cdot 10^9$  keys/sec
- 'Might' have taken 32 days

# DES Breaker (1998)

---

- ➡ Electronic Frontier Foundation
- ➡ Design cost \$ 80,000
- ➡ Manufacturing cost \$130,000
- ➡ Test key found in 56 hours
- ➡ Complete search in 220 hours
- ➡ 90 Billion keys per second
- ➡ Design details published

# Current usage of Data Encryption Standard

---

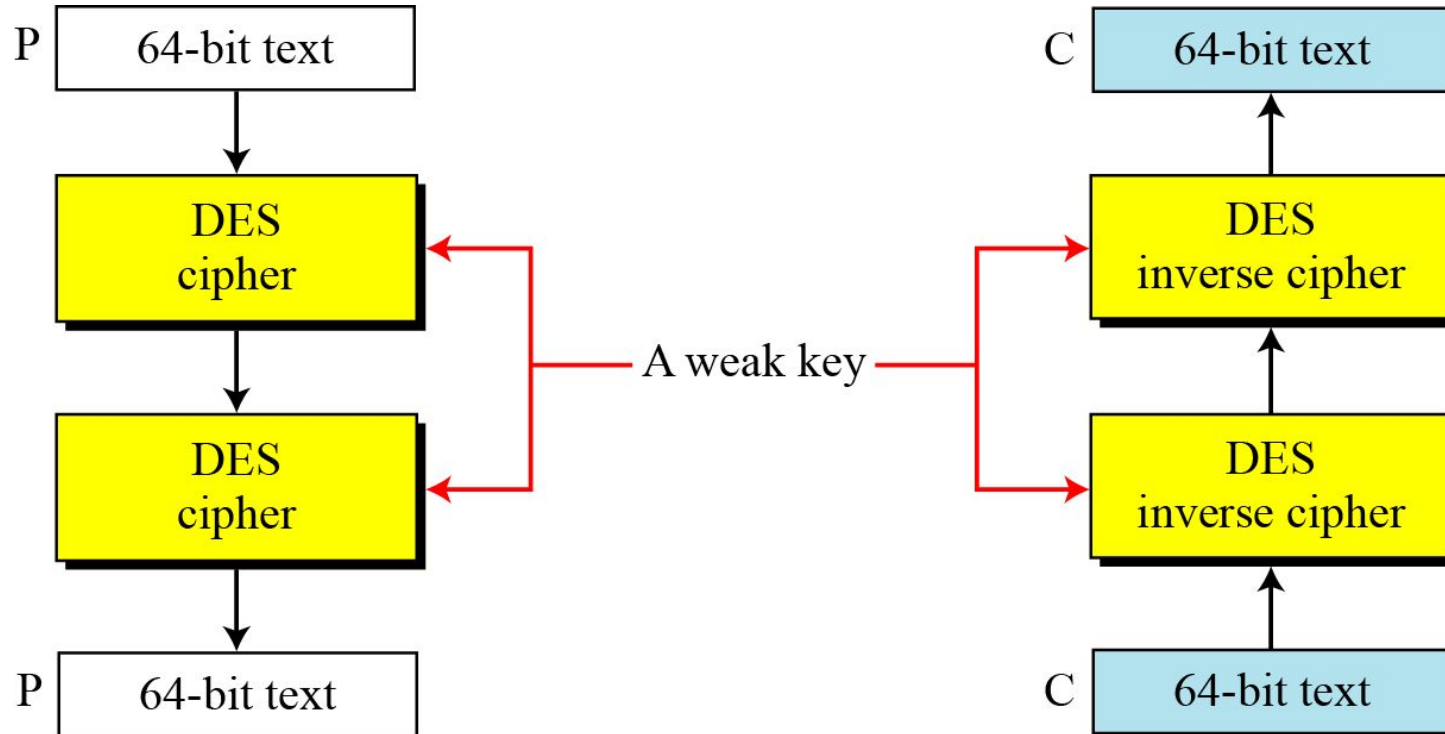
- ➡ Today, Data Encryption Standard is not used for encrypting any kind of confidential data.
- ➡ However, DES and 3DES are still used in a limited capacity for providing training on cryptography.
- ➡ 3DES is just too slow for modern use, and it's fatally vulnerable to memory-timing attacks.

# If DES is Becoming Irrelevant, Why Learn It?

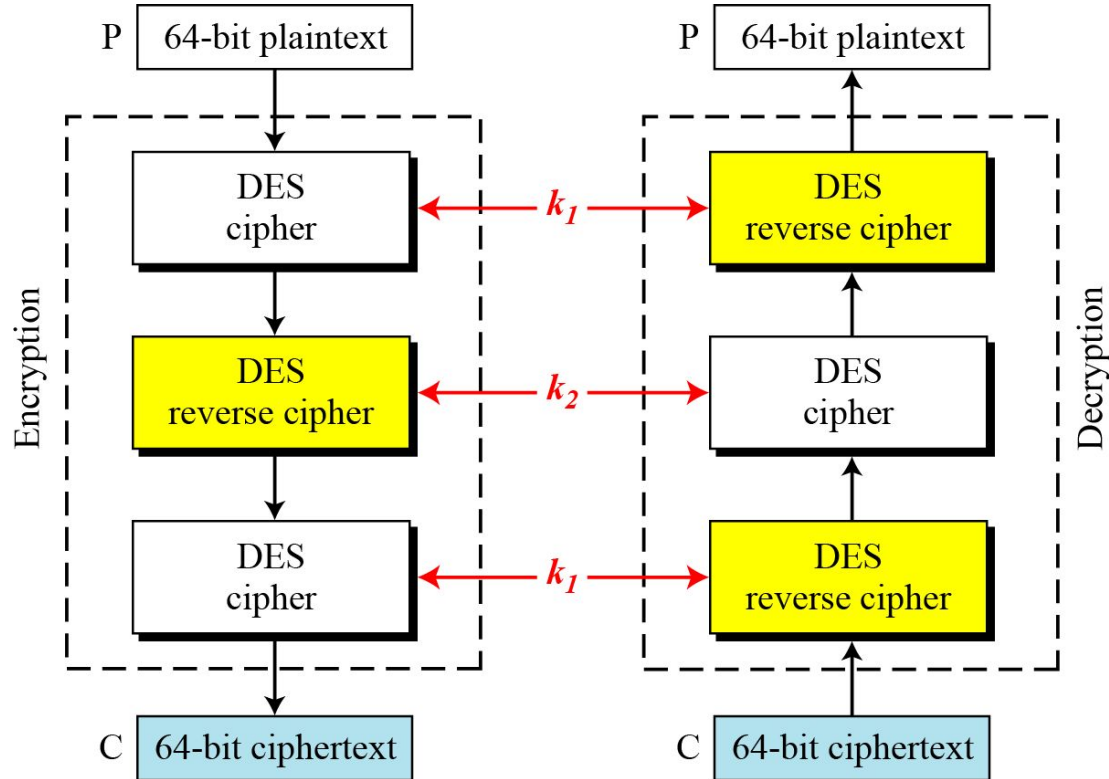
---

- ➡ The reason being the DES algorithm was the foundation of all subsequent algorithms for encryption.
- ➡ Once the origin of data encryption is understood, then it becomes easy to understand the fundamentals of modern-day encryption methods.
- ➡ Hence, it is important to learn DES encryption and get clarity on its fundamentals.

# Double DES



# Triple DES (with 2 key)



## ☞ *Triple DES with Three Keys*

*The possibility of known-plaintext attacks on triple DES with two keys has enticed some applications to use triple DES with three keys. Triple DES with three keys is used by many applications such as PGP*

# Question 1

---

**The DES algorithm is based on which block cipher?**

- A. AES
- B. LUCIFER**
- C. RSA
- D. Blowfish

**DES is based on the Feistel block cipher, called LUCIFER, developed in 1971 by IBM cryptography researcher Horst Feistel.**



## Question 2

The Feistel structure of DES uses a plaintext block size of \_\_\_\_\_ and \_\_\_\_\_ number of rounds.

- A. 32, 8
- B. 48, 12
- C. 64, 16**
- D. 128, 20

**DES uses 16 rounds of the Feistel structure, using a different key for each round. The algorithm takes the plain text in 64-bit blocks and converts them into ciphertext using 48-bit keys.**

## Question 3

**Which function is performed first on the 64-bit plaintext block in the DES algorithm process?**

- A. Final Permutation (FP)
- B. XOR operation
- C. S-box substitution
- D. Initial Permutation (IP)**

**The process begins with the 64-bit plain text block getting handed over to an initial permutation (IP) function. The initial permutation (IP) is then performed on the plain text.**

# Question 4

---

**What is the purpose of the E-box in DES?**

- A. To expand and permute 32 bits to 48 bits**
- B. To perform the final permutation
- C. To reduce the key size
- D. To mix (confusion) the bits

**During the expansion permutation, the RPT is expanded from 32 bits to 48 bits.**

## Question 5

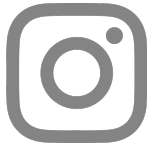
---

**Why is it important to learn the DES algorithm despite its irrelevance for confidential data encryption today?**

- A. It is still widely used in modern applications.
- B. It helps in understanding the fundamentals of modern-day encryption methods.**
- C. It offers superior security compared to newer algorithms.
- D. It is a mandatory requirement for all cryptography certifications.

# Thank you!

## Follow us



isfcr.pesu



[www.isfcr.pes.edu](http://www.isfcr.pes.edu)



ISFCR



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience





PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



Welcome to  
**PES University**  
Ring Road Campus, Bengaluru



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



# Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 16

# A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping



# A Note on Security

- ➡ In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- ➡ To be clear, **you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network** without the express consent of the owner.
- ➡ In particular, **you will comply with all my instructions when doing the labs.**
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- ➡ Any violation is at **YOUR RISK!**  
**And may result in severe consequences.**

- ➡ Cryptography requires sets of integers and specific operations that are defined for those sets.
- ➡ The combination of the set and the operations that are applied to the elements of the set is called an algebraic structure.
- ➡ Topics to be discussed in this section:
  - Groups
  - Rings
  - Fields



A group  $(G)$  is a set of elements with a binary operation  $(\bullet)$  that satisfies four properties (or axioms).

- **Closure:** For any two elements  $a, b \in G$ ,  $c = a \cdot b \in G$
- **Associativity:** For any three elements  $a, b, c \in G$ ,  $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
- **Identity:** There exists an Identity element  $e \in G$  such that  $\forall a \in G$ ,  $a \cdot e = e \cdot a = a$ .
- **Inverse:** Each element in  $G$  has an inverse i.e.  
 $\forall a \in G \exists a^{-1} \in G, a \cdot a^{-1} = a^{-1} \cdot a = e$ .



However, it is said to be an **Abelian group** if in addition to the above the set follows the axiom:

- **Commutativity:** For any  $a, b \in G$ ,  $a \cdot b = b \cdot a$



A Ring  $\{Rg, +, \times\}$  is a set with two binary operations addition and multiplication that satisfies the following axioms:

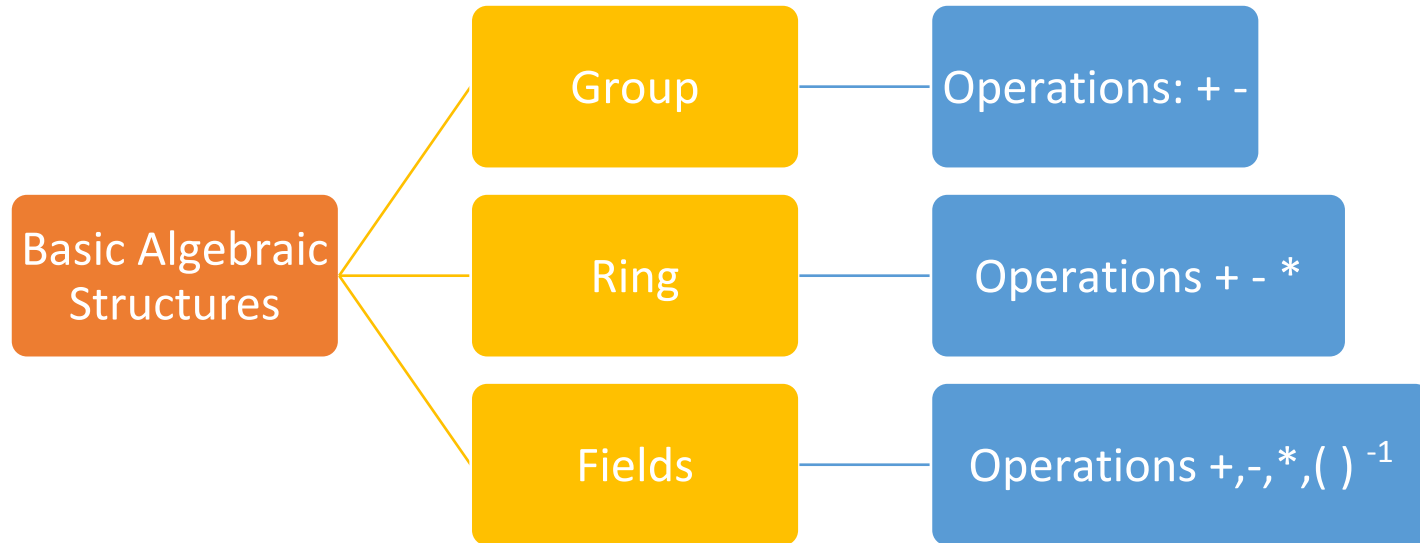
- Abelian Group under addition ( $A_1 \rightarrow A_5$ ): It satisfies all of the axioms for an abelian group with the operation of addition. The identity element is 0 and the inverse is denoted  $-a$ .
- Closure under multiplication ( $M_1$ ): For any two elements  $a, b \in Rg$ ,  $c = ab \in Rg$ .
- Associativity of multiplication ( $M_2$ ): For any elements  $a, b, c \in Rg$ ,  $(ab)c = a(bc)$ .
- Distributive ( $M_3$ ): For any elements  $a, b, c \in Rg$ ,  $a(b + c) = ab + ac$ .



A field  $F$  is an integral domain( Commutative Ring) that satisfies one additional property

- Assume the operations are  $+$  and  $\times$ , the additive identity is  $0$  and the multiplicative identity is  $1$
- For each  $a$  in  $F$ , except  $0$ , there is a multiplicative inverse  $a^{-1}$  such that  $a a^{-1} = a^{-1} a = 1$

# Algebraic Structures



# Galois Field

---

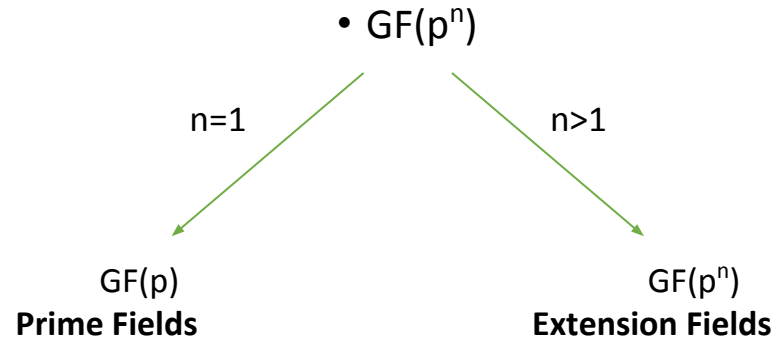
- ➡ Finite Fields, also known as Galois Fields, are cornerstones for understanding any cryptography.
- ➡ A field can be defined as a set of numbers that we can add, subtract, multiply and divide together and only ever end up with a result that exists in our set of numbers.
- ➡ This is particularly useful for crypto as we can deal with a limited set of extremely large numbers.
- ➡ The most crucial property of a finite field is that it has  $p^m$  elements where  $p$  is a prime number and  $m$  is whatever you choose.



# Examples

- ➡ A finite field with 11 elements can be defined as  $\text{GF}(11^1)$ .
- ➡ A finite field with 256 elements would be written as  $\text{GF}(2^8)$ .
- ➡ You can't have a finite field with 12 elements since you'd have to write it as  $2^2 * 3$  which breaks the convention of  $p^m$ .

# Types of Finite Fields





Prime fields are fields of prime order, i.e., fields with  $n = 1$ .

- Elements of the field  $GF(p)$  can be represented by integers  $0, 1, \dots, p-1$ .
- E.g.,  $GF(5) = \{0, 1, 2, 3, 4\}$



## **Addition, Subtraction, Multiplication:**

- Let  $a, b \in GF(p) = \{0, 1, 2, \dots, p-1\}$ , then
  - $a + b \equiv c \pmod{p}$
  - $a - b \equiv c \pmod{p}$
  - $a \cdot b \equiv c \pmod{p}$



## **Inversion:**

- Let  $a \in GF(p)$ , then an inverse ( $a^{-1}$ ) must satisfy
  - $a \cdot a^{-1} = 1 \pmod{p}$

- ➡ Unlike finite fields, whose elements are integers, **extension fields' elements are polynomials**. Extension fields =  $GF(2^m)$  where  $m > 1$
- ➡ These polynomials take the form of:

$$A(x) = a_2X^2 + a_1X^1 + a_0$$

# Example

- ☞  $GF(2^3) = GF(8)$  which means there'll be a total of 8 elements in this set.

(a2, a1, a0)

$$(0, 0, 0) = 0$$

$$(0, 0, 1) = 1$$

$$(0, 1, 0) = x$$

$$(1, 0, 0) = x^2$$

$$(0, 1, 1) = x+1$$

$$(1, 1, 0) = x^2+x$$

$$(1, 0, 1) = x^2+1$$

$$(1, 1, 1) = x^2+x+1$$

- ☞ Putting it all together,  $GF(2^3) = \{0, 1, x, x^2, x+1, x^2+x, x^2+1, x^2+x+1\}$

- ➡ When  $g(x)$  divides  $f(x)$  without leaving a remainder, we say  $g(x)$  is a factor of  $f(x)$ .
- ➡ A polynomial  $f(x)$  over a field  $F$  is called **irreducible** if  $f(x)$  cannot be expressed as a product of two polynomials, both over  $F$  and both of degree lower than that of  $f(x)$ .
- ➡ An irreducible polynomial is also referred to as a **prime polynomial**.
- ➡ AES uses  $GF(2^8)$  with irreducible polynomial  $x^8 + x^4 + x^3 + x + 1$ .

# Why Bother about Galois field?

---

- ➡ Numbers stay small--everything in  $GF(2^8)$  is always one byte, even after multiplication or division.
- ➡ Inverses exist for each operation, which makes decryption possible.
- ➡ There are enough operations for them to combine in rich ways.

# Question 1

---

Which property distinguishes an Abelian group from a general group?

- A. Closure
- B. Associativity
- C. Commutativity**
- D. Identity



## Question 2

---

What is the defining characteristic that differentiates a Field from

an integral domain (commutative ring)?

A. Existence of an additive identity

**B. Existence of a multiplicative inverse for every non-zero element**

C. Associativity of multiplication

D. Distributivity of multiplication over addition

## Question 3

Why is  $GF(56)$  not a valid finite field according to the given definition?

- A. It is not a prime number.
- B. It cannot be expressed in the form  $p^m$ .**
- C. It does not have an inverse for all elements.
- D. It violates the closure property.

**You can't have a finite field with 56 elements since you'd have to write it as  $28 * 7$  which breaks the convention of  $p^m$ .**

## Question 4

---

In a Prime Field  $GF(p)$ , how are addition and multiplication performed for elements 'a' and 'b'?

- A. Standard arithmetic operations
- B. Operations modulo  $p$**
- C. Polynomial addition and multiplication
- D. Matrix operations

## Question 5

---

What is a key advantage of using Galois fields in cryptography, as mentioned in the slides?

- A. They ensure numbers grow infinitely large.
- B. They guarantee the existence of inverses for operations.**
- C. They eliminate the need for prime numbers.
- D. They simplify complex number theory.

# Thank you!

## Follow us



isfcr.pesu



[www.isfcr.pes.edu](http://www.isfcr.pes.edu)



ISFCR



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience





PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



Welcome to  
**PES University**  
Ring Road Campus, Bengaluru



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



# Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 16

# A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping



# A Note on Security

- ➡ In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- ➡ To be clear, **you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network** without the express consent of the owner.
- ➡ In particular, **you will comply with all my instructions when doing the labs.**
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- ➡ Any violation is at **YOUR RISK!**  
**And may result in severe consequences.**

# AES

---

## Advanced Encryption Standard (Rijndael Cipher)

- ➡ A replacement for DES was needed due to its limitations.
  - Small key and block sizes
- ➡ Can use Triple-DES – but slow, small block.
- ➡ US NIST issued call for ciphers in 1997.
- ➡ 15 candidates submitted around 1998.
- ➡ 5 were shortlisted in Aug 1999.
- ➡ Rijndael from Belgium chosen as the AES in 2001 after an open process.

# AES Competition Requirements

---



In the selection process, NIST asked for

- 128-bit data, 128/192/256-bit keys.
- Stronger & faster than Triple-DES.
- Active life of 20-30 years.
- Provide full specification and design details.
- Royalty-free.



After testing and evaluation, shortlisted in Aug 1999

- MARS (IBM) - complex, fast, high security margin
- RC6 (USA) - v. simple, v. fast, low security margin
- Rijndael (Belgium) - clean, fast, good security margin
- Serpent (Euro) - slow, clean, v. high security margin
- Twofish (USA) - complex, v. fast, high security margin



Found contrast between algorithms with:

- Few complex rounds versus many simple rounds.
- Refined versions of existing ciphers versus new proposals.

# The AES Cipher - Rijndael

- Rijndael was selected as the AES in Oct. 2000.
  - by Vincent Rijmen & Joan Daemen from Belgium
- Not Feistel network, substitution–permutation network (SPN), iterated block cipher.
- It has three allowable key sizes  $|K| = \{128, 192, 256\}$ .
- It has variable number of rounds.
  - If  $|K| = 128$  then  $N_r = 10$
  - If  $|K| = 192$  then  $N_r = 12$
  - If  $|K| = 256$  then  $N_r = 14$
- Chosen because of its security, performance, efficiency, ease of implementation, and flexibility.



V. Rijmen



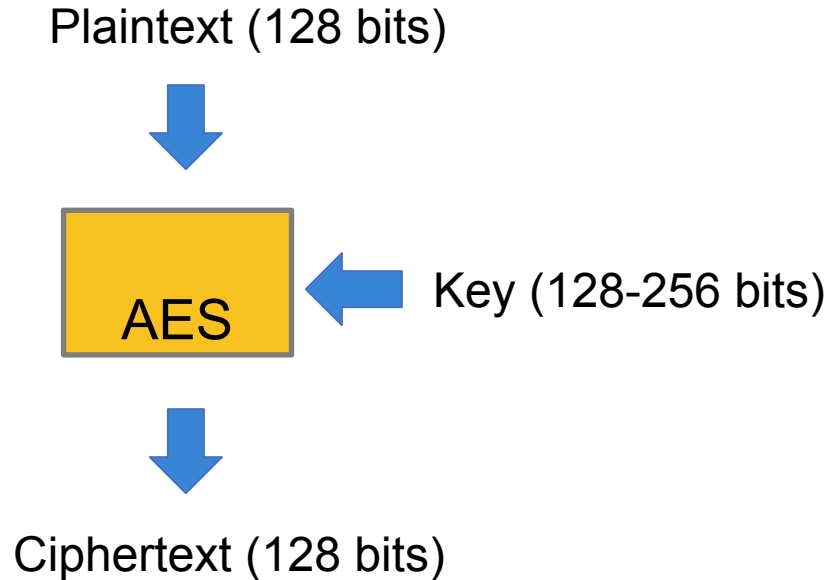
J. Daemen

# DES vs AES Cipher

---

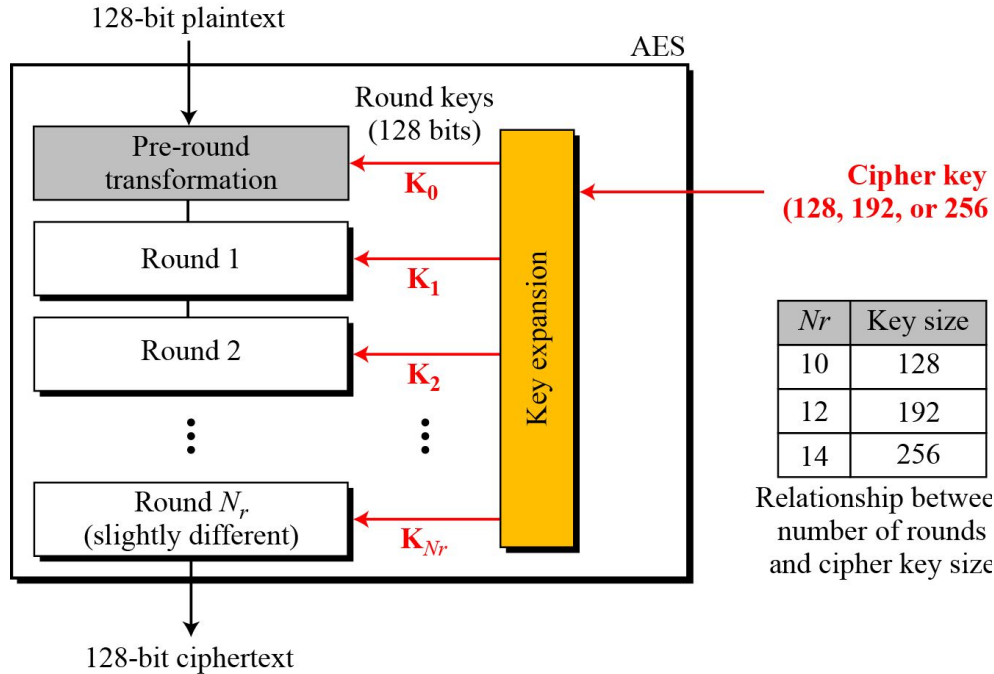
- ➡ DES is a bit-oriented cipher, AES is a byte-oriented cipher.
- ➡ DES - Feistel network, AES - Substitution Permutation Network (SPN),
- ➡ Iterated block cipher.
- ➡ Key-alternating block ciphers (Diffusion – transformation operation, Confusion – round key)

# AES Conceptual Scheme





# Overall Structure of AES



- Rounds are (almost) identical.
- First and last round are a little different.

- **Key Expansion:**

- Round keys are derived from the cipher key using Rijndael's key schedule.

- **Initial Round:** Add RoundKey

- Each byte of the state is combined with the round key using bitwise Xor

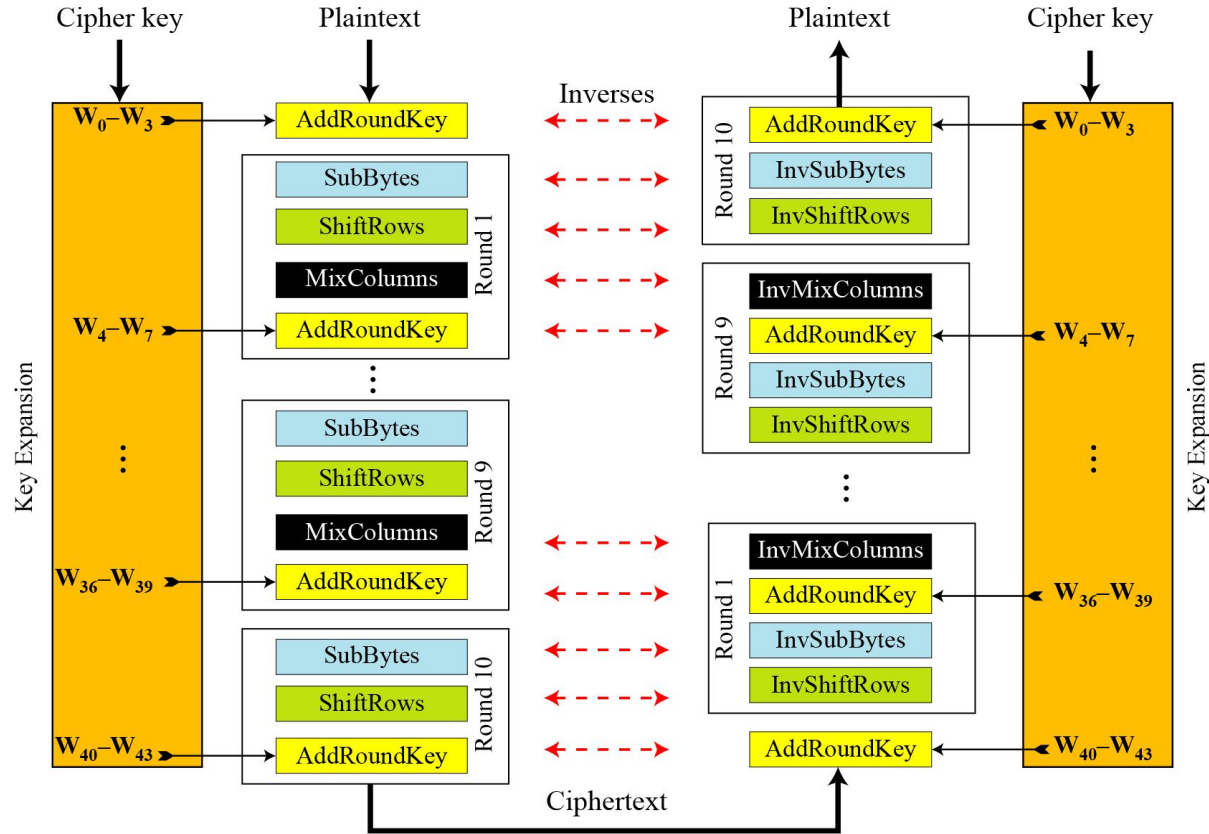
- **Rounds:**

- SubBytes : non-linear substitution step
- ShiftRows : transposition step
- MixColumns : mixing operation of each column
- AddRoundKey

- **Final Round:**

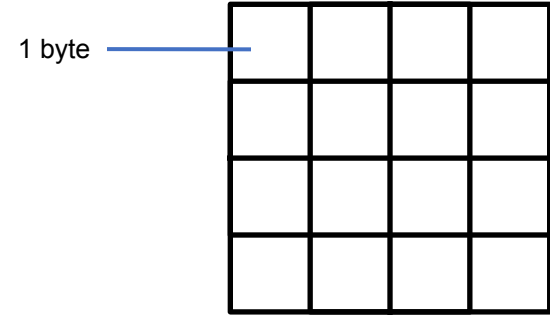
- SubBytes
- ShiftRows
- AddRoundKey

# Overall Structure of AES



# 128-bit values

- ➡ Data block viewed as 4 x 4 array of bytes (matrix of 8-bit bytes) - state array in AES.
- ➡ **First four** bytes of a 128-bit input block occupy the first column in the 4 × 4 array of bytes.
- ➡ A **word** consists of four bytes, that is 32 bits. Therefore, each column of the state array is a word, as is each row.
- ➡ Each round of processing -> works on input state array and produces output state array.



$$\begin{bmatrix} \text{byte}_0 & \text{byte}_4 & \text{byte}_8 & \text{byte}_{12} \\ \text{byte}_1 & \text{byte}_5 & \text{byte}_9 & \text{byte}_{13} \\ \text{byte}_2 & \text{byte}_6 & \text{byte}_{10} & \text{byte}_{14} \\ \text{byte}_3 & \text{byte}_7 & \text{byte}_{11} & \text{byte}_{15} \end{bmatrix}$$

# The Encryption Key and its Expansion

- ➡ Key is also arranged in the form of an array of  $4 \times 4$  bytes.
- ➡ As with the input block, the first **word** from the key fills the first column of the array, and so on.
- ➡ The four column words of the key array are expanded into a **schedule** of 44 words.
- ➡ Each round consumes four words from the key schedule.

|       |       |          |          |
|-------|-------|----------|----------|
| $k_0$ | $k_4$ | $k_8$    | $k_{12}$ |
| $k_1$ | $k_5$ | $k_9$    | $k_{13}$ |
| $k_2$ | $k_6$ | $k_{10}$ | $k_{14}$ |
| $k_3$ | $k_7$ | $k_{11}$ | $k_{15}$ |



|       |       |       |       |       |       |     |  |  |  |  |  |  |  |  |  |          |          |
|-------|-------|-------|-------|-------|-------|-----|--|--|--|--|--|--|--|--|--|----------|----------|
| $w_0$ | $w_1$ | $w_2$ | $w_3$ | $w_4$ | $w_5$ | ... |  |  |  |  |  |  |  |  |  | $w_{42}$ | $w_{43}$ |
|-------|-------|-------|-------|-------|-------|-----|--|--|--|--|--|--|--|--|--|----------|----------|

Of these, the first four words are used for adding to the input state array before any round-based processing can begin, and the remaining 40 words used for the ten rounds of processing that are required for the case a 128-bit encryption key.

## Given Plaintext: AES USES A MATRIX

Has 14 characters =  $14 * 8 = 112$  bits

Converting plaintext to state which is 128 bits.

$128 - 112 = 16$  bits needed

So, add 2 extra characters.

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| Plaintext → | a  | b  | c  | d  | e  | f  | g  | h  | i  | j  | k  | l  | m  | n  | o  | p  | q  | r  | s  | t  | u  | v  | w  | x  | y  | z  |
| Value →     | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 |

# Changing Plaintext to State

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| Text        | A  | E  | S  | U  | S  | E  | S  | A  | M  | A  | T  | R  | I  | X  | Z  | Z  |
| Hexadecimal | 00 | 04 | 12 | 14 | 12 | 04 | 12 | 00 | 0C | 00 | 13 | 11 | 08 | 17 | 19 | 19 |

We show each byte as an integer with two hexadecimal digits.

For example, the character “S” is first changed to 18 (since S is the 19th letter of the alphabet) and then written as 12 in hexadecimal.

$$\begin{bmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 17 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{bmatrix} \text{State}$$

The state matrix is then filled up, column by column.

# Changing Plaintext to State - Example 2

Using the phrase - CRYPTOGRAPHYAES

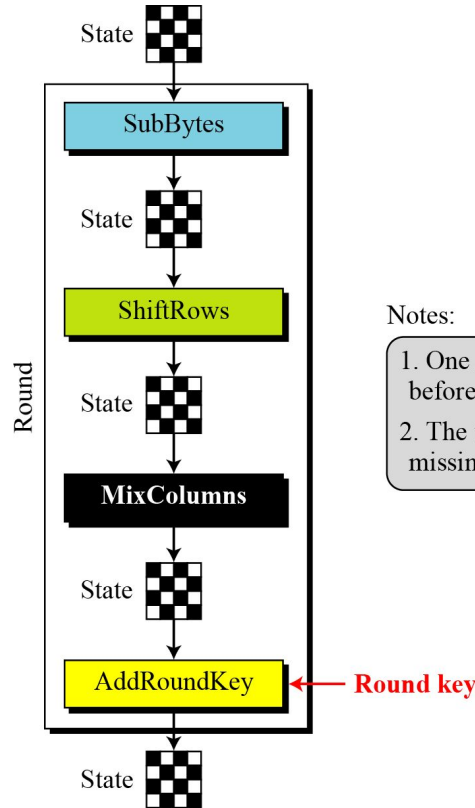
|       | C  | R  | Y  | P  | T  | O  | G  | R  | A  | P  | H  | Y  | A  | E  | S  |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| Index | 2  | 17 | 24 | 15 | 19 | 14 | 6  | 17 | 0  | 15 | 7  | 24 | 0  | 4  | 18 |
| Hexa  | 02 | 11 | 18 | 0F | 13 | 0E | 06 | 11 | 00 | 0F | 07 | 18 | 00 | 04 | 12 |

State Matrix

|    |    |    |    |
|----|----|----|----|
| 02 | 13 | 00 | 00 |
| 11 | 0E | 0F | 04 |
| 18 | 06 | 07 | 12 |
| 0F | 11 | 18 | 00 |



# Details of Each Round



Notes:

1. One AddRoundKey is applied before the first round.
2. The third transformation is missing in the last round.

Each round of processing includes:

- one single-byte based substitution step (byte-level),
- a row-wise permutation step (word level),
- a column-wise mixing step &
- the addition of the round key.

# Plaintext: AES USES MATRIX

- State

|      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Text | A | E | S | U | S | E | S | A | M | A | T | R | I | X | Z | Z |
|------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| Hexadecimal | 00 | 04 | 12 | 14 | 12 | 04 | 12 | 00 | 0C | 00 | 13 | 11 | 08 | 17 | 19 | 19 |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 00 | 12 | 0C | 08 |
| 04 | 04 | 00 | 17 |
| 12 | 12 | 13 | 19 |
| 14 | 00 | 11 | 19 |

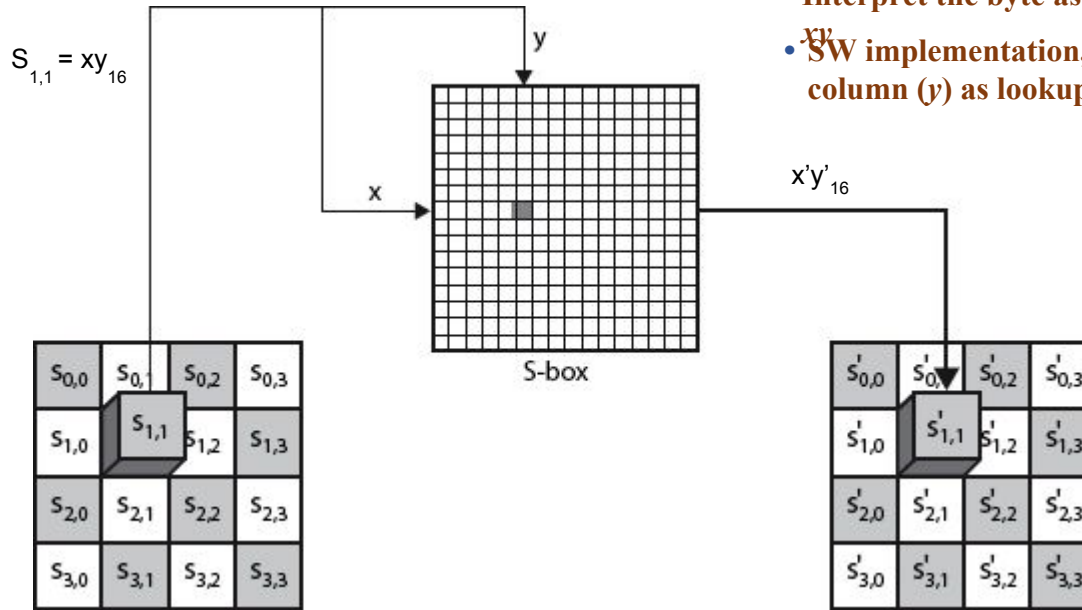
State

# Step 1: SubBytes - Byte Substitution

---

- Byte-by-byte substitution in all rounds.
  - provides diffusion
- Uses 16x16 lookup table (S-box) containing a permutation of all 256 8-bit values.
- Each byte of state is replaced by byte indexed by row (left 4-bits) & column (right 4-bits).
  - E.g., byte {95} is replaced by byte in row 9, column 5 which has value {2A}
- The entries in the lookup table are created by using the notions of multiplicative inverses in  $GF(2^8)$ .

- The SubBytes operation involves 16 independent byte-to-byte transformations.



- Interpret the byte as two hexadecimal digits
- SW implementation, use row ( $x$ ) and column ( $y$ ) as lookup pointer.

**Goal:** Bit scrambling to destroy the bit-level correlations inside each byte.

# SubBytes Table

$$\begin{bmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 17 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{bmatrix}$$

AFTER  
SUBBYTES

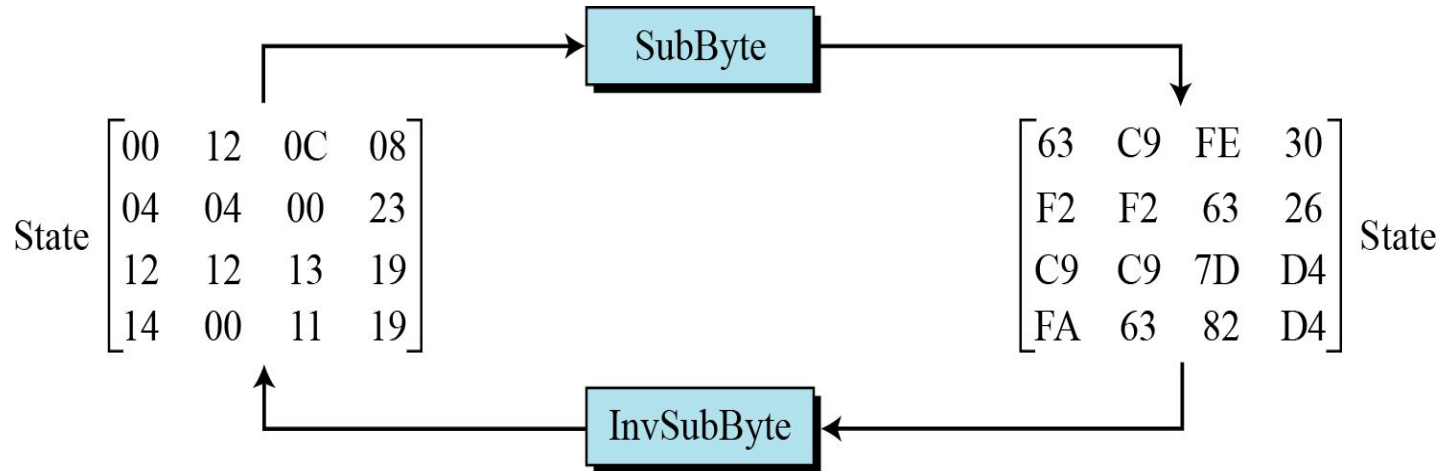
|   |   | y  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|   |   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
| x | 0 | 63 | 7C | 77 | 7B | F2 | 6B | 6F | C5 | 30 | 01 | 67 | 2B | FE | D7 | AB | 76 |
|   | 1 | CA | 82 | C9 | 7D | FA | 59 | 47 | F0 | AD | D4 | A2 | AF | 9C | A4 | 72 | C0 |
|   | 2 | B7 | FD | 93 | 26 | 36 | 3F | F7 | CC | 34 | A5 | E5 | F1 | 71 | D8 | 31 | 15 |
|   | 3 | 04 | C7 | 23 | C3 | 18 | 96 | 05 | 9A | 07 | 12 | 80 | E2 | EB | 27 | B2 | 75 |
|   | 4 | 09 | 83 | 2C | 1A | 1B | 6E | 5A | A0 | 52 | 3B | D6 | B3 | 29 | E3 | 2F | 84 |
|   | 5 | 53 | D1 | 00 | ED | 20 | FC | B1 | 5B | 6A | CB | BE | 39 | 4A | 4C | 58 | CF |
|   | 6 | D0 | EF | AA | FB | 43 | 4D | 33 | 85 | 45 | F9 | 02 | 7F | 50 | 3C | 9F | A8 |
|   | 7 | 51 | A3 | 40 | 8F | 92 | 9D | 38 | F5 | BC | B6 | DA | 21 | 10 | FF | F3 | D2 |
|   | 8 | CD | 0C | 13 | EC | 5F | 97 | 44 | 17 | C4 | A7 | 7E | 3D | 64 | 5D | 19 | 73 |
|   | 9 | 60 | 81 | 4F | DC | 22 | 2A | 90 | 88 | 46 | EE | B8 | 14 | DE | 5E | 0B | DB |
|   | A | E0 | 32 | 3A | 0A | 49 | 06 | 24 | 5C | C2 | D3 | AC | 62 | 91 | 95 | E4 | 79 |
|   | B | E7 | C8 | 37 | 6D | 8D | D5 | 4E | A9 | 6C | 56 | F4 | EA | 65 | 7A | AE | 08 |
|   | C | BA | 78 | 25 | 2E | 1C | A6 | B4 | C6 | E8 | DD | 74 | 1F | 4B | BD | 8B | 8A |
|   | D | 70 | 3E | B5 | 66 | 48 | 03 | F6 | 0E | 61 | 35 | 57 | B9 | 86 | C1 | 1D | 9E |
|   | E | E1 | F8 | 98 | 11 | 69 | D9 | 8E | 94 | 9B | 1E | 87 | E9 | CE | 55 | 28 | DF |
|   | F | 8C | A1 | 89 | 0D | BF | E6 | 42 | 68 | 41 | 99 | 2D | 0F | B0 | 54 | BB | 16 |

# InvSubBytes Table

|    |    |    |    |
|----|----|----|----|
| 63 | C9 | FE | 30 |
| F2 | F2 | 63 | 26 |
| C9 | C9 | 7D | D4 |
| FA | 63 | 82 | D4 |

|   |   | y  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|   |   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
| x | 0 | 52 | 09 | 6A | D5 | 30 | 36 | A5 | 38 | BF | 40 | A3 | 9E | 81 | F3 | D7 | FB |
|   | 1 | 7C | E3 | 39 | 82 | 9B | 2F | FF | 87 | 34 | 8E | 43 | 44 | C4 | DE | E9 | CB |
|   | 2 | 54 | 7B | 94 | 32 | A6 | C2 | 23 | 3D | EE | 4C | 95 | 0B | 42 | FA | C3 | 4E |
|   | 3 | 08 | 2E | A1 | 66 | 28 | D9 | 24 | B2 | 76 | 5B | A2 | 49 | 6D | 8B | D1 | 25 |
|   | 4 | 72 | F8 | F6 | 64 | 86 | 68 | 98 | 16 | D4 | A4 | 5C | CC | 5D | 65 | B6 | 92 |
|   | 5 | 6C | 70 | 48 | 50 | FD | ED | B9 | DA | 5E | 15 | 46 | 57 | A7 | 8D | 9D | 84 |
|   | 6 | 90 | D8 | AB | 00 | 8C | BC | D3 | 0A | F7 | E4 | 58 | 05 | B8 | B3 | 45 | 06 |
|   | 7 | D0 | 2C | 1E | 8F | CA | 3F | 0F | 02 | C1 | AF | BD | 03 | 01 | 13 | 8A | 6B |
|   | 8 | 3A | 91 | 11 | 41 | 4F | 67 | DC | EA | 97 | F2 | CF | CE | F0 | B4 | E6 | 73 |
|   | 9 | 96 | AC | 74 | 22 | E7 | AD | 35 | 85 | E2 | F9 | 37 | E8 | 1C | 75 | DF | 6E |
|   | A | 47 | F1 | 1A | 71 | 1D | 29 | C5 | 89 | 6F | B7 | 62 | 0E | AA | 18 | BE | 1B |
|   | B | FC | 56 | 3E | 4B | C6 | D2 | 79 | 20 | 9A | DB | C0 | FE | 78 | CD | 5A | F4 |
|   | C | 1F | DD | A8 | 33 | 88 | 07 | C7 | 31 | B1 | 12 | 10 | 59 | 27 | 80 | EC | 5F |
|   | D | 60 | 51 | 7F | A9 | 19 | B5 | 4A | 0D | 2D | E5 | 7A | 9F | 93 | C9 | 9C | EF |
|   | E | A0 | E0 | 3B | 4D | AE | 2A | F5 | B0 | C8 | EB | BB | 3C | 83 | 53 | 99 | 61 |
|   | F | 17 | 2B | 04 | 7E | BA | 77 | D6 | 26 | E1 | 69 | 14 | 63 | 55 | 21 | 0C | 7D |

# Sample SubByte Transformation



# Sample SubByte Transformation - 2

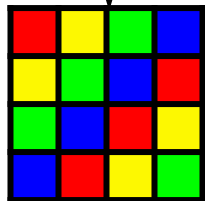
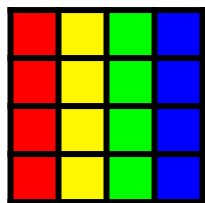
|    |    |    |    |
|----|----|----|----|
| EA | 04 | 65 | 85 |
| 83 | 45 | 5D | 96 |
| 5C | 33 | 98 | B0 |
| F0 | 2D | AD | C5 |

→

|    |    |    |    |
|----|----|----|----|
| 87 | F2 | 4D | 97 |
| EC | 6E | 4C | 90 |
| 4A | C3 | 46 | E7 |
| 8C | D8 | 95 | A6 |



# Step 2: ShiftRows



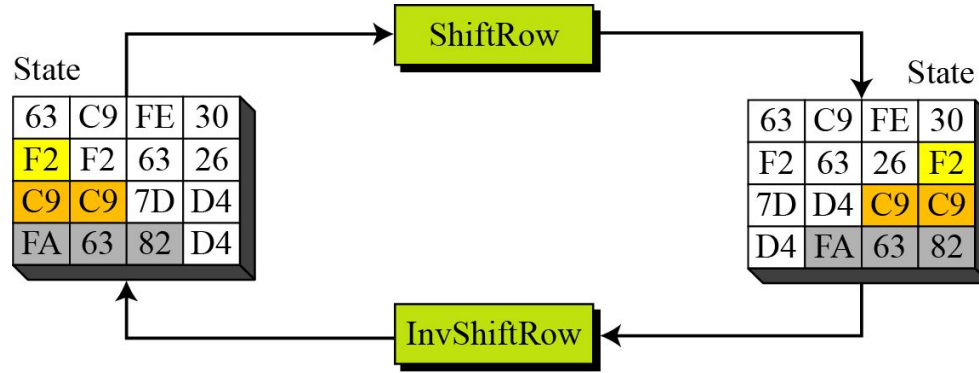
**Goal:** to scramble the byte order inside each 128-bit block.

- Shifting - permutes the bytes.
- A circular byte shift in each,
  - 1<sup>st</sup> row is unchanged
  - 2<sup>nd</sup> row does 1-byte circular shift to left
  - 3<sup>rd</sup> row does 2 bytes circular shift to left
  - 4<sup>th</sup> row does 3 bytes circular shift to left

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \implies \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{bmatrix}$$

- In the encryption, the transformation is called ShiftRows.
- In the decryption, the transformation is called **InvShiftRows** and the shifting is to the right.

# ShiftRows and InvShiftRows



# ShiftRows example

|    |    |    |    |
|----|----|----|----|
| 87 | F2 | 4D | 97 |
| EC | 6E | 4C | 90 |
| 4A | C3 | 46 | E7 |
| 8C | D8 | 95 | A6 |

→

|    |    |    |    |
|----|----|----|----|
| 87 | F2 | 4D | 97 |
| 6E | 4C | 90 | EC |
| 46 | E7 | 4A | C3 |
| A6 | 8C | D8 | 95 |

# Step 3: MixColumns

- ➡ ShiftRows and MixColumns provide diffusion to the cipher.
- ➡ In DES, one bit of plaintext affected roughly 31 bits of ciphertext. But now we want each bit of the plaintext to affect every bit position of the ciphertext block of 128 bits.
- ➡ Replaces each byte of a column by a function of all the bytes in the same column.
- ➡ Effectively a matrix multiplication in  $GF(2^8)$  using prime poly  $m(x) = x^8 + x^4 + x^3 + x + 1$ .
- ➡ **Goal: to further scramble up the 128-bit input block.**

# Step 3: MixColumns – Matrix Multiplication

---

- ➡ The multiplication is performed one column at a time (4 bytes).
- ➡ Each value in the column is eventually multiplied against every value of the matrix (16 total multiplications).
- ➡ The results of these multiplications are XORed together to produce only 4 result bytes for the next state.
- ➡ Therefore 4 bytes input, 16 multiplications 12 XORs and 4 bytes output.
- ➡ The multiplication is performed one matrix row at a time against each value of a state column.

# Step 3: MixColumns – Matrix Multiplication

## Multiplication Matrix

|   |   |   |   |
|---|---|---|---|
| 2 | 3 | 1 | 1 |
| 1 | 2 | 3 | 1 |
| 1 | 1 | 2 | 3 |
| 3 | 1 | 1 | 2 |

## 16 byte State

|    |    |     |     |
|----|----|-----|-----|
| b1 | b5 | b9  | b13 |
| b2 | b6 | b10 | b14 |
| b3 | b7 | b11 | b15 |
| b4 | b8 | b12 | b16 |

The first result byte is calculated by multiplying 4 values of the state column against 4 values of the first row of the matrix. The result of each multiplication is then XORed to produce 1 Byte.

$$b1 = (b1 * 2) \text{ XOR } (b2 * 3) \text{ XOR } (b3 * 1) \text{ XOR } (b4 * 1)$$

$$b2 = (b1 * 1) \text{ XOR } (b2 * 2) \text{ XOR } (b3 * 3) \text{ XOR } (b4 * 1)$$

$$b3 = (b1 * 1) \text{ XOR } (b2 * 1) \text{ XOR } (b3 * 2) \text{ XOR } (b4 * 3)$$

$$b4 = (b1 * 3) \text{ XOR } (b2 * 1) \text{ XOR } (b3 * 1) \text{ XOR } (b4 * 2)$$

The multiplication mentioned above is performed over a Galois Field.

# Step 3: MixColumns – GF Multiplication

**L Table**

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F |
| 1 | 64 | 0E | 1B | 34 | 4A | 7D | 81 | 9F | 4C | 71 | 08 | C8 | F8 | 69 | 1C | C1 |
| 2 | 7D | C2 | 1D | B5 | F9 | B9 | 27 | 6A | 4D | E4 | A6 | 72 | 9A | C9 | 09 | 78 |
| 3 | 65 | 2F | 8A | 05 | 21 | 0F | E1 | 24 | 12 | F0 | 82 | 45 | 35 | 93 | DA | 8E |
| 4 | 96 | 8F | DB | BD | 36 | D0 | CE | 94 | 13 | 5C | D2 | F1 | 40 | 46 | 83 | 38 |
| 5 | 66 | DD | FD | 30 | BF | 06 | 8B | 62 | B3 | 25 | E2 | 98 | 22 | 88 | 91 | 10 |
| 6 | 7E | 6E | 48 | C3 | A3 | B6 | 1E | 42 | 3A | 6B | 28 | 54 | FA | 85 | 3D | BA |
| 7 | 2B | 79 | 0A | 15 | 9B | 9F | 5E | CA | 4E | D4 | AC | E5 | F3 | 73 | A7 | 57 |
| 8 | AF | 58 | A8 | 50 | F4 | EA | D6 | 74 | 4F | AE | E9 | D5 | E7 | E6 | AD | E8 |
| 9 | 2C | D7 | 75 | 7A | EB | 16 | 0B | F5 | 59 | CB | 5F | B0 | 9C | A9 | 51 | A0 |
| A | 7F | 0C | F6 | 6F | 17 | C4 | 49 | EC | D8 | 43 | 1F | 2D | A4 | 76 | 7B | B7 |
| B | CC | BB | 3E | 5A | FB | 60 | B1 | 86 | 3B | 52 | A1 | 6C | AA | 55 | 29 | 9D |
| C | 97 | B2 | 87 | 90 | 61 | BE | DC | FC | BC | 95 | CF | CD | 37 | 3F | 5B | D1 |
| D | 53 | 39 | 84 | 3C | 41 | A2 | 6D | 47 | 14 | 2A | 9E | 5D | 56 | F2 | D3 | AB |
| E | 44 | 11 | 92 | D9 | 23 | 20 | 2E | 89 | B4 | 7C | B8 | 26 | 77 | 99 | E3 | A5 |
| F | 67 | 4A | ED | DE | C5 | 31 | FE | 18 | 0D | 63 | 8C | 80 | C0 | F7 | 70 | 07 |

The result of the multiplication is simply the result of a lookup of the L table, followed by the addition of the results, followed by a lookup to the E table (shown on the next slide).

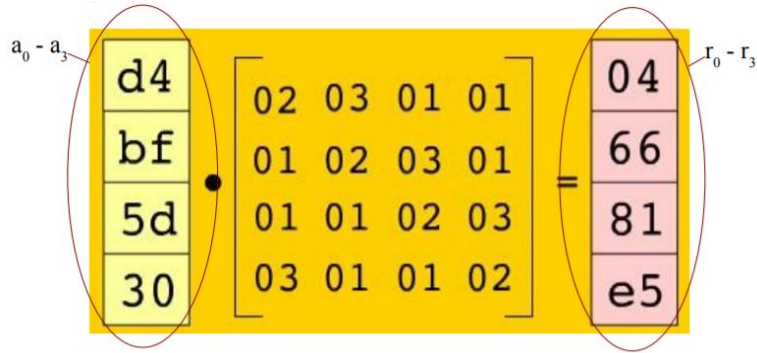
# Step 3: MixColumns – GF Multiplication

**E Table**

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 01 | 03 | 05 | 0F | 11 | 33 | 55 | FF | 1A | 2E | 72 | 96 | A1 | F8 | 13 | 35 |
| 1 | 5F | E1 | 38 | 48 | D8 | 73 | 95 | A4 | F7 | 02 | 06 | 0A | 1E | 22 | 66 | AA |
| 2 | E5 | 34 | 5C | E4 | 37 | 59 | EB | 26 | 6A | BE | D9 | 70 | 90 | AB | E6 | 31 |
| 3 | 53 | F5 | 04 | 0C | 14 | 3C | 44 | CC | 4F | D1 | 68 | B8 | D3 | 6E | B2 | CD |
| 4 | 4C | D4 | 67 | A9 | E0 | 3B | 4D | D7 | 62 | A6 | F1 | 08 | 18 | 28 | 78 | 88 |
| 5 | 83 | 9E | B9 | D0 | 6B | BD | DC | 7F | 81 | 98 | B3 | CE | 49 | DB | 76 | 9A |
| 6 | B5 | C4 | 57 | F9 | 10 | 30 | 50 | F0 | 0B | 1D | 27 | 69 | BB | D6 | 61 | A3 |
| 7 | FE | 19 | 2B | 7D | 87 | 92 | AD | EC | 2F | 71 | 93 | AE | E9 | 20 | 60 | A0 |
| 8 | FB | 16 | 3A | 4E | D2 | 6D | B7 | C2 | 5D | E7 | 32 | 56 | FA | 15 | 3F | 41 |
| 9 | C3 | 5E | E2 | 3D | 47 | C9 | 40 | C0 | 5B | ED | 2C | 74 | 9C | BF | DA | 75 |
| A | 9F | BA | D5 | 64 | AC | EF | 2A | 7E | 82 | 9D | BC | DF | 7A | 8E | 89 | 80 |
| B | 9B | B6 | C1 | 58 | E8 | 23 | 65 | AF | EA | 25 | 6F | B1 | C8 | 43 | C5 | 54 |
| C | FC | 1F | 21 | 63 | A5 | F4 | 07 | 09 | 1B | 2D | 77 | 99 | B0 | CB | 46 | CA |
| D | 45 | CF | 4A | DE | 79 | 8B | 86 | 91 | A8 | E3 | 3E | 42 | C6 | 51 | F3 | 0E |
| E | 12 | 36 | 5A | EE | 29 | 7B | 8D | 8C | 8F | 8A | 85 | 94 | A7 | F2 | 0D | 17 |
| F | 39 | 4B | DD | 7C | 84 | 97 | A2 | FD | 1C | 24 | 6C | B4 | C7 | 52 | F6 | 01 |



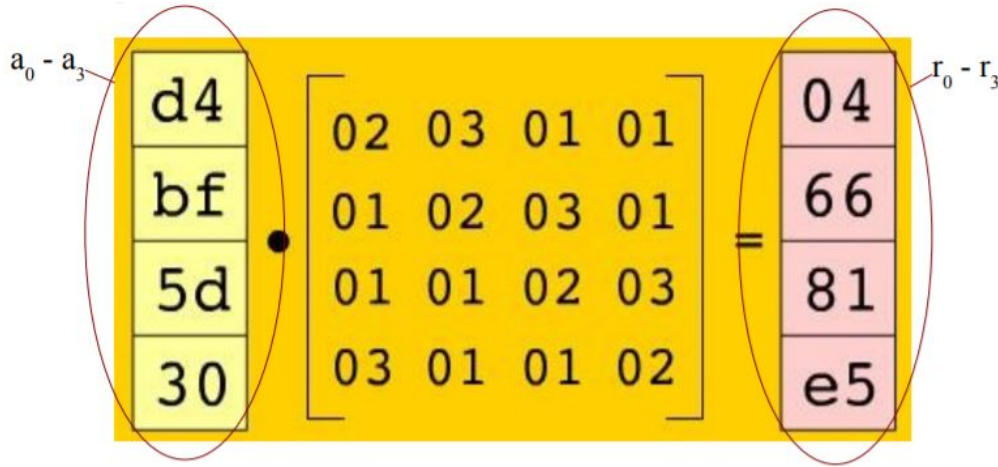
# MixColumn – Example



Input = D4 BF 5D 30

Output(0) = (D4 \* 2) XOR (BF\*3) XOR (5D\*1) XOR (30\*1)  
= E(L(D4) + L(02)) XOR E(L(BF) + L(03)) XOR 5D XOR 30  
= E(41 + 19) XOR E(9D + 01) XOR 5D XOR 30  
= E(5A) XOR E(9E) XOR 5D XOR 30  
= B3 XOR DA XOR 5D XOR 30  
= **04**

# MixColumn – Example


$$\begin{matrix} a_0 - a_3 \\ \begin{bmatrix} d4 \\ bf \\ 5d \\ 30 \end{bmatrix} \end{matrix} \cdot \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} = \begin{matrix} r_0 - r_3 \\ \begin{bmatrix} 04 \\ 66 \\ 81 \\ e5 \end{bmatrix} \end{matrix}$$

- In this example, our  $a_0 - a_3$  is equals to  $d4 - 30$  and  $r_0 - r_3$  is equals to  $04 - e5$ .
- To get the  $r_0$  value, the formula goes like this:  
$$r_0 = \{02.d4\} + \{03.bf\} + \{01.5d\} + \{01.30\}$$

# MixColumn and InvMixColumn

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \xleftrightarrow{\text{Inverse}} \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix}$$

$C \qquad C^{-1}$

# MixColumns Example

|    |    |    |    |
|----|----|----|----|
| 87 | F2 | 4D | 97 |
| 6E | 4C | 90 | EC |
| 46 | E7 | 4A | C3 |
| A6 | 8C | D8 | 95 |

→

|    |    |    |    |
|----|----|----|----|
| 47 | 40 | A3 | 4C |
| 37 | D4 | 70 | 9F |
| 94 | E4 | 3A | 42 |
| ED | A5 | A6 | BC |

# MixColumns Example - Explanation

$$\begin{aligned}(\{02\} \cdot \{87\}) \oplus (\{03\} \cdot \{6E\}) \oplus \{46\} &\oplus \{A6\} = \{47\} \\ \{87\} \oplus (\{02\} \cdot \{6E\}) \oplus (\{03\} \cdot \{46\}) \oplus \{A6\} &= \{37\} \\ \{87\} \oplus \{6E\} \oplus (\{02\} \cdot \{46\}) \oplus (\{03\} \cdot \{A6\}) &= \{94\} \\ (\{03\} \cdot \{87\}) \oplus \{6E\} \oplus \{46\} \oplus (\{02\} \cdot \{A6\}) &= \{ED\}\end{aligned}$$

For the first equation, we have  $\{02\} \cdot \{87\} = (0000\ 1110) \oplus (0001\ 1011) = (0001\ 0101)$  and  $\{03\} \cdot \{6E\} = \{6E\} \oplus (\{02\} \cdot \{6E\}) = (0110\ 1110) \oplus (1101\ 1100) = (1011\ 0010)$ . Then,

$$\begin{aligned}\{02\} \cdot \{87\} &= 0001\ 0101 \\ \{03\} \cdot \{6E\} &= 1011\ 0010 \\ \{46\} &= 0100\ 0110 \\ \{A6\} &= \underline{1010\ 0110} \\ &0100\ 0111 = \{47\}\end{aligned}$$

## Step 4: AddRoundKey

---

- Adding the round key to the output of the previous step during the forward process.
- The corresponding step during decryption is denoted **InvAddRoundKey**.

# Step 4: AddRoundKey Example

|    |    |    |    |
|----|----|----|----|
| 47 | 40 | A3 | 4C |
| 37 | D4 | 70 | 9F |
| 94 | E4 | 3A | 42 |
| ED | A5 | A6 | BC |

 $\oplus$ 

|    |    |    |    |
|----|----|----|----|
| AC | 19 | 28 | 57 |
| 77 | FA | D1 | 5C |
| 66 | DC | 29 | 00 |
| F3 | 21 | 41 | 6A |

 $=$ 

|    |    |    |    |
|----|----|----|----|
| EB | 59 | 8B | 1B |
| 40 | 2E | A1 | C3 |
| F2 | 38 | 13 | 42 |
| 1E | 84 | E7 | D6 |

# AES Key Scheduling

---

Subkey / Round key Generation



# AES Key Expansion

- ➡ Each round has its own key derived from original 128-bit encryption key.
- ➡ One of the four steps – XORing this round key with the state array.
- ➡ AES Key Expansion algorithm is used to derive the 128-bit round key for each round from the original 128-bit encryption key.
- ➡ Algorithm first arranges the 16 bytes of the encryption key in the form of a 4×4 array of bytes.

$$\begin{bmatrix} k_0 & k_4 & k_8 & k_{12} \\ k_1 & k_5 & k_9 & k_{13} \\ k_2 & k_6 & k_{10} & k_{14} \\ k_3 & k_7 & k_{11} & k_{15} \end{bmatrix}$$



$$\begin{bmatrix} w_0 & w_1 & w_2 & w_3 \end{bmatrix}$$

- The first four bytes of the encryption key constitute the word  $w_0$ , the next four bytes the word  $w_1$ , and so on.
- The algorithm subsequently expands the words  $[w_0, w_1, w_2, w_3]$  into a 44-word **key schedule** that can be labeled

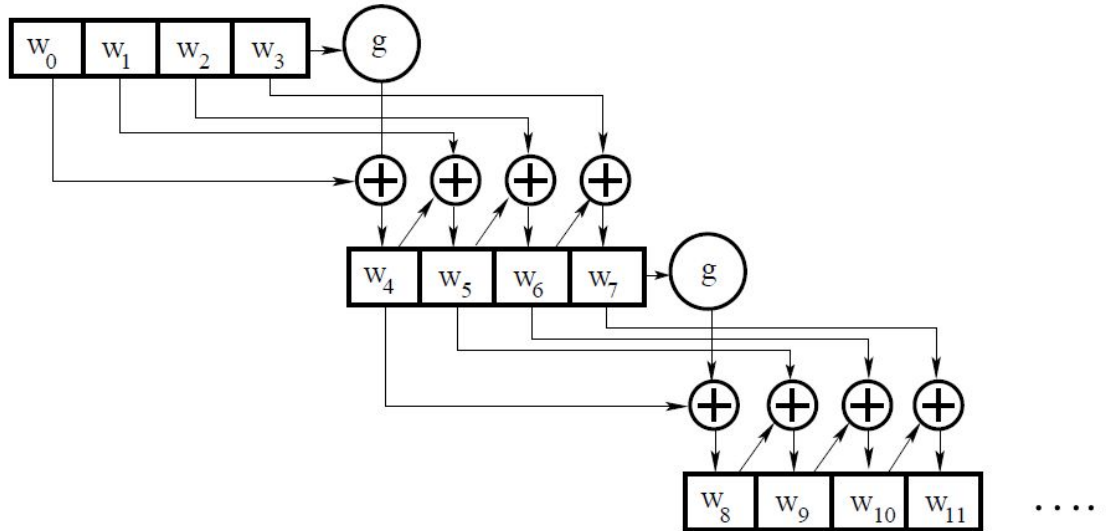
$w_0, w_1, w_2, w_3, \dots, w_{43}$ .

• How?

- Of these, the words  $[w_0, w_1, w_2, w_3]$  are bitwise XOR'ed with the input block before the round-based processing begins.
- The remaining 40 words of the key schedule are used 4 words at a time in each of the 10 rounds.

# Key Expansion Algorithm

- The key expansion takes place on a four-word to four-word basis.



# Key Expansion Algorithm

- The function **g** consists of the following subfunctions.
  - 1) **RotWord** performs a one-byte circular left shift on a word.
  - 2) **SubWord** performs a byte substitution on each byte of its input word, using the S-box (16 x 16 lookup table used in **SubBytes** step).
  - 3) The result of steps 1 and 2 is XORed with a round constant, **Rcon[i]**.
- The round constant is a word in which the three rightmost bytes are always 0.

| j     | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|-------|----|----|----|----|----|----|----|----|----|----|
| RC[j] | 01 | 02 | 04 | 08 | 10 | 20 | 40 | 80 | 1B | 36 |

- **Goal:** To destroy any symmetries that may have been introduced by the other steps.

# Key Expansion Calculation

For example, suppose that the round key for round 8 is

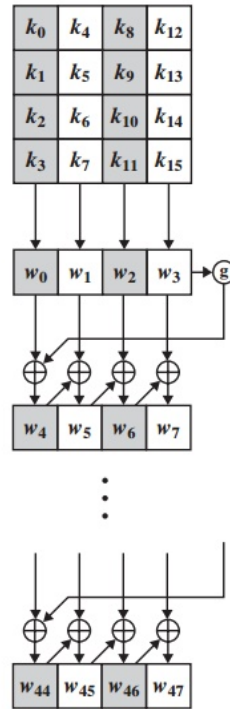
EA D2 73 21 B5 8D BA D2 31 2B F5 60 7F 8D 29 2F

Then the first 4 bytes (first column) of the round key for round 9 are calculated as follows:

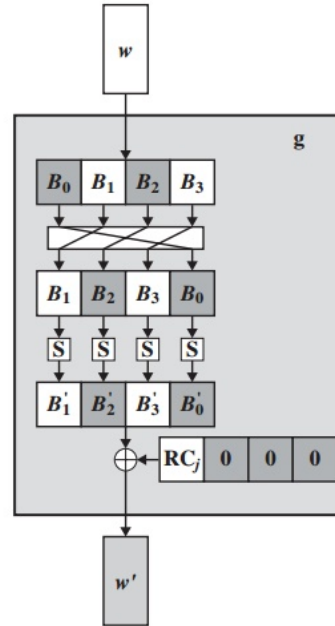
| i (decimal) | temp     | After RotWord | After SubWord | Rcon (9) | After XOR with Rcon | w[i-4]   | w[i] = temp $\oplus$ w[i-4] |
|-------------|----------|---------------|---------------|----------|---------------------|----------|-----------------------------|
| 36          | 7F8D292F | 8D292F7F      | 5DA515D2      | 1B000000 | 46A515D2            | EAD27321 | AC7766F3                    |

- ➡ Words in positions that are a multiple of 4 ( $w_4, w_8, \dots, w_{40}$ ) are calculated by:
  - Applying the RotWord and SubBytes transformation to the previous word  $w_{i-1}$ .
- ➡ Adding (XOR) this result to the word 4 positions earlier  $w_{i-4}$ , plus a round constant  $R_{con}$ .
- ➡ The remaining 32 bit words  $w_i$ , are calculated by adding (XOR) the previous word  $w_{i-1}$ , with the word 4 positions earlier  $w_{i-4}$ .

# AES Key Expansion - Diagram

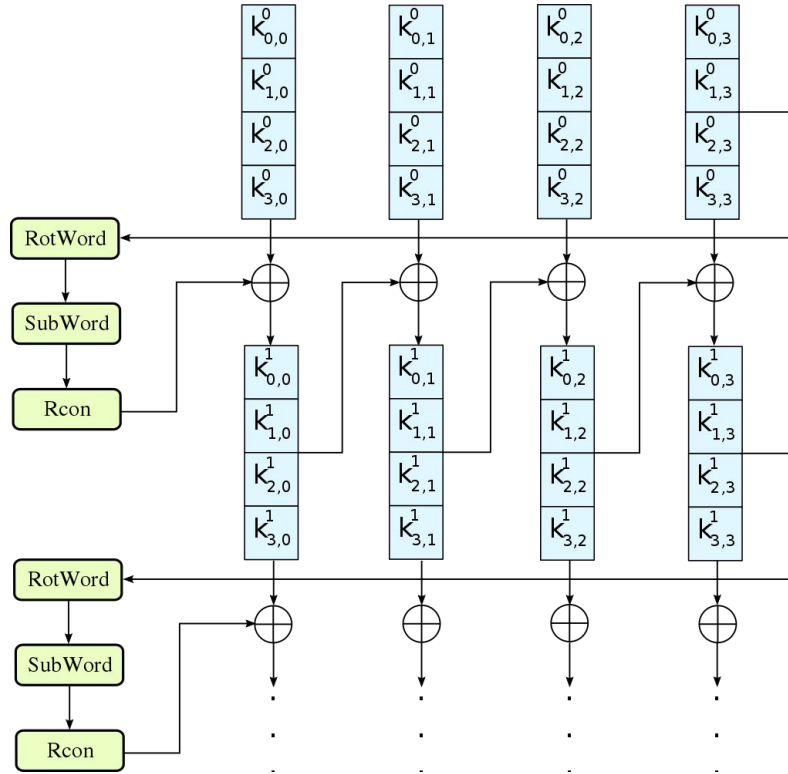


(a) Overall algorithm



(b) Function  $g$

# AES Key Expansion - Diagram





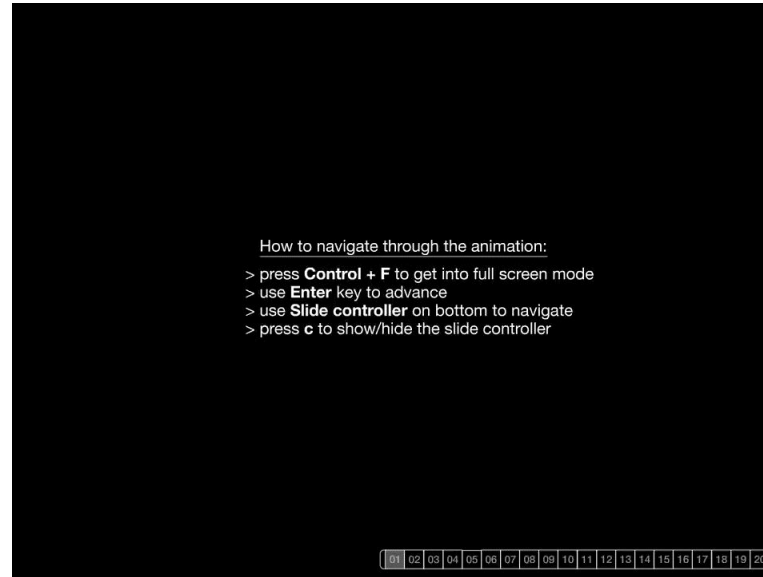
# AES Avalanche Effect: Plaintext

| Round |                                                                        | Number of bits that differ |
|-------|------------------------------------------------------------------------|----------------------------|
|       | 0123456789abcdef fedcba9876543210<br>0023456789abcdef fedcba9876543210 | 1                          |
| 0     | 0e3634aece7225b6f26b174ed92b5588<br>0f3634aece7225b6f26b174ed92b5588   | 1                          |
| 1     | 657470750fc7ff3fc0e8e8ca4dd02a9c<br>c4a9ad090fc7ff3fc0e8e8ca4dd02a9c   | 20                         |
| 2     | 5c7bb49a6b72349b05a2317ff46d1294<br>fe2ae569f7ee8bb8c1f5a2bb37ef53d5   | 58                         |
| 3     | 7115262448dc747e5cdac7227da9bd9c<br>ec093dfb7c45343d689017507d485e62   | 59                         |
| 4     | f867aee8b437a5210c24c1974cffeabc<br>43efdb697244df808e8d9364ee0ae6f5   | 61                         |
| 5     | 721eb200ba06206dcbd4bce704fa654e<br>7b28a5d5ed643287e006c099bb375302   | 68                         |
| 6     | 0ad9d85689f9f77bc1c5f71185e5fb14<br>3bc2d8b6798d8ac4fe36ald891ac181a   | 64                         |
| 7     | db18a8ffa16d30d5f88b08d777ba4eaa<br>9fb8b5452023c70280e5c4bb9e555a4b   | 67                         |
| 8     | f91b4fbfe934c9bf8f2f85812b084989<br>20264e1126b219aef7feb3f9b2d6de40   | 65                         |
| 9     | cca104a13e678500ff59025f3bafaa34<br>b56a0341b2290ba7dfdfbddcd8578205   | 61                         |
| 10    | ff0b844a0853bf7c6934ab4364148fb9<br>612b89398d0600cde116227ce72433f0   | 58                         |

40

# AES Avalanche Effect: Key

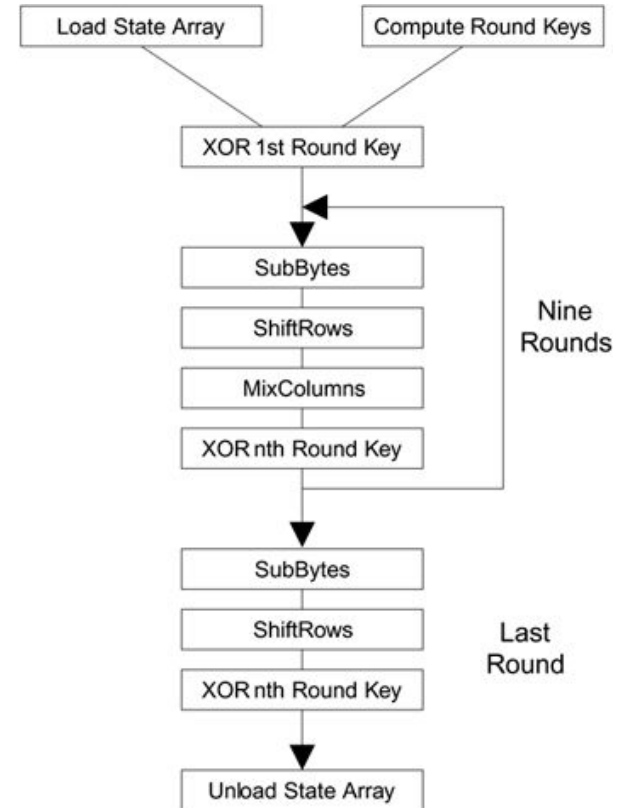
| Round |                                                                      | Number of Bits that Differ |
|-------|----------------------------------------------------------------------|----------------------------|
|       | 0123456789abcdeffedcba9876543210<br>0123456789abcdeffedcba9876543210 | 0                          |
| 0     | 0e3634aece7225b6f26b174ed92b5588<br>0f3634aece7225b6f26b174ed92b5588 | 1                          |
| 1     | 657470750fc7ff3fc0e8e8ca4dd02a9c<br>c5a9ad090ec7ff3fc1e8e8ca4cd02a9c | 22                         |
| 2     | 5c7bb49a6b72349b05a2317ff46d1294<br>90905fa9563356d15f3760f3b8259985 | 58                         |
| 3     | 7115262448dc747e5cdac7227da9bd9c<br>18aeb7aa794b3b66629448d575c7cebf | 67                         |
| 4     | f867aee8b437a5210c24c1974cffeabc<br>f81015f993c978a876ae017cb49e7eec | 63                         |
| 5     | 721eb200ba06206dcbd4bce704fa654e<br>5955c91b4e769f3cb4a94768e98d5267 | 81                         |
| 6     | 0ad9d85689f9f77bc1c5f71185e5fb14<br>dc60a24d137662181e45b8d3726b2920 | 70                         |
| 7     | db18a8ffa16d30d5f88b08d777ba4eaa<br>fe8343b8f88bef66cab7e977d005a03c | 74                         |
| 8     | f91b4fbfe934c9bf8f2f85812b084989<br>da7dad581d1725c5b72fa0f9d9d1366a | 67                         |
| 9     | cca104a13e678500ff59025f3bafaa34<br>0ccb4c66bbfd912f4b511d72996345e0 | 59                         |
| 10    | ff0b844a0853bf7c6934ab4364148fb9<br>fc8923ee501a7d207ab670686839996b | 53                         |



<https://legacy.cryptool.org/en/cto/aes-animation>

# Summary of AES Algorithm

- AES encrypts 128-bit blocks with 128-bit, 192-bit, or 256-bit using 10, 12, or 14 rounds, respectively.
- Is not a Feistel cipher => All 128 bits are encrypted.
- Each round = 4 steps of SubBytes, ShiftRows, MixColumns, and AddRoundKey.
- Last round has only 3 steps. No MixColumns.
- Decryption is not the same as encryption (as in DES). It consists of inverse steps.



# Question 1

1. AES uses a \_\_\_\_\_ bit block size and a key size of \_\_\_\_\_ bits.

- a) 128; 128 or 256
- b) 64; 128 or 192
- c) 256; 128, 192, or 256
- d) 128; 128, 192, or 256**

**Explanation:** It uses a 128-bit block size and a key size of 128, 192, or 256 bits.

# Question 2

---

**2. Like DES, AES also uses Feistel Structure.**

A. True

**B. False**

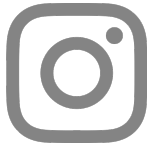
# Question 3

## 3. Match the following

|              |          |
|--------------|----------|
| 1) $ K =128$ | A. Nr=12 |
| 2) $ K =256$ | B. Nr=14 |
| 3) $ K =192$ | C. Nr=10 |

# Thank you!

## Follow us



isfcr.pesu



[www.isfcr.pes.edu](http://www.isfcr.pes.edu)



ISFCR



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience







PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



Welcome to  
**PES University**  
Ring Road Campus, Bengaluru



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



# Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 17

# A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

# A Note on Security

- ➡ In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- ➡ To be clear, **you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network** without the express consent of the owner.
- ➡ In particular, **you will comply with all my instructions when doing the labs.**
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- ➡ Any violation is at **YOUR RISK!**  
**And may result in severe consequences.**

# Modes of Operation- Block Ciphers

---

- ➡ The modes of operation of block ciphers are configuration methods that allow those ciphers to work with large data streams, without the risk of compromising the provided security.
- ➡ A mode of operation is essentially a way of encrypting arbitrary-length messages using a block cipher.

- ➡ We just do not run a cipher- modes of operation
- ➡ Fixed length block
- ➡ 'b' bits input and 'b' bits output
- ➡ If the amount of PT to be encrypted is  $>b$  bits??
- ➡ Break the PT into b bits block each
- ➡ NIST defines 5 modes of operation
- ➡ Different applications use different modes of operation

# Types of Modes of Operations

---

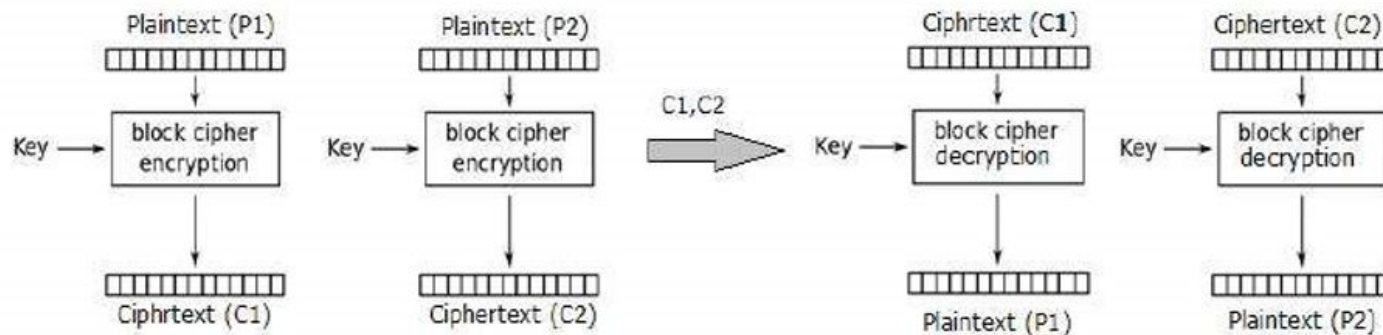
- **ECB** – Electronic Codebook
- **CBC** – Cipher Block Chaining
- **CTR** – Counter Mode
- **GCM** – Galois/Counter Mode
- **CCM** – Counter with CBC-MAC

# Electronic code book (ECB) mode

➡ Electronic code book. Given a plaintext message  $m = m_1, \dots, m_i$ , the ciphertext is obtained by “**encrypting**” each block **separately**, i.e.,  $c = \{ F_k(m_1), \dots, F_k(m_i) \}$ .

## ➡ Operation

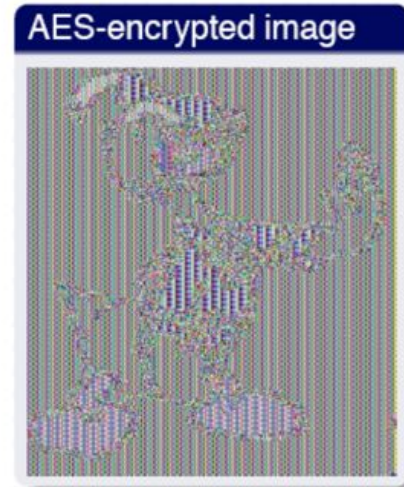
- The user takes the first block of plaintext and encrypts it with the key to produce the first block of ciphertext.
- He then takes the second block of plaintext and follows the same process with same key and so on so forth.





# Electronic code book (ECB) mode

- ✎ ECB is deterministic and therefore cannot be CPA-secure.
- ✎ Worse, ECB-mode encryption does not even have indistinguishable encryptions in the presence of eavesdroppers.
- ✎ Apart from revealing the hints regarding the content of plaintext, the ciphers that are used in ECB mode are also more vulnerable to replay attacks.
- ✎ \*Uncompressed bitmap format encrypted using AES in ECB mode.



# Electronic code book (ECB) mode

---

## ECB Encryption

1. **Divide** the plaintext into fixed-size blocks.
2. **Use the same secret key** for all blocks.
3. **Encrypt each block independently** using AES.
4. Each plaintext block produces a **corresponding ciphertext block**.
5. No chaining or dependency exists between blocks.
6. **Same plaintext block** → **Same ciphertext block**.
7. Simple and fast, but **reveals patterns** in the plaintext.

**Formula:**  $C_i = \text{AES}(K, P_i)$

## ECB Decryption

1. **Divide** the ciphertext into fixed-size blocks.
2. **Use the same secret key** used during encryption.
3. **Decrypt each block independently** using AES.
4. Each ciphertext block produces the **corresponding plaintext block**.
5. There is **no chaining** between blocks.
6. Combine all decrypted blocks to obtain the **original plaintext**.

**Formula:**  $P_i = \text{AES}^{-1}(K, C_i)$

# Electronic code book (ECB) mode

---

## Pros

*Simplest mode*

*Ideal for small amount of data.*

*Independent*

*Fast-Parallelism*

## Cons

*Not secure for lengthy messages*

*Cryptanalyst can exploit the regularities of the message.*

# Electronic code book (ECB) mode



Original Image



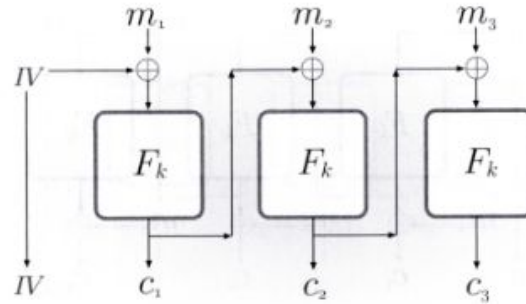
Encrypted using  
ECB



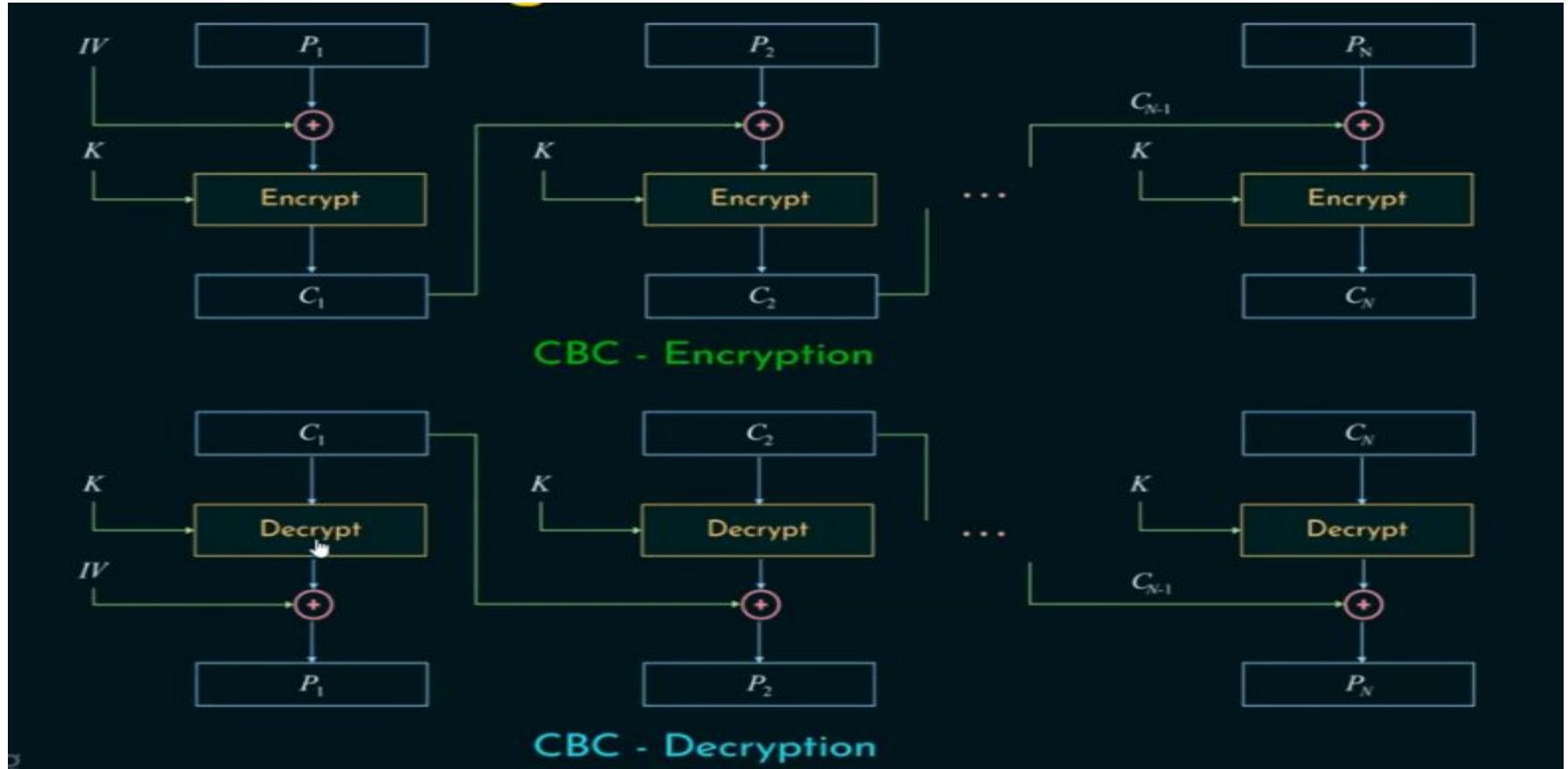
Encrypted using  
other modes

# CBC (Cipher-Block Chaining) Mode

- 👉 The CBC encryption mode was invented in IBM in 1976.
- 👉 We choose a random initial vector (IV ) of length  $n$ .
- 👉 The ciphertext is obtained by applying the pseudorandom permutation to the XOR of the current plaintext block and the previous ciphertext block.
- 👉 That is, we set  $c_0 = IV$  and then, for  $i = 1$  to  $\ell$ , set  $c_i = F_k (c_{i-1} \oplus m_i)$ .



# CBC (Cipher-Block Chaining) Mode



# CBC (Cipher-Block Chaining) Mode

## CBC Encryption

1. **Divide** the plaintext into fixed-size blocks.
2. Generate a random **Initialization Vector (IV)**.
3. **XOR** the first plaintext block with the IV.
4. **Encrypt** the result using AES and the secret key.
5. For the next blocks, **XOR** each plaintext block with the previous ciphertext block.
6. **Encrypt** the XOR result to generate the ciphertext block.
7. Repeat the process for all plaintext blocks.
8. The final output is the **ciphertext + IV**.

**Formula:**  $C_1 = \text{AES}(K, P_1 \oplus IV)$

**For next blocks:**  $C_i = \text{AES}(K, P_i \oplus C_{i-1})$



# CBC (Cipher-Block Chaining) Mode

## CBC Decryption

1. **Divide** the ciphertext into fixed-size blocks.
2. Use the same **secret key and IV** used during encryption.
3. **Decrypt** the first ciphertext block using AES.
4. **XOR** the decrypted result with the IV to obtain the first plaintext block.
5. For the next blocks, **decrypt** the ciphertext block using AES.
6. **XOR** the decrypted result with the previous ciphertext block.
7. Repeat the process for all ciphertext blocks.
8. The final output is the **original plaintext**.

**Formula:**  $P_1 = \text{AES}^{-1}(K, C_1) \oplus IV$

**For next blocks:**  $P_i = \text{AES}^{-1}(K, C_i) \oplus C_{i-1}$

# CBC (Cipher-Block Chaining) Mode

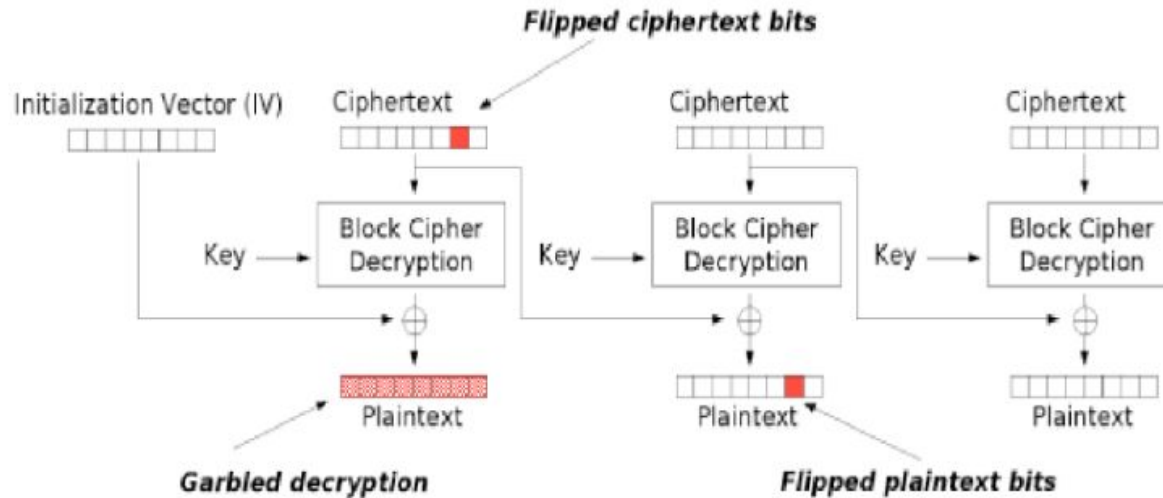
- ➡ CBC mode of operation provides message dependence for generating ciphertext and makes the system non-deterministic.
- ➡ Encryption in CBC is probabilistic, and it has been proven that if  $F$  is a pseudorandom permutation, then CBC-mode encryption is CPA-secure.
- ➡ Encryption must be carried out sequentially.
- ➡ If parallel processing is available CBC may not be the most efficient choice.
- ➡ The IV should be integrity-protected.
- ➡ The IV can be sent in the cleartext.



**Donald disappears\***

# CBC Error Propagation

- ➡ The error in transmission gets propagated to few further block during decryption due to chaining effect.



Modification attack or transmission error for CBC

## Pros

*Appropriate mode for encrypting messages of the length greater than  $b$  bits*

*Confidentiality + Authentication*

## Cons

*Slow- no parallelism*

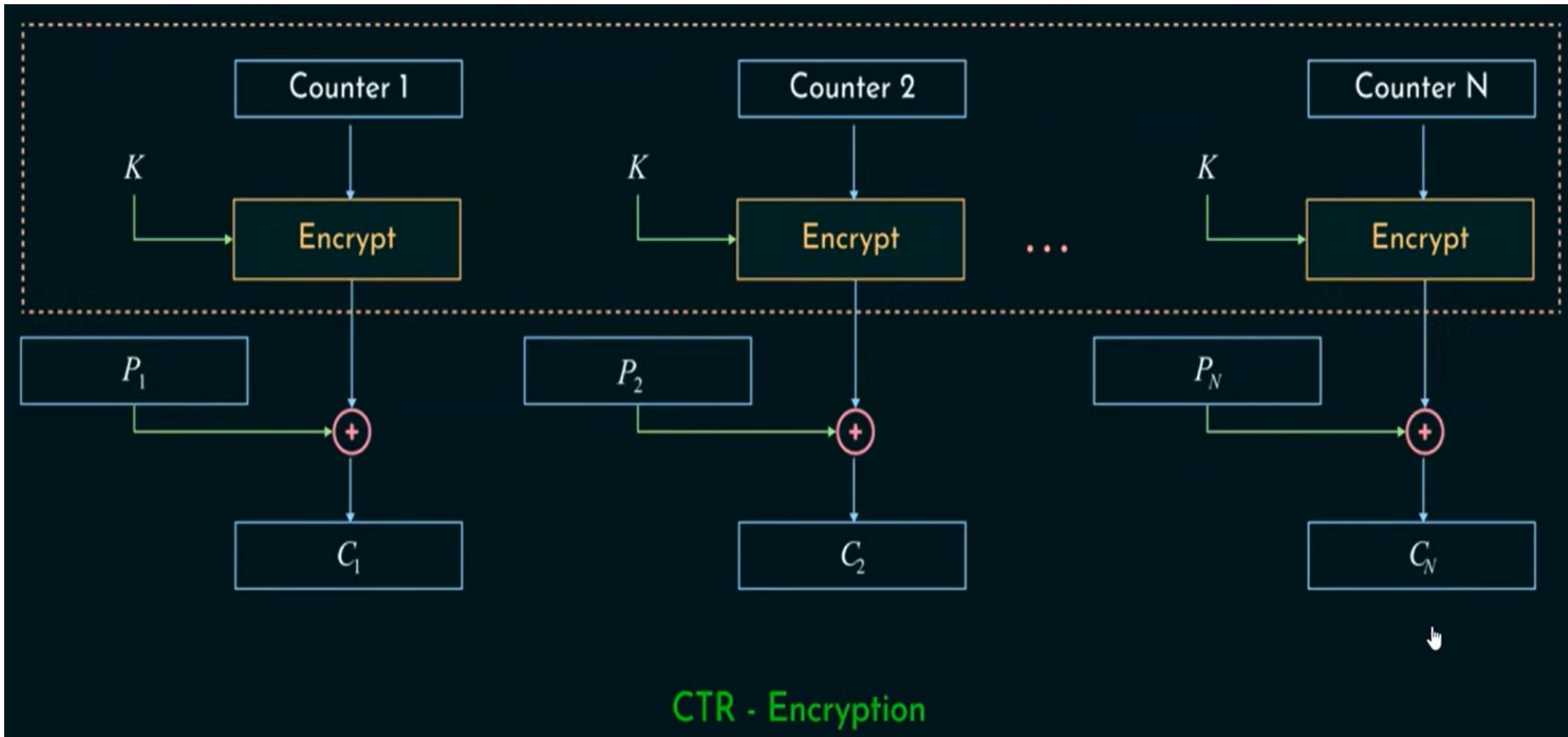
*Cannot encrypt any block since we need the ciphertext of previous block*

# CTR (Counter) Mode

---

- ➡ It can be considered as a counter-based version of CFB mode without the feedback.
- ➡ In this mode, both the sender and receiver need to access to a reliable counter, which computes a new shared value each time a ciphertext block is exchanged.
- ➡ This shared counter is not necessarily a secret value, but challenge is that both sides must keep the counter synchronized.

# CTR (Counter) Mode

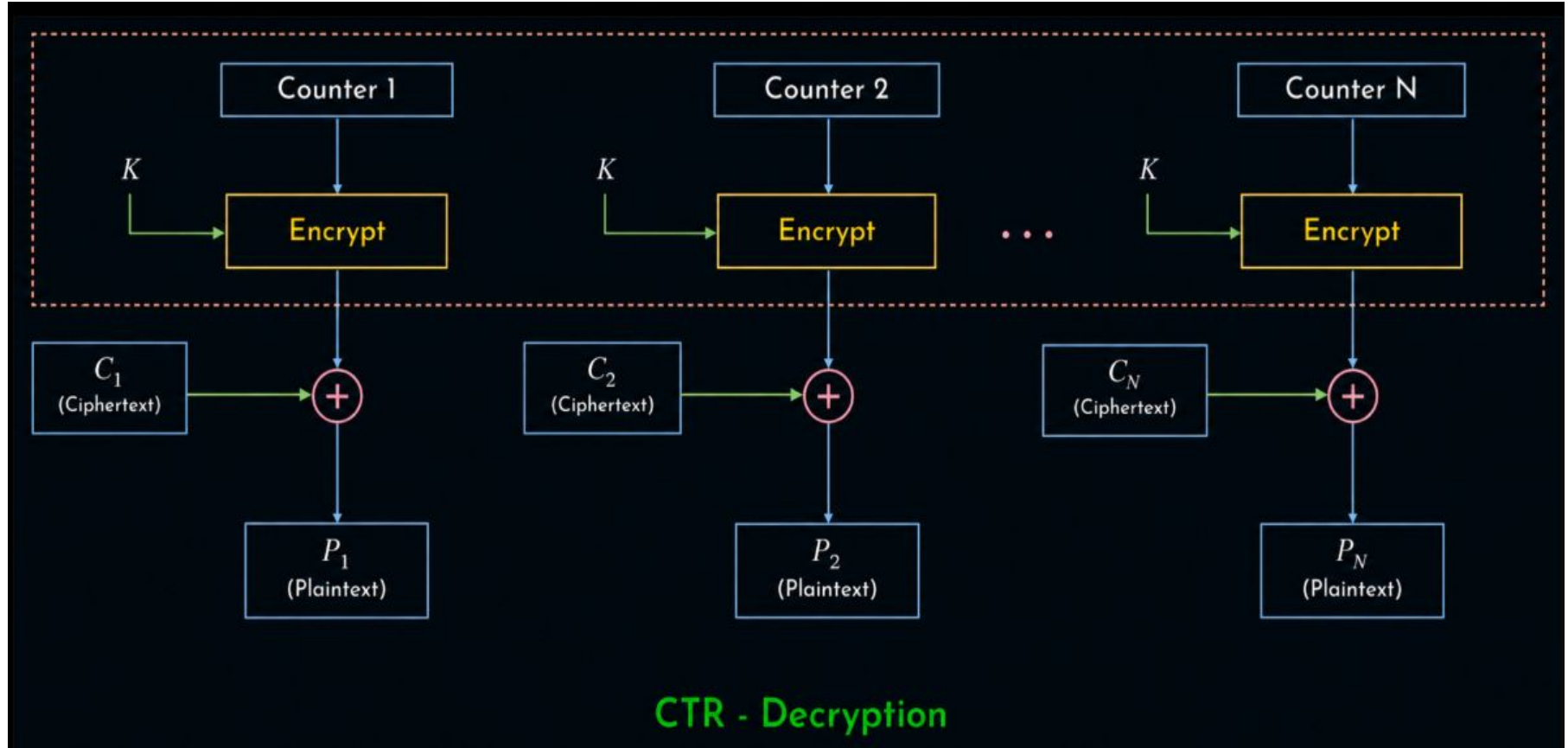


## CTR Encryption

1. **Divide** the plaintext into fixed-size blocks.
2. Generate a **nonce/initial counter value**.
3. Combine the nonce with a **counter value** for each block.
4. Encrypt the counter using **AES and the secret key** to generate a keystream.
5. **XOR** the plaintext block with the keystream.
6. Increase the counter for the next block.
7. Repeat the process for all plaintext blocks.
8. The final output is the **ciphertext**.

**Formula:**  $C_i = P_i \oplus \text{AES}(K, \text{Counter}_i)$

# CTR (Counter) Mode





## CTR Decryption

1. **Divide** the ciphertext into fixed-size blocks.
2. Use the same **secret key and nonce** used during encryption.
3. Generate the same **counter values** for each block.
4. Encrypt each counter using **AES** to generate the keystream.
5. **XOR** the ciphertext block with the keystream.
6. Repeat the process for all ciphertext blocks.
7. The final output is the **original plaintext**.

**Formula:**  $P_i = C_i \oplus \text{AES}(K, \text{Counter}_i)$

- ➡ It is one of the most popular block ciphers modes of operation.
- ➡ Both encryption and decryption can be performed using many threads at the same time.
- ➡ If one bit of a plaintext or ciphertext message is damaged, only one corresponding output bit is damaged as well.
- ➡ Thus, it is possible to use various correction algorithms to restore the previous value of damaged parts of received messages.



## Efficiency

- One can do parallel encryptions
- Good for bursty high speed links
  - Random access to encrypted data blocks
  - Provable security (good as other modes)
  - but Must ensure never to reuse key/counter values

# Why do we need GCM/CCM?

Traditional encryption gives confidentiality:

$$P \xrightarrow{\text{Encryption}} C$$

The attacker cannot easily read the plaintext.

But what if the attacker changes the ciphertext?

For example:

Original ciphertext:

10110110 01101001 ...

Attacker modifies:

10110110 11101001 ...



changed bit

If there is no authentication mechanism, the receiver may decrypt the modified ciphertext.

So modern systems want:

Confidentiality + Integrity + Authentication

This is called:

Authenticated Encryption

GCM and CCM are two important **authenticated encryption modes** used in modern cryptographic systems. Unlike traditional encryption modes that focus only on hiding information, these modes also help ensure that the transmitted data has not been modified.

They are commonly used with the **Advanced Encryption Standard (AES)** and provide a combination of:

- **Confidentiality** – protecting data from unauthorized access
- **Integrity** – detecting changes to data
- **Authentication** – verifying the validity of the message

Both GCM and CCM belong to the category of **AEAD (Authenticated Encryption with Associated Data)** algorithms.

# Additional Authenticated Data (AAD)

---

Both GCM and CCM support **Additional Authenticated Data**.

AAD refers to information that:

- Should be authenticated
- Does not need to be encrypted

For example, a communication protocol may contain a header that must remain visible to intermediate systems but must not be modified.

In this case:

Header / Metadata → Authenticated but not encrypted

Message Data → Authenticated and encrypted

This provides flexibility because important protocol information can remain accessible while still being protected against unauthorized modification.

## Galois/Counter Mode

GCM is an authenticated encryption mode that combines **Counter Mode (CTR)** encryption with an authentication mechanism based on operations in a mathematical structure called a **Galois Field**.

The encryption process uses Counter Mode to convert plaintext into ciphertext. At the same time, GCM calculates an authentication value called an **authentication tag**.

The authentication tag is generated using a function called **GHASH**, which processes the ciphertext and any additional authenticated information.

Thus, GCM provides:

- Encryption of sensitive information
- Authentication of the message
- Detection of unauthorized modifications
- Authentication of additional metadata

GCM is particularly popular because it is designed for **high performance**.

GCM consists mainly of two components:

## 1. Counter Mode (CTR)

Counter Mode is responsible for encrypting the plaintext. A sequence of counter values is encrypted using AES, and the resulting output is combined with the plaintext to produce ciphertext.

One major advantage of Counter Mode is that different blocks can be processed efficiently, making it suitable for high-speed systems.

## 2. GHASH

GHASH is responsible for generating authentication information. It processes the ciphertext and any Additional Authenticated Data to calculate an authentication value.

The final result is used to generate the **authentication tag**, which allows the receiver to verify the integrity and authenticity of the message.



# GCM Encryption Process

---

## Step 1: Generate a Nonce

A nonce or initialization value is selected. The nonce must be unique for every encryption operation performed with the same key.

## Step 2: Encrypt the Plaintext

The plaintext is encrypted using AES in Counter Mode. This produces the ciphertext.

## Step 3: Process Authentication Data

Any Additional Authenticated Data (AAD), along with the ciphertext, is processed using GHASH.

AAD may include information such as headers or metadata that should be authenticated but does not need to be encrypted.

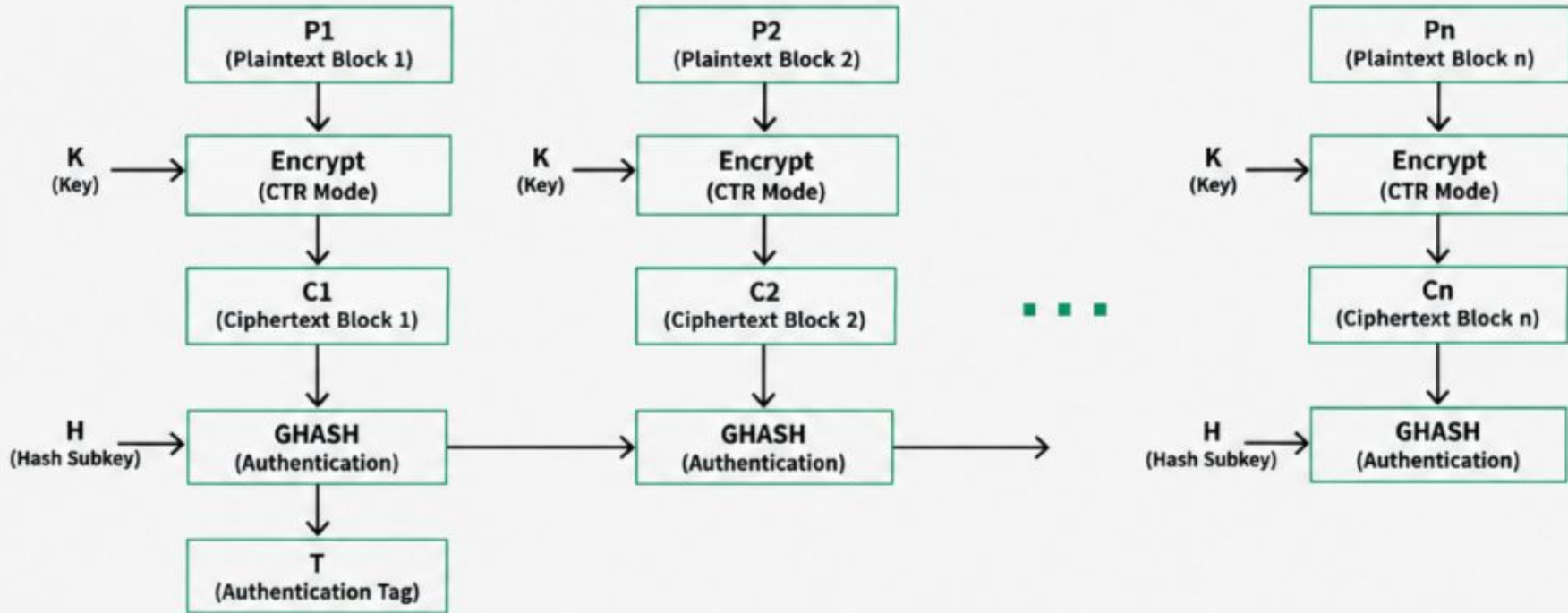
## Step 4: Generate Authentication Tag

The output of GHASH is combined with an encrypted counter value to generate the final authentication tag.

The sender then transmits: **Nonce + Ciphertext + Authentication Tag**

# GCM Encryption Process

## GCM – Encryption



## 1. Receive the encrypted message

The receiver gets the **ciphertext**, **authentication tag**, and **Additional Authenticated Data (AAD)**.

## 2. Use the shared secret key and nonce

The receiver uses the same **secret key** and **nonce** that were used during encryption.

## 3. Calculate the expected authentication tag

Using the received ciphertext and AAD, the receiver calculates a new authentication tag.

## 4. Compare both authentication tags

The receiver compares:

- **Received Authentication Tag**
- **Calculated Authentication Tag**

## 5. Verify authenticity and integrity

- If both tags **match**, the message is considered authentic and unchanged.
- If the tags **do not match**, authentication has failed.

## 6.Decrypt the ciphertext

Only after successful authentication, the receiver decrypts the ciphertext to obtain the **original plaintext**.

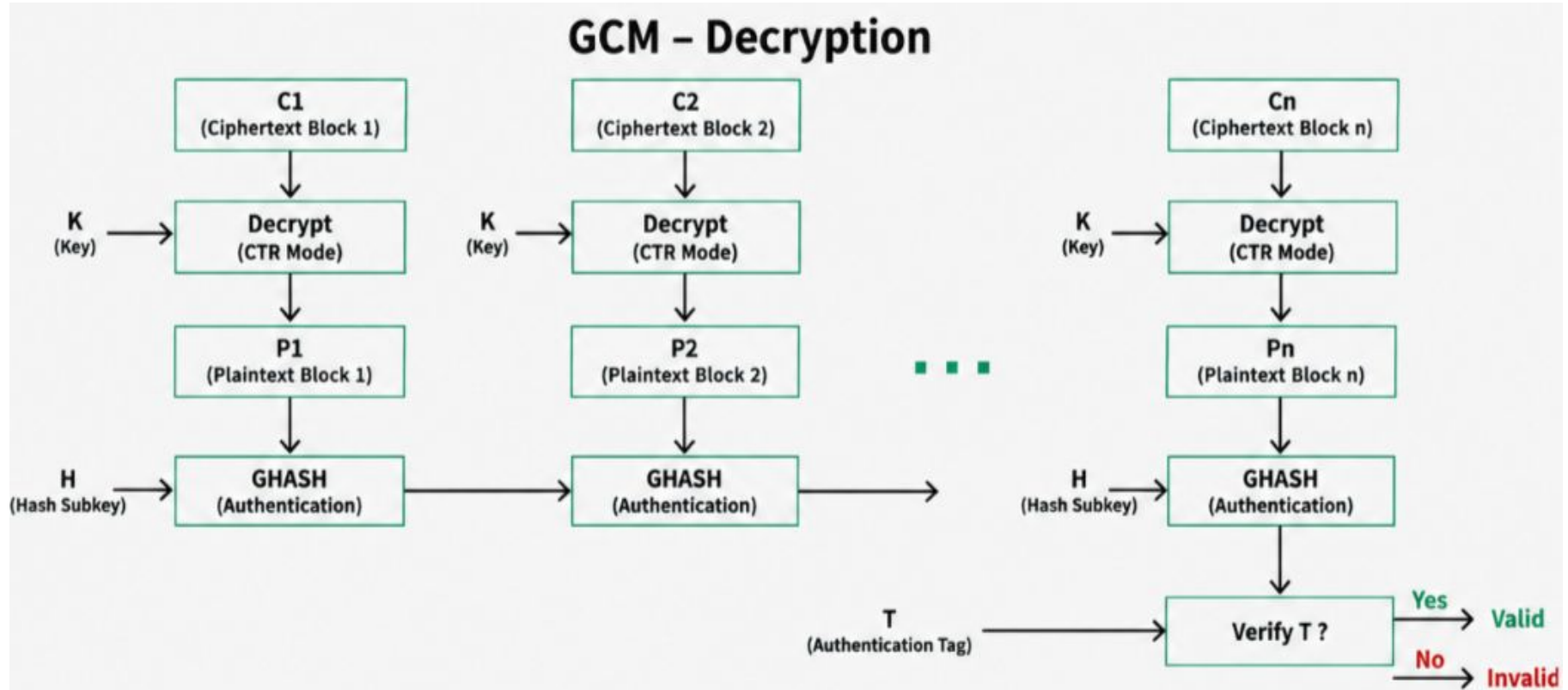
## 7.Reject invalid messages

If authentication fails, the message must be rejected because it may have been **modified, corrupted, or sent by an unauthorized source**.

## 8.Final result

This process ensures **confidentiality and integrity together**, which is a key feature of authenticated encryption.

# GCM Decryption and Verification



GCM is widely used because of several important properties:

- It provides encryption and authentication in a single construction.
- It is computationally efficient.
- It supports parallel processing effectively.
- It can authenticate additional data without encrypting it.
- It is suitable for high-speed communication systems.

CCM is another authenticated encryption mode designed to provide both confidentiality and authentication.

CCM combines two cryptographic techniques:

## **CBC-MAC**

Used to calculate authentication information and verify the integrity of the message.

## **Counter Mode (CTR)**

Used to encrypt the plaintext and provide confidentiality.

The name **CCM** comes from the combination of: **Counter Mode + CBC-MAC**

Like GCM, CCM can be used with AES and supports both encrypted data and Additional Authenticated Data.

## CBC-MAC

CBC-MAC is used to calculate a Message Authentication Code based on the plaintext and associated data.

This value helps the receiver verify whether the message has been modified.

## Counter Mode

After authentication information is generated, Counter Mode is used to encrypt the plaintext.

Counter Mode produces a sequence of encrypted counter values that are combined with the plaintext to generate ciphertext.

The authentication information is also protected as part of the CCM construction.



# CCM Encryption Process

## Step 1: Prepare the Data

The plaintext, nonce and Additional Authenticated Data are prepared.

## Step 2: Generate Authentication Value

CBC-MAC processes the message and authenticated data to generate an authentication value.

## Step 3: Encrypt the Plaintext

The plaintext is encrypted using AES in Counter Mode.

## Step 4: Protect the Authentication Tag

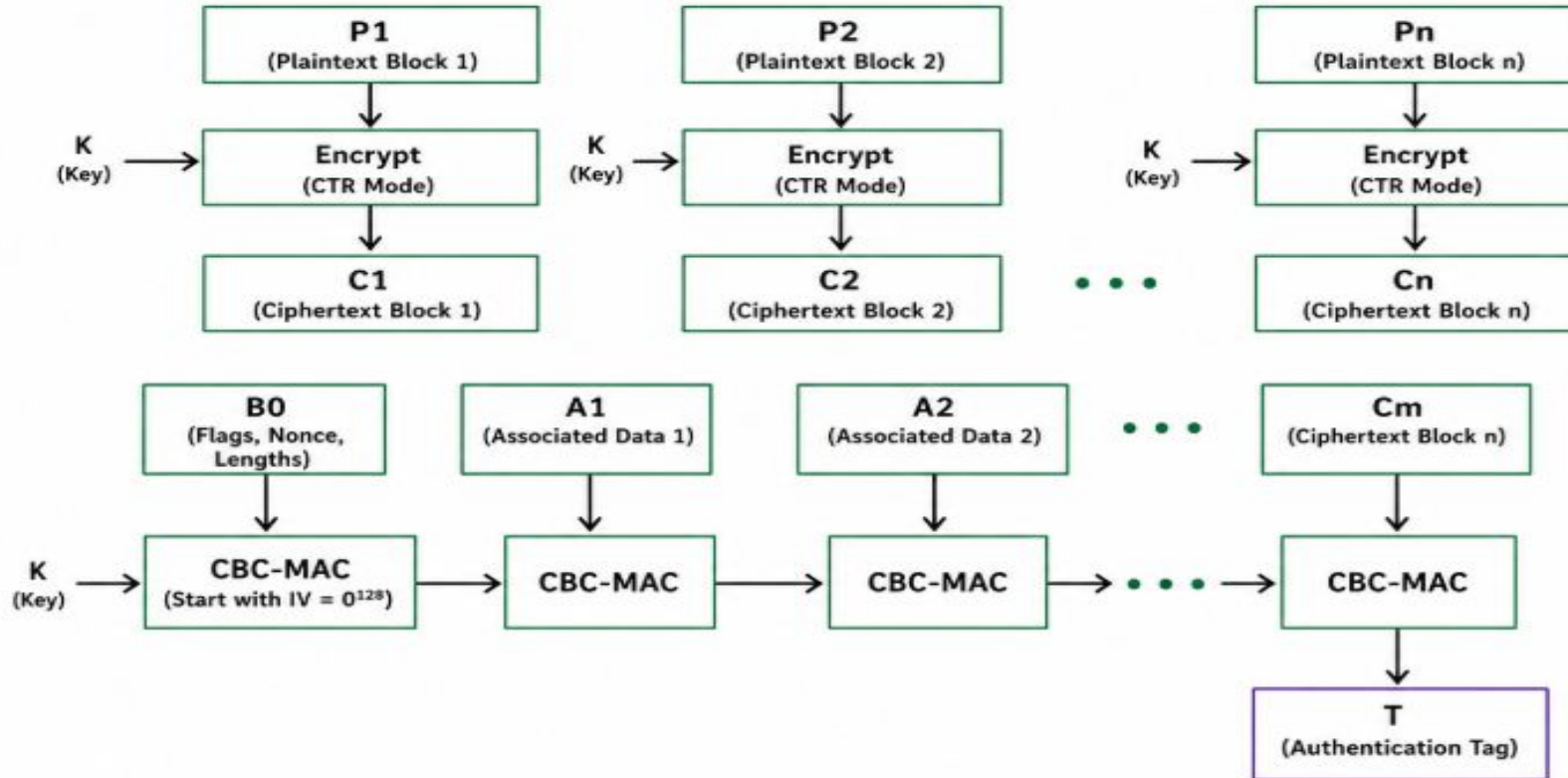
The authentication value is combined with the Counter Mode output to generate the transmitted authentication tag.

The sender then sends:

**Nonce + Ciphertext + Authentication Tag**

At the receiver side, the tag is verified before the message is accepted.

## CCM – Encryption



## 1. Receive the Data

Receiver gets **Nonce**, **Ciphertext**, **Authentication Tag**, and **AAD**.

## 2. Decrypt the Ciphertext

AES-CTR mode uses the **secret key and nonce** to decrypt the ciphertext and recover the **plaintext**.

## 3. Process Plaintext and AAD

The receiver combines the recovered **plaintext** with the **Additional Authenticated Data (AAD)**.

## 4. Calculate Authentication Tag

**CBC-MAC** is applied to the plaintext and AAD to generate a **calculated authentication tag**.

## 5. Compare the Tags

The calculated tag is compared with the **received authentication tag**.

## 6. Tag Matches → Accept

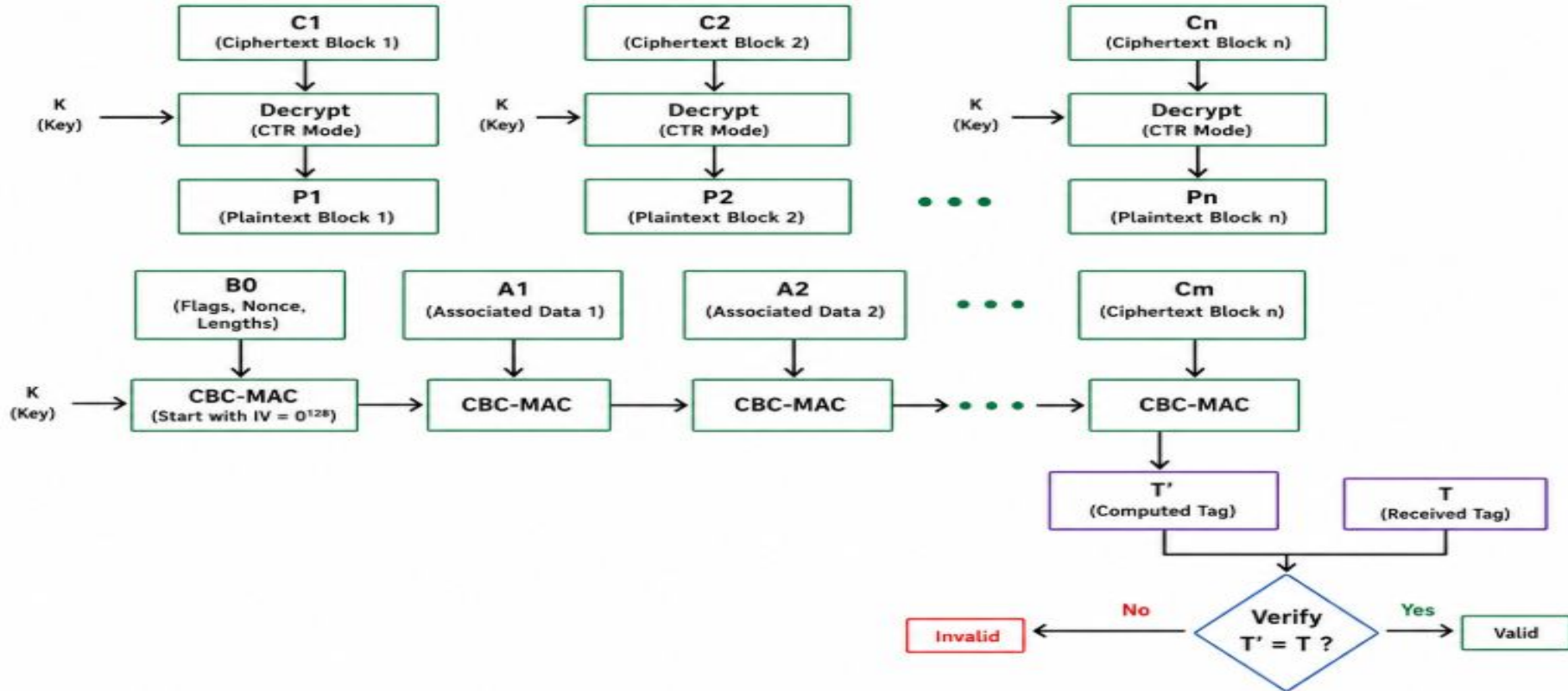
The message is considered **authentic and unchanged**, and the plaintext is accepted.

## 7. Tag Does Not Match → Reject

The message is considered **modified or invalid** and is rejected.

# CCM Decryption

## CCM – Decryption



## 1. Wireless Networks

Used in **IEEE 802.15.4** and other wireless communication systems to secure data.

## 2. IoT Devices

Provides encryption and authentication for **resource-constrained IoT devices**.

## 3. Bluetooth Security

Can be used to protect data transmitted between connected devices.

## 4. Embedded Systems

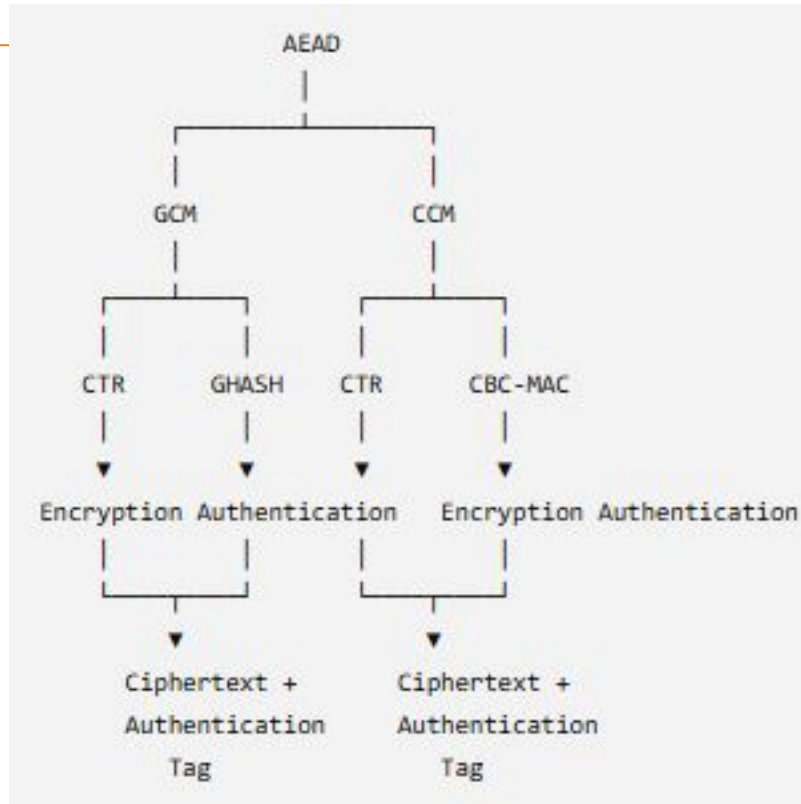
Suitable for devices with limited **memory, processing power, and battery**.

## 5. Network Security

Used with protocols such as **IPsec** to provide secure communication.

## 6. Secure Communication

Protects data from **unauthorized access and modification**.



# Question 1

---

**Why is ECB mode generally considered insecure for encrypting large data?**

- A) It requires too much padding.
- B) It reveals patterns in the plaintext because identical plaintext blocks encrypt to identical ciphertext blocks.**
- C) It uses too many keys.
- D) It does not support key expansion.

## Question 2

---

**Which security property does CBC mode achieve when used with a random IV?**

- A) Integrity protection
- B) CPA-security (Chosen Plaintext Attack security)**
- C) Forward secrecy
- D) Digital signatures



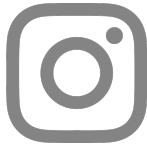
## Question 3

**Group given operation mode as block or stream cipher**  
ECB,CBC,CFB,OFB,CTR

| Block   | Stream      |
|---------|-------------|
| ECB,CBC | CFB,OFB,CTR |

# Thank you!

## Follow us



isfcr.pesu



[www.isfcr.pes.edu](http://www.isfcr.pes.edu)



ISFCR



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience





PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



Welcome to  
**PES University**  
Ring Road Campus, Bengaluru



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



# Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 18

# A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

# A Note on Security

- ➡ In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- ➡ To be clear, **you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network** without the express consent of the owner.
- ➡ In particular, **you will comply with all my instructions when doing the labs.**
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- ➡ Any violation is at **YOUR RISK!**  
**And may result in severe consequences.**

# Random Numbers

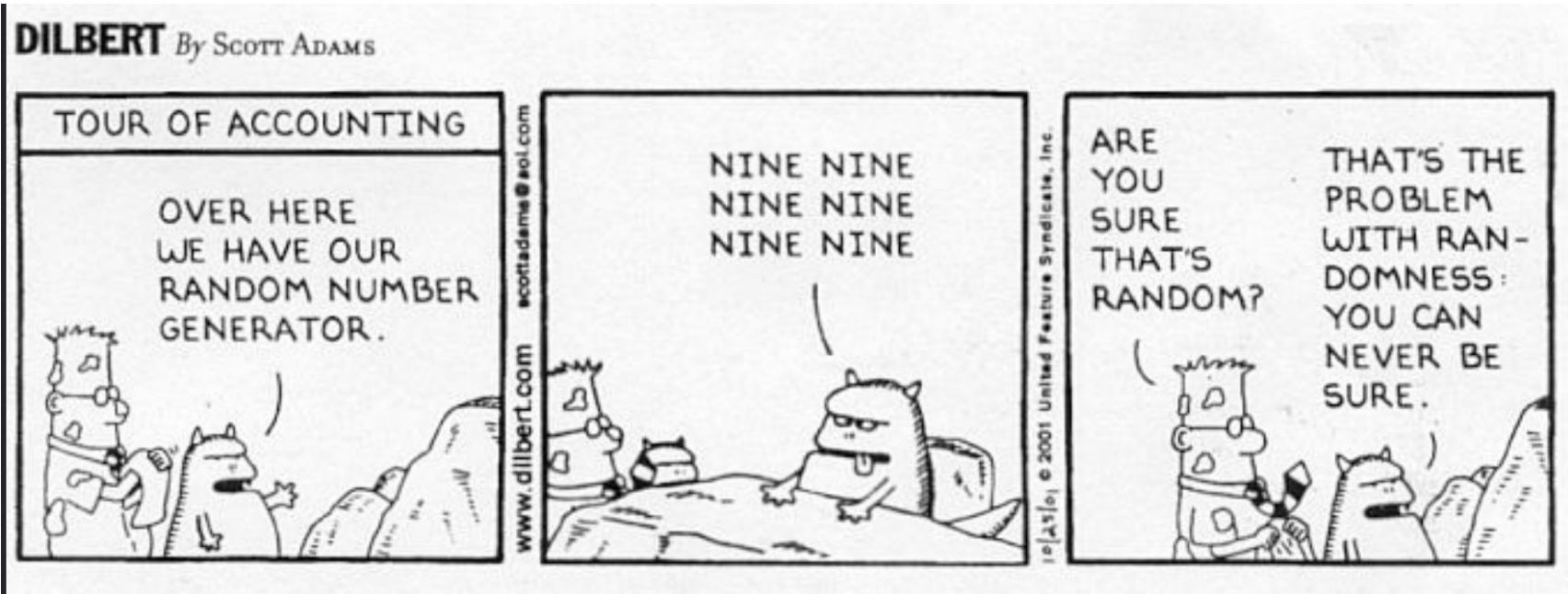
---

PRNG, TRNG

- ➡ In cryptography the **randomness** (entropy) plays very important role.
- ➡ In many algorithms, we need **random (i.e. unpredictable) numbers**.
- ➡ If these numbers are not unpredictable, the algorithms will be compromised.
- ➡ **Random numbers** usually come from a **pseudo-random number generators** (PRNG), initialized by some unpredictable initial randomness (**entropy**)



# Where are random numbers used, and are they Really Random?



# The Essence of Random Numbers in Cryptography

---

- 👉 **Unpredictability and Entropy**
- 👉 **Key Generation and Initialisation Vectors**
- 👉 **Nonces and Challenge-Response Protocols**
- 👉 **Random Salts in Hash Functions**

# Desired Properties

---

- 👉 **Unbiased** (uniform distribution): All values of whatever sample size is collected are equiprobable
- 👉 **Unpredictable** (independence): It is impossible to predict what the next output will be, given all the previous outputs, but not the internal "hidden" state.
- 👉 **Unreproducible**: Two of the same generators, given the same starting conditions, will produce different outputs.

## Pseudo-Random Number Generators (PRNGs)

- Pseudo-random number generators are algorithms that generate sequences of numbers that appear random but are ultimately deterministic. PRNGs rely on an initial seed value, and once the seed is known, the entire sequence becomes predictable.
- While suitable for some applications, PRNGs may fall short in cryptographic contexts where true randomness is paramount.

## True Random Number Generators (TRNGs)

- True random number generators, in contrast, harness physical processes to generate randomness.
- These processes may include electronic noise, radioactive decay, or atmospheric noise.
- TRNGs provide a higher level of unpredictability compared to PRNGs, making them more suitable for cryptographic applications.



Let us suppose there are  $n$  uniformly random bits.

Pseudorandomness can be defined as finding a deterministic (polynomial-time) algorithm  $f : \{0, 1\}^n \rightarrow \{0, 1\}^{l(n)}$  such that:

- the output of  $f(x)$  contains  $l(n)$  bits
- $\{0, 1\}^{l(n)}$  appears to be as “random looking” as a truly random string

# Pseudorandom Number Generators (PRNG)

- ➡ **Pseudo Random Number Generator (PRNG)** refers to an algorithm that uses mathematical formulas to produce sequences of random numbers.
- ➡ PRNGs generate a sequence of numbers approximating the properties of random numbers. A PRNG starts from an arbitrary starting state using a **seed state**.
- ➡ A PRNG is an algorithm that takes a seed as input and returns a longer string such that no one can easily say if it was calculated or not. The function computed by the algorithm is called **G**.

# A quick question!

---

Can a stream cipher have perfect secrecy?

- ☐ Yes, if the PRG is really “secure”
- ☐ No, there are no ciphers with perfect secrecy
- ☐ Yes, every cipher has perfect secrecy
- ☐ No, since the key is shorter than the message





Wide application in several field

- Generation of primes
- Cryptography: Most cryptographic applications require random numbers, for example:
  - key generation
  - nonces
  - salts in certain signature schemes



- ➡ Randomness is essential for symmetric-key encryption
  - A random key
  - A random IV/nonce
  - Universally unique identifiers (we'll see this shortly)
  - We'll see more applications later
- ➡ If an attacker can predict a random number, things can catastrophically fail
- ➡ How do we securely generate random numbers?

# True Randomness



To generate truly random numbers, we need a physical source of entropy

- An unpredictable circuit on a CPU
- Human activity measured at very fine time scales (e.g. the microsecond you pressed a key)



However, true randomness is expensive and slow



- ☞ True randomness is expensive
- ☞ **Pseudorandom number generator (PRNGs):** An algorithm that uses a little bit of true randomness to generate a lot of random-looking output
  - Also called **deterministic random bit generators (DRBGs)**
- ☞ Usage
  - Generate some expensive true randomness (e.g. noisy circuit on your CPU)
  - Use the true randomness as input to the PRNG
  - Generate random-looking numbers quickly and cheaply with the PRNG
- ☞ PRNGs are deterministic: Output is generated according to a set algorithm
  - However, for an attacker who can't see the internal state, the output is *computationally indistinguishable* from true randomness

- A PRNG has three functions:
  - PRNG.Seed(randomness): Initializes the internal state using some random bits
    - Input: Some truly random bits
  - PRNG.Reseed(randomness): Updates the internal state using the existing state and the random bits
    - Input: More truly random bits
  - PRNG.Generate( $n$ ): Generate  $n$  pseudorandom bits
    - Input: A number  $n$
    - Output:  $n$  pseudorandom bits
    - Updates the internal state as needed
- Properties
  - **Correctness**: Deterministic
  - **Efficiency**: Efficient to generate pseudorandom bits
  - **Security**: Indistinguishability from random
  - **Additional security**: Rollback resistance

- True randomness requires sampling a physical process
  - Slow and expensive (low entropy)
- PRNG: An algorithm that uses a little bit of true randomness to generate a lot of random-looking output
  - Seed(entropy): Initialize internal state
  - Reseed(entropy): Add additional entropy to the internal state
  - Generate(n): Generate n bits of pseudorandom output
  - Security: Computationally indistinguishable from truly random bits
- CTR-DRBG: Use a block cipher in CTR mode to generate pseudorandom bits
- HMAC-DRBG: Use repeated applications of HMAC to generate pseudorandom bits
- Application: UUIDs

# TRNG – True Random Number Generator

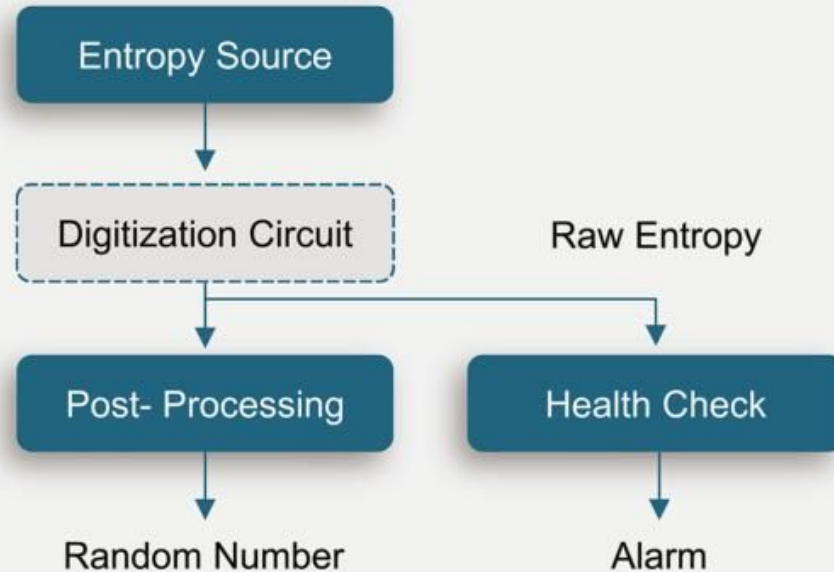
---

-  **TRNG** obtains randomness from a **physical source**.
-  The source is based on unpredictable physical behavior.
-  Examples include **electronic noise and thermal noise**.
-  The physical source provides **entropy**.
-  TRNGs are also described in NIST terminology through **non-deterministic random bit generation / entropy sources**.
-  The quality of the entropy source must be evaluated and tested.

# TRNG – True Random Number Generator

TRNG obtains unpredictable information from the physical world.

## Generic Structure of a TRNG



## Feature

👉 **Full Name**

### PRNG

- Pseudo-Random Number Generator

👉 **Source**

- Algorithm

👉 **Deterministic**

- Yes

👉 **Seed**

- Required

👉 **Speed**

- Generally fast

👉 **Reproducibility**

- Same seed → same sequence

👉 **Main Purpose**

- Expand random seed

👉 **Cryptographic Use**

- Generate random bits efficiently

### ● TRNG

- True Random Number Generator

- Physical phenomenon

- No

- Physical entropy source

- Depends on hardware

- Not reproducible

- Provide entropy

- Supply unpredictable entropy

NIST describes entropy sources as inputs that can be combined with deterministic random-bit-generator mechanisms to construct random-bit generators.



# Entropy and Seed

---



Entropy represents the amount of unpredictable information available from a source.



A good entropy source provides information that is difficult for an attacker to predict.



The collected entropy can be used to create a seed.



The seed initializes a cryptographic PRNG/DRBG(Deterministic Random Bit Generator).

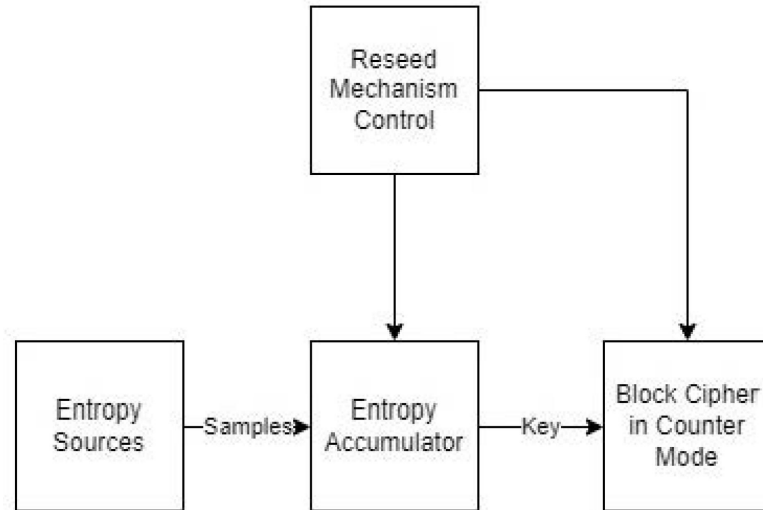


A strong seed is essential for generating unpredictable cryptographic values.

## Important Point

**A 128-bit output does not automatically mean 128 bits of security.**

If a 128-bit key is generated from only an **8-bit seed**, an attacker may only need to test **256 possible seeds**. RFC 4086 gives this type of example to illustrate why the amount of unpredictability matters.

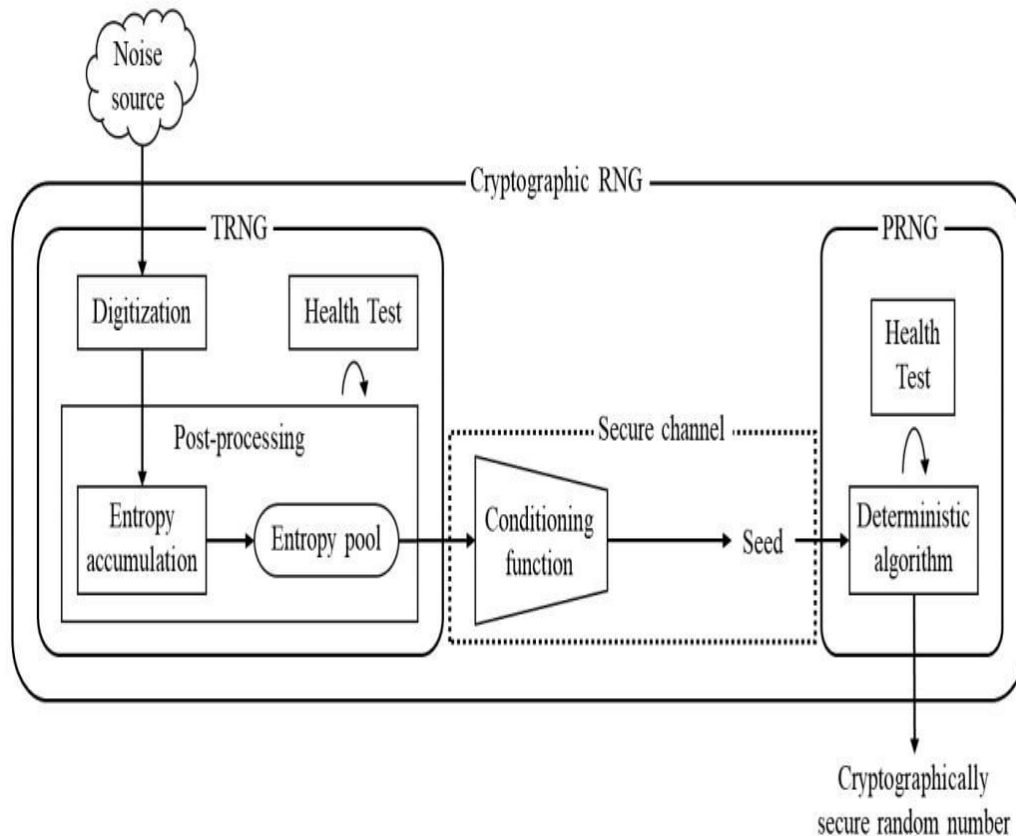
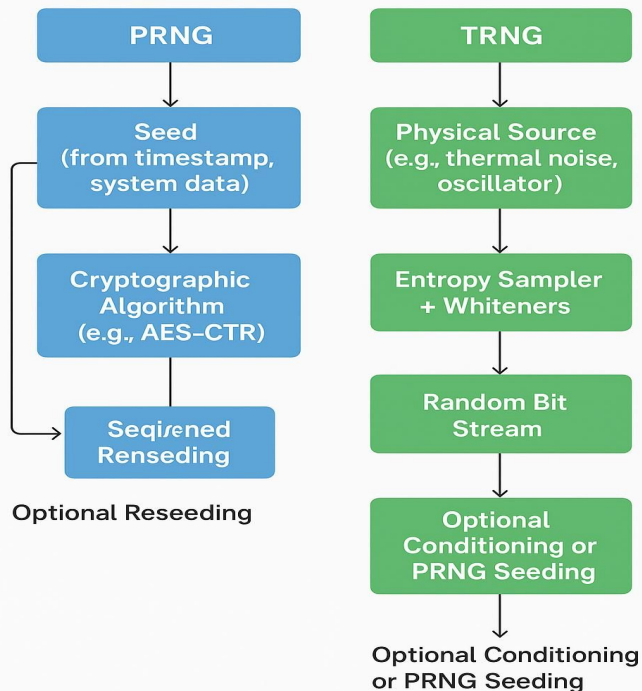


# Role of Randomness in Key Generation

- ➡ Cryptographic keys must be **secret and unpredictable**.
- ➡ Randomness provides the unpredictable material needed to generate keys.
- ➡ A physical entropy source can provide the initial entropy.
- ➡ A cryptographic PRNG/DRBG can expand that entropy efficiently.
- ➡ The generated random bits can then be used as a **cryptographic key**.
- ➡ The seed and entropy input must be protected appropriately when used for key generation.

# Role of Randomness in Key Generation

## How PRNGs and TRNGs Generate Randomness



# Why Weak Randomness is Dangerous

- ➡ A strong encryption algorithm cannot compensate for a **predictable key**.
- ➡ Attackers may try to reproduce the conditions used to generate the key.
- ➡ A small seed creates a smaller search space.
- ➡ Statistical tests alone do not prove that a sequence is **unpredictable**.
- ➡ Predictable randomness can reduce the effective security of a cryptographic key.
- ➡ Therefore, cryptographic systems require **high-quality entropy and secure random-bit generation**

# Why Weak Randomness is Dangerous

Strong Algorithm + Weak Randomness = Weak Security

AES-128

+

Weak 8-bit Seed



Only 256 possible seeds



Attacker tests the seeds



Key may be recovered

# TRNG + PRNG Working Together

---

-  TRNG/entropy source provides **unpredictable entropy**.
-  Entropy is used to create a strong **seed**.
-  PRNG/DRBG expands the seed into many random bits.
-  Those bits can be used for cryptographic keys and other security values.
-  This approach combines **unpredictability** with **efficient generation**.
-  NIST's random-bit-generation framework explicitly combines entropy sources with DRBG mechanisms.

# Question 1

---

**A PRNG (Pseudo-Random Number Generator) is:**

- a) Completely unpredictable
- b) Deterministic and predictable if the seed is known**
- c) Based on physical random processes
- d) Impossible to reproduce the sequence



## Question 2

---

The randomness in a PRNG comes from:

- a) Natural physical events
- b) Algorithmic computations based on a seed**
- c) Atmospheric conditions
- d) Radioactive decay

## Question 3

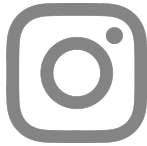
---

**TRNGs (True Random Number Generators) differ from PRNGs in that they:**

- a) Are slower but based on physical phenomena**
- b) Are faster and algorithmic
- c) Always produce the same output for a given seed
- d) Cannot be used in cryptography

# Thank you!

## Follow us



isfcr.pesu



[www.isfcr.pes.edu](http://www.isfcr.pes.edu)



ISFCR



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience





PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



Welcome to  
**PES University**  
Ring Road Campus, Bengaluru



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience



# Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 19

# A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

- ➡ One of the most significant recent advances in cryptanalysis
- ➡ Powerful method to analyse block ciphers
- ➡ Publicly discovered and disclosed in 1980s by Eli Biham and Adi Shamir
- ➡ However, it was known to the IBM DES team when creating DES, and the S-boxes are specifically chosen to resist this attack

- ➡ It requires being able to perform a Chosen Plaintext Attack.
- ➡ The basic idea is to observe the difference (XOR) between two plaintexts before encryption, and then observe the difference between the ciphertexts after encryption.
- ➡ Maps these patterns using a Difference Distribution Table (DDT), which records the probability of input differences yielding specific output differences across the S-Boxes..
- ➡ In a perfect cipher, all differentials have equal probability. If any differential has a skewed probability, it is possible to recover the key.

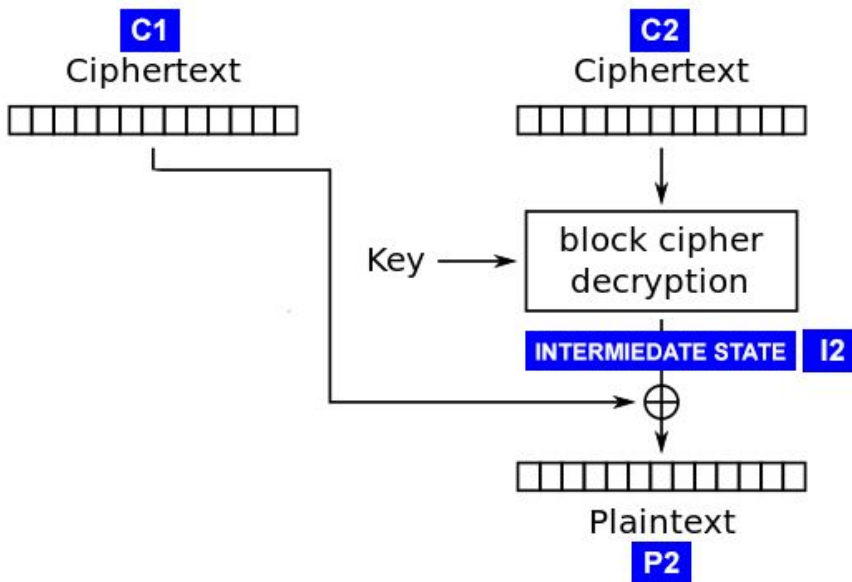


- ➡ It was developed by Matsui in the early 1990s.
- ➡ The basic idea is to consider linear relationships between the input and output that hold with higher probability than would be expected for a random function.
- ➡ It does not require Chosen Plaintext, instead Known Plaintexts is sufficient.
- ➡ DES can be broken with just  $2^{43}$  known plaintext/ciphertext pairs.

- ➡ It is a type of Adaptive Chosen Ciphertext Attack (IND-CCA2) on CBC mode, which uses padding validation to decrypt a ciphertext.
- ➡ The padding technique must be known, and invalid padding must return an error.
- ➡ A commonly used pad is PKCS7, which pads 0x01 for 1 bytes, 0x02 0x02 for two bytes, ..., and 16\*[0x16] for 16 bytes of padding.

# Padding Oracle Attack

- ➡ Replace C1 with dummy ciphertext C1'
- ➡ Randomize C1'[0:15], try all 256 possible values for C1[15].
- ➡ If P2'[15] is not 0x01, then we get invalid padding, For some value of C1[15], we get valid padding.
- ➡ We can find  $I2[15] = C1'[15] \oplus P2'[15]$
- ➡ Since we know the actual C1[15], we can find  $P2[15] = C1[15] \oplus I2[15]$
- ➡ We can repeat this to find the entire plaintext for C2.



- ➡ Software and hardware implementations leak sensitive information in the form of power consumption, time required for execution, electromagnetic emission, etc.
- ➡ Simple power analysis (SPA), differential power analysis (DPA), and correlation power analysis (CPA) are three fundamental techniques of power-based side channel attacks.
- ➡ SPA relies on direct visual inspection of power traces when leakage is immediately evident, whereas DPA and CPA employ advanced statistical methods across numerous power measurements to uncover hidden signals amidst hardware noise.

- ➡ Software and hardware implementations leak sensitive information in the form of power consumption, time required for execution, electromagnetic emission, etc.
- ➡ Simple power analysis (SPA), differential power analysis (DPA), and correlation power analysis (CPA) are three fundamental techniques of power-based side channel attacks.
- ➡ SPA relies on direct visual inspection of power traces when leakage is immediately evident, whereas DPA and CPA employ advanced statistical methods across numerous power measurements to uncover hidden signals amidst hardware noise.

- ➡ The total number of 1's being processed is known as the Hamming Weight. An IC consumes more power for each bit it flips from 0 to 1.
- ➡ AES operations XOR plaintext with a key before a non-linear S-Box, causing different key guesses to produce distinct predicted bit transitions
- ➡ Oscilloscopes can record voltage fluctuations as power traces, allowing Correlation Power Analysis (CPA) to isolate clock cycles and match real physical leakage, recovering the secret key

- ➡ <http://theamazingking.com/crypto-diff.php>
- ➡ <https://robertheaton.com/2013/07/29/padding-oracle-attack/>
- ➡ [https://github.com/newaetech/chipwhisperer-jupyter/blob/main/courses/sca101/Lab%203\\_2%20-%20Recovering%20Data%20from%20a%20Single%20Bit.ipynb](https://github.com/newaetech/chipwhisperer-jupyter/blob/main/courses/sca101/Lab%203_2%20-%20Recovering%20Data%20from%20a%20Single%20Bit.ipynb)

# A Note on Security

- ☞ In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- ☞ To be clear, **you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network** without the express consent of the owner.
- ☞ In particular, **you will comply with all my instructions when doing the labs.**
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- ☞ Any violation is at **YOUR RISK!**  
**And may result in severe consequences.**



# Thank you!

## Follow us



isfcr.pesu



[www.isfcr.pes.edu](http://www.isfcr.pes.edu)



ISFCR



PESU Center for  
Information Security,  
Forensics and  
Cyber Resilience

